GraphQLite

GraphQLite is a SQLite extension that brings graph database capabilities to SQLite using the Cypher query language. Load it into any SQLite connection and immediately start creating nodes, traversing relationships, running graph algorithms, and expressing complex graph patterns — all without a separate database server, network configuration, or migration scripts. Your graph lives in a single `.db` file alongside the rest of your application data.

Quick Example

Python

```python
from graphqlite import Graph

g = Graph(":memory:")

# Add nodes
g.upsert_node("alice", {"name": "Alice", "age": 30}, label="Person")
g.upsert_node("bob",   {"name": "Bob",   "age": 25}, label="Person")
g.upsert_node("carol", {"name": "Carol", "age": 35}, label="Person")

# Add relationships
g.upsert_edge("alice", "bob",   {"since": 2020}, rel_type="KNOWS")
g.upsert_edge("alice", "carol", {"since": 2018}, rel_type="KNOWS")

# Query with Cypher (parameterized)
results = g.connection.cypher(
    "MATCH (a:Person {name: $name})-[:KNOWS]->(friend) RETURN friend.name,
friend.age",
    {"name": "Alice"}
)
for row in results:
    print(f"{row['friend.name']} — age {row['friend.age']}")
# Bob — age 25
# Carol — age 35

# Run a graph algorithm
for r in sorted(g.pagerank(), key=lambda x: x["score"], reverse=True):
    print(f"{r['user_id']}: {r['score']:.4f}")
```

SQL (sqlite3 CLI)

```
.load build/graphqlite

SELECT cypher('CREATE (a:Person {name: "Alice", age: 30})');
SELECT cypher('CREATE (b:Person {name: "Bob",   age: 25})');
SELECT cypher('
    MATCH (a:Person {name: "Alice"}), (b:Person {name: "Bob"})
    CREATE (a)-[:KNOWS {since: 2020}]->(b)
');

-- Query
SELECT cypher('MATCH (a:Person)-[:KNOWS]->(b) RETURN a.name, b.name');

-- Run PageRank
SELECT
    json_extract(value, '$.user_id') AS person,
    printf('%.4f', json_extract(value, '$.score')) AS score
FROM json_each(cypher('RETURN pageRank(0.85, 20)'))
ORDER BY score DESC;
```

Feature Highlights

Feature	Details
Cypher query language	MATCH, CREATE, MERGE, SET, DELETE, WITH, UNWIND, FOREACH, UNION, LOAD CSV, OPTIONAL MATCH, variable-length paths, pattern predicates, and more
15+ graph algorithms	PageRank, degree/betweenness/closeness/eigenvector centrality, label propagation, Louvain, Dijkstra, A*, APSP, BFS, DFS, WCC, SCC, node similarity, KNN, triangle count
Three interfaces	Python (`pip install graphqlite`), Rust (`graphqlite` crate), and raw SQL via the `cypher()` function
Zero dependencies	Only requires SQLite — no server, no daemon, no Docker
Embedded operation	Graphs live in `.db` files; no network, no port, no configuration
Typed property storage	EAV model with separate tables for text, integer, real, boolean, and JSON properties
Parameterized queries	First-class support for `$param` substitution — safe by design
Transactions	Full SQLite transaction support; reads and writes are ACID

How This Documentation Is Organized

This documentation follows the Diátaxis framework:

- **Tutorials** — Step-by-step lessons that build something real. Start here if you are new to GraphQLite.
    - Getting Started (Python)
    - Getting Started (SQL)
    - Query Patterns (SQL)
    - Building a Knowledge Graph
    - Graph Analytics
    - Graph Algorithms (SQL)
    - Building a GraphRAG System
- **How-to Guides** — Practical guides for specific tasks such as installation, multi-graph management, parameterized queries, and using GraphQLite alongside other SQLite extensions.
- **Reference** — Complete technical descriptions of supported Cypher syntax, built-in functions and operators, all 15+ graph algorithms, and the Python and Rust APIs.
- **Explanation** — Background reading on architecture, the storage model, query dispatch, and performance characteristics.

Version and License

Current version: **0.4.4** — MIT License

Source code and issue tracker: https://github.com/colliery-io/graphqlite

Getting Started with Python

This tutorial walks you through installing GraphQLite and building a small social network graph from scratch. By the end you will know how to create nodes and relationships, query the graph with Cypher, explore the graph using built-in API methods, run a graph algorithm, and persist your work to a file.

What You Will Learn

- Install GraphQLite for Python
- Create nodes and relationships using the `Graph` class
- Inspect graph statistics and explore connections
- Run Cypher queries with parameters
- Compute PageRank to find influential nodes
- Save the graph to a file and reopen it

Prerequisites

- Python 3.8 or later
- `pip` package manager

Step 1: Install GraphQLite

```
pip install graphqlite
```

Verify the installation:

```
import graphqlite
print(graphqlite.__version__)  # 0.4.4
```

Step 2: Create an In-Memory Graph

```
from graphqlite import Graph

# ':memory:' creates a temporary in-memory database
g = Graph(":memory:")
print(g.stats())
# {'nodes': 0, 'edges': 0}
```

The `Graph` class is the high-level API. It manages the SQLite connection, loads the extension, and initialises the schema for you.

Step 3: Add Person Nodes

Add three people. Each node has a unique string ID, a dictionary of properties, and an optional label.

```
g.upsert_node("alice", {"name": "Alice", "age": 30, "city": "London"},
label="Person")
g.upsert_node("bob",   {"name": "Bob",   "age": 25, "city": "Paris"},
label="Person")
g.upsert_node("carol", {"name": "Carol", "age": 35, "city": "London"},
label="Person")

print(g.stats())
# {'nodes': 3, 'edges': 0}
```

`upsert_node` creates the node if the ID is new, or updates its properties if it already exists (merge semantics). This makes it safe to call repeatedly.

Verify a node was stored:

```
node = g.get_node("alice")
print(node)
# {'id': 'alice', 'label': 'Person', 'properties': {'name': 'Alice', 'age': 30,
'city': 'London'}}
```

Step 4: Add KNOWS Relationships

Connect the people:

```
g.upsert_edge("alice", "bob",   {"since": 2020, "strength": "close"},
rel_type="KNOWS")
g.upsert_edge("alice", "carol", {"since": 2018, "strength": "close"},
rel_type="KNOWS")
g.upsert_edge("bob",   "carol", {"since": 2021, "strength": "casual"},
rel_type="KNOWS")

print(g.stats())
# {'nodes': 3, 'edges': 3}
```

Relationships are directed. `alice -> bob` is not the same as `bob -> alice`.

Step 5: Inspect the Graph

Use built-in methods to explore the graph without writing Cypher:

```
# Check if specific connections exist
print(g.has_edge("alice", "bob"))    # True
print(g.has_edge("bob",   "alice"))  # False (directed)

# Get Alice's outgoing neighbors
neighbors = g.get_neighbors("alice")
print([n["id"] for n in neighbors])
# ['bob', 'carol']

# Get degree (total connected edges, both directions)
print(g.node_degree("alice"))  # 2
print(g.node_degree("carol"))  # 2  (one in, one out)
```

List all nodes filtered by label:

```
people = g.get_all_nodes(label="Person")
for p in people:
    print(p["id"], p["properties"]["name"])
# alice Alice
# bob Bob
# carol Carol
```

Step 6: Query with Cypher

The `query()` method runs a Cypher string and returns a list of dictionaries. For queries without user-supplied data, this is convenient:

```
results = g.query("""
    MATCH (a:Person)-[:KNOWS]->(b:Person)
    RETURN a.name AS from, b.name AS to, a.city AS city
    ORDER BY a.name
""")

for row in results:
    print(f"{row['from']} ({row['city']}) knows {row['to']}")
# Alice (London) knows Bob
# Alice (London) knows Carol
# Bob (Paris) knows Carol
```

Step 7: Parameterized Queries

When any part of the query comes from user input, use parameterized queries. Access the underlying `Connection` object via `g.connection`:

```
# Find everyone in a specific city — city name comes from user input
city = "London"
results = g.connection.cypher(
    "MATCH (p:Person {city: $city}) RETURN p.name AS name, p.age AS age ORDER
BY p.age",
    {"city": city}
)

for row in results:
    print(f"{row['name']}, age {row['age']}")
# Alice, age 30
# Carol, age 35
```

Parameters are passed as a Python dictionary and serialised to JSON internally. This protects against injection and handles special characters cleanly. See the parameterized queries guide for more detail.

Multi-hop traversal using parameters:

```
# Who does a given person know transitively (up to 2 hops)?
results = g.connection.cypher(
    """
    MATCH (start:Person {name: $name})-[:KNOWS*1..2]->(other:Person)
    RETURN DISTINCT other.name AS name
    """,
    {"name": "Alice"}
)

print([r["name"] for r in results])
# ['Bob', 'Carol']
```

Step 8: Run PageRank

Load the graph into the algorithm cache, then run PageRank. PageRank scores nodes by how many high-scoring nodes point to them.

```
# Load the graph cache (required before algorithms)
g.connection.cypher("RETURN gql_load_graph()")

results = g.pagerank(damping=0.85, iterations=20)

print("PageRank scores:")
for r in sorted(results, key=lambda x: x["score"], reverse=True):
    print(f"  {r['user_id']}: {r['score']:.4f}")
# PageRank scores:
#   carol: 0.2282
#   bob:   0.1847
#   alice: 0.1471
```

Carol scores highest because two people point to her. See Graph Analytics for a full walkthrough of all 15+ algorithms.

Step 9: Persist to a File

Switching from `:memory:` to a file path makes the graph persistent:

```
# Save to a file
g_file = Graph("social.db")

g_file.upsert_node("alice", {"name": "Alice", "age": 30, "city": "London"},
label="Person")
g_file.upsert_node("bob",   {"name": "Bob",   "age": 25, "city": "Paris"},
label="Person")
g_file.upsert_node("carol", {"name": "Carol", "age": 35, "city": "London"},
label="Person")
g_file.upsert_edge("alice", "bob",   {"since": 2020}, rel_type="KNOWS")
g_file.upsert_edge("alice", "carol", {"since": 2018}, rel_type="KNOWS")
g_file.upsert_edge("bob",   "carol", {"since": 2021}, rel_type="KNOWS")

print(f"Saved: {g_file.stats()}")
# Saved: {'nodes': 3, 'edges': 3}
```

Reopen it later:

```
g_reopen = Graph("social.db")
print(g_reopen.stats())
# {'nodes': 3, 'edges': 3}

node = g_reopen.get_node("alice")
print(node["properties"]["name"])
# Alice
```

The database is a standard SQLite file. You can inspect it with any SQLite tool.

Summary

In this tutorial you:

1. Installed GraphQLite with `pip install graphqlite`
2. Created an in-memory graph using the `Graph` class
3. Added Person nodes with `upsert_node()`
4. Added KNOWS relationships with `upsert_edge()`
5. Explored the graph with `get_neighbors()`, `node_degree()`, and `stats()`
6. Queried with Cypher via `g.query()` and `g.connection.cypher()`
7. Used parameterized queries to safely handle user input
8. Ran PageRank with `g.pagerank()`
9. Persisted the graph to a `.db` file

Next Steps

- Building a Knowledge Graph — A more complex domain with multiple node and relationship types
- Graph Analytics — All 15+ algorithms with worked examples
- Query Patterns (SQL) — Advanced Cypher patterns: UNWIND, WITH pipelines, CASE, UNION
- Python API Reference — Complete method documentation for `Graph`, `Connection`, and `GraphManager`
- Parameterized Queries Guide — Best practices for safe query construction

Getting Started with SQL

This tutorial shows how to use GraphQLite directly from the SQLite command-line interface. No Python or Rust required — just `sqlite3` and the compiled extension.

What You Will Learn

- Build the extension from source
- Load it into the SQLite CLI
- Create nodes and relationships with Cypher
- Query patterns with MATCH and WHERE
- Use parameterized queries
- Run a graph algorithm and read the results

Prerequisites

- SQLite 3.x CLI (`sqlite3 --version`)
- A C compiler, Bison, and Flex (for building from source), **or** a pre-built binary from the Python wheel

Step 1: Get the Extension

Option A — Build from source

```
# Clone the repository
git clone https://github.com/colliery-io/graphqlite.git
cd graphqlite

# macOS
brew install bison flex sqlite
export PATH="$(brew --prefix bison)/bin:$PATH"
make extension RELEASE=1

# Linux (Debian/Ubuntu)
sudo apt-get install build-essential bison flex libsqlite3-dev
make extension RELEASE=1
```

The compiled extension lands in:

- `build/graphqlite.dylib` (macOS)
- `build/graphqlite.so` (Linux)
- `build/graphqlite.dll` (Windows)

Option B — Extract from the Python package

```
pip install graphqlite
python -c "import graphqlite; print(graphqlite.loadable_path())"
# /path/to/site-packages/graphqlite/graphqlite.dylib
```

Use that path anywhere this tutorial says `build/graphqlite`.

Step 2: Open SQLite and Load the Extension

```
sqlite3 social.db
```

Inside the SQLite prompt:

```
-- Load the extension (adjust extension for your platform)
.load build/graphqlite

-- Confirm it loaded
SELECT cypher('RETURN 1 + 1 AS result');
-- [{"result":2}]
```

Enable column headers for readable output:

```
.mode column
.headers on
```

Step 3: Create Nodes

Create a small social network. Each `CREATE` call returns an empty JSON array `[]` — that is normal for write operations.

```
SELECT cypher('CREATE (a:Person {name: "Alice", age: 30, city: "London"})');
SELECT cypher('CREATE (b:Person {name: "Bob",   age: 25, city: "Paris"})');
SELECT cypher('CREATE (c:Person {name: "Carol", age: 35, city: "London"})');
SELECT cypher('CREATE (d:Person {name: "Dave",  age: 28, city: "Berlin"})');
```

Verify the nodes exist:

```
SELECT cypher('MATCH (p:Person) RETURN p.name, p.age, p.city ORDER BY p.name');
```

Output:

```
[{"p.name":"Alice","p.age":30,"p.city":"London"},
 {"p.name":"Bob","p.age":25,"p.city":"Paris"},
 {"p.name":"Carol","p.age":35,"p.city":"London"},
 {"p.name":"Dave","p.age":28,"p.city":"Berlin"}]
```

Use `json_each()` to get one row per result:

```
SELECT
    json_extract(value, '$.p.name') AS name,
    json_extract(value, '$.p.age')  AS age,
    json_extract(value, '$.p.city') AS city
FROM json_each(cypher('MATCH (p:Person) RETURN p.name, p.age, p.city ORDER BY
p.name'));
```

Output:

```
name   age  city
-----  ---  ------
Alice  30   London
Bob    25   Paris
Carol  35   London
Dave   28   Berlin
```

Step 4: Create Relationships

```
-- Alice knows Bob
SELECT cypher('
    MATCH (a:Person {name: "Alice"}), (b:Person {name: "Bob"})
    CREATE (a)-[:KNOWS {since: 2020}]->(b)
');

-- Alice knows Carol
SELECT cypher('
    MATCH (a:Person {name: "Alice"}), (c:Person {name: "Carol"})
    CREATE (a)-[:KNOWS {since: 2018}]->(c)
');

-- Bob knows Dave
SELECT cypher('
    MATCH (b:Person {name: "Bob"}), (d:Person {name: "Dave"})
    CREATE (b)-[:KNOWS {since: 2022}]->(d)
');

-- Carol knows Dave
SELECT cypher('
    MATCH (c:Person {name: "Carol"}), (d:Person {name: "Dave"})
    CREATE (c)-[:KNOWS {since: 2021}]->(d)
');
```

Step 5: Query Patterns with MATCH and WHERE

Who does Alice know?

```
SELECT
    json_extract(value, '$.friend.name') AS friend,
    json_extract(value, '$.r.since')     AS since
FROM json_each(cypher('
    MATCH (a:Person {name: "Alice"})-[r:KNOWS]->(friend)
    RETURN friend, r
'));
```

Output:

```
friend  since
------  -----
Bob     2020
Carol   2018
```

People in a specific city

```
SELECT
    json_extract(value, '$.name') AS name
FROM json_each(cypher('
    MATCH (p:Person)
    WHERE p.city = "London"
    RETURN p.name AS name
'));
```

Output:

```
name
-----
Alice
Carol
```

Friends of friends (two hops)

Who can Alice reach through two KNOWS relationships?

```
SELECT
    json_extract(value, '$.fof') AS friend_of_friend
FROM json_each(cypher('
    MATCH (a:Person {name: "Alice"})-[:KNOWS]->()-[:KNOWS]->(fof)
    RETURN DISTINCT fof.name AS fof
'));
```

Output:

```
friend_of_friend
----------------
Dave
```

Filter with WHERE and aggregation

People who know more than one person:

```
SELECT
    json_extract(value, '$.name')  AS person,
    json_extract(value, '$.count') AS knows_count
FROM json_each(cypher('
    MATCH (a:Person)-[:KNOWS]->(b)
    WITH a.name AS name, count(b) AS count
    WHERE count > 1
    RETURN name, count
    ORDER BY count DESC
'));
```

Output:

```
person  knows_count
------  -----------
Alice   2
```

Step 6: Use Parameterized Queries

For any value that comes from outside the query — user input, a variable, application data — use the `$param` syntax with a JSON parameters string as the second argument to `cypher()`:

```
-- Find a person by name using a parameter
SELECT cypher(
    'MATCH (p:Person {name: $name}) RETURN p.name, p.age, p.city',
    '{"name": "Carol"}'
);
-- [{"p.name":"Carol","p.age":35,"p.city":"London"}]
```

Multiple parameters:

```
SELECT cypher(
    'MATCH (p:Person) WHERE p.age >= $min_age AND p.city = $city RETURN p.name,
p.age',
    '{"min_age": 28, "city": "London"}'
);
-- [{"p.name":"Alice","p.age":30},{"p.name":"Carol","p.age":35}]
```

Parameters protect against injection and correctly handle special characters in string values.

Step 7: Run PageRank

GraphQLite includes 15+ graph algorithms. Load the graph cache first, then call the algorithm function inside a `RETURN` clause.

```
-- Load the graph into the algorithm cache
SELECT cypher('RETURN gql_load_graph()');
```

Run PageRank and display the results as a table:

```
SELECT
    json_extract(value, '$.user_id')                    AS person,
    printf('%.4f', json_extract(value, '$.score'))      AS pagerank
FROM json_each(cypher('RETURN pageRank(0.85, 20)'))
ORDER BY json_extract(value, '$.score') DESC;
```

Output:

```
person  pagerank
------  --------
carol   0.2000
dave    0.1800
bob     0.1600
alice   0.1200
```

Dave and Carol score highest because they have multiple incoming paths from well-connected nodes.

Complete Script

Save the following as `social.sql` and run it with `sqlite3 < social.sql`:

```
.load build/graphqlite
.mode column
.headers on

-- Nodes
SELECT cypher('CREATE (a:Person {name: "Alice", age: 30, city: "London"})');
SELECT cypher('CREATE (b:Person {name: "Bob",   age: 25, city: "Paris"})');
SELECT cypher('CREATE (c:Person {name: "Carol", age: 35, city: "London"})');
SELECT cypher('CREATE (d:Person {name: "Dave",  age: 28, city: "Berlin"})');

-- Relationships
SELECT cypher('MATCH (a:Person {name: "Alice"}), (b:Person {name: "Bob"})
CREATE (a)-[:KNOWS {since: 2020}]->(b)');
SELECT cypher('MATCH (a:Person {name: "Alice"}), (c:Person {name: "Carol"})
CREATE (a)-[:KNOWS {since: 2018}]->(c)');
SELECT cypher('MATCH (b:Person {name: "Bob"}),   (d:Person {name: "Dave"})
CREATE (b)-[:KNOWS {since: 2022}]->(d)');
SELECT cypher('MATCH (c:Person {name: "Carol"}), (d:Person {name: "Dave"})
CREATE (c)-[:KNOWS {since: 2021}]->(d)');

SELECT '--- All people ---';
SELECT json_extract(value, '$.p.name') AS name,
       json_extract(value, '$.p.age')  AS age
FROM json_each(cypher('MATCH (p:Person) RETURN p.name, p.age ORDER BY
p.name'));

SELECT '--- Who Alice knows ---';
SELECT json_extract(value, '$.friend') AS friend
FROM json_each(cypher('MATCH (:Person {name: "Alice"})-[:KNOWS]->(f) RETURN
f.name AS friend'));

SELECT '--- PageRank ---';
SELECT cypher('RETURN gql_load_graph()');
SELECT json_extract(value, '$.user_id')               AS person,
       printf('%.4f', json_extract(value, '$.score')) AS score
FROM json_each(cypher('RETURN pageRank(0.85, 20)'))
ORDER BY json_extract(value, '$.score') DESC;
```

Next Steps

- Query Patterns (SQL) — Variable-length paths, OPTIONAL MATCH, WITH, UNWIND, CASE, UNION
- Graph Algorithms (SQL) — All 15+ algorithms with SQL extraction patterns
- SQL Interface Reference — Complete `cypher()` function documentation and schema tables

Query Patterns in SQL

This tutorial covers intermediate and advanced Cypher patterns using GraphQLite from the SQLite CLI. The examples use a movie database: movies, actors, directors, and the relationships between them.

Setup: Build the Movie Database

Save this block as `movie_setup.sql` and run it with `sqlite3 movies.db < movie_setup.sql`, or paste it into an interactive session after `.load build/graphqlite`.

```
.load build/graphqlite
.mode column
.headers on

-- Movies
SELECT cypher('CREATE (m:Movie {title: "Inception",       year: 2010, rating:
8.8})');
SELECT cypher('CREATE (m:Movie {title: "The Dark Knight", year: 2008, rating:
9.0})');
SELECT cypher('CREATE (m:Movie {title: "Interstellar",    year: 2014, rating:
8.6})');
SELECT cypher('CREATE (m:Movie {title: "Memento",         year: 2000, rating:
8.4})');
SELECT cypher('CREATE (m:Movie {title: "Dunkirk",         year: 2017, rating:
7.9})');

-- Actors
SELECT cypher('CREATE (a:Actor {name: "Leonardo DiCaprio", born: 1974})');
SELECT cypher('CREATE (a:Actor {name: "Christian Bale",    born: 1974})');
SELECT cypher('CREATE (a:Actor {name: "Tom Hardy",         born: 1977})');
SELECT cypher('CREATE (a:Actor {name: "Cillian Murphy",    born: 1976})');
SELECT cypher('CREATE (a:Actor {name: "Ken Watanabe",      born: 1959})');
SELECT cypher('CREATE (a:Actor {name: "Guy Pearce",        born: 1967})');
SELECT cypher('CREATE (a:Actor {name: "Mark Rylance",      born: 1960})');

-- Directors
SELECT cypher('CREATE (d:Director {name: "Christopher Nolan", born: 1970})');

-- ACTED_IN
SELECT cypher('MATCH (a:Actor {name: "Leonardo DiCaprio"}), (m:Movie {title:
"Inception"})       CREATE (a)-[:ACTED_IN {role: "Cobb"}]->(m)');
SELECT cypher('MATCH (a:Actor {name: "Ken Watanabe"}),      (m:Movie {title:
"Inception"})       CREATE (a)-[:ACTED_IN {role: "Saito"}]->(m)');
SELECT cypher('MATCH (a:Actor {name: "Tom Hardy"}),         (m:Movie {title:
"Inception"})       CREATE (a)-[:ACTED_IN {role: "Eames"}]->(m)');
SELECT cypher('MATCH (a:Actor {name: "Christian Bale"}),    (m:Movie {title:
"The Dark Knight"}) CREATE (a)-[:ACTED_IN {role: "Bruce Wayne"}]->(m)');
SELECT cypher('MATCH (a:Actor {name: "Tom Hardy"}),         (m:Movie {title:
"The Dark Knight"}) CREATE (a)-[:ACTED_IN {role: "Bane"}]->(m)');
SELECT cypher('MATCH (a:Actor {name: "Cillian Murphy"}),    (m:Movie {title:
"The Dark Knight"}) CREATE (a)-[:ACTED_IN {role: "Scarecrow"}]->(m)');
SELECT cypher('MATCH (a:Actor {name: "Cillian Murphy"}),    (m:Movie {title:
"Inception"})       CREATE (a)-[:ACTED_IN {role: "Fischer"}]->(m)');
SELECT cypher('MATCH (a:Actor {name: "Guy Pearce"}),        (m:Movie {title:
"Memento"})         CREATE (a)-[:ACTED_IN {role: "Leonard"}]->(m)');
SELECT cypher('MATCH (a:Actor {name: "Cillian Murphy"}),    (m:Movie {title:
"Dunkirk"})         CREATE (a)-[:ACTED_IN {role: "Shivering Soldier"}]->(m)');
SELECT cypher('MATCH (a:Actor {name: "Tom Hardy"}),         (m:Movie {title:
"Dunkirk"})         CREATE (a)-[:ACTED_IN {role: "Farrier"}]->(m)');
SELECT cypher('MATCH (a:Actor {name: "Mark Rylance"}),      (m:Movie {title:
"Dunkirk"})         CREATE (a)-[:ACTED_IN {role: "Mr. Dawson"}]->(m)');

-- DIRECTED
SELECT cypher('MATCH (d:Director {name: "Christopher Nolan"}), (m:Movie {title:
"Inception"})       CREATE (d)-[:DIRECTED]->(m)');
SELECT cypher('MATCH (d:Director {name: "Christopher Nolan"}), (m:Movie {title:
"The Dark Knight"}) CREATE (d)-[:DIRECTED]->(m)');
```

```
SELECT cypher('MATCH (d:Director {name: "Christopher Nolan"}), (m:Movie {title:
"Interstellar"})    CREATE (d)-[:DIRECTED]->(m)');
SELECT cypher('MATCH (d:Director {name: "Christopher Nolan"}), (m:Movie {title:
"Memento"})         CREATE (d)-[:DIRECTED]->(m)');
SELECT cypher('MATCH (d:Director {name: "Christopher Nolan"}), (m:Movie {title:
"Dunkirk"})         CREATE (d)-[:DIRECTED]->(m)');
```

1. Multi-Hop Traversals

Walk across more than one relationship in a single pattern.

Two-hop: actors who appeared in movies by the same director

```
SELECT
    json_extract(value, '$.actor1') AS actor1,
    json_extract(value, '$.actor2') AS actor2,
    json_extract(value, '$.movie')  AS shared_movie
FROM json_each(cypher('
    MATCH (a1:Actor)-[:ACTED_IN]->(m:Movie)<-[:ACTED_IN]-(a2:Actor)
    WHERE a1.name < a2.name
    RETURN a1.name AS actor1, a2.name AS actor2, m.title AS movie
    ORDER BY m.title, actor1
'));
```

Output:

```
actor1           actor2              shared_movie
---------------  ------------------  ------------
Christian Bale   Cillian Murphy      The Dark Knight
Christian Bale   Tom Hardy           The Dark Knight
Cillian Murphy   Tom Hardy           The Dark Knight
...
```

Three-hop: director -> movie -> actor -> movie

Which other movies have actors from a director's film appeared in?

```
SELECT
    json_extract(value, '$.director') AS director,
    json_extract(value, '$.actor')    AS actor,
    json_extract(value, '$.other')    AS other_movie
FROM json_each(cypher('
    MATCH (d:Director)-[:DIRECTED]->(m:Movie)<-[:ACTED_IN]-(a:Actor)-
[:ACTED_IN]->(other:Movie)
    WHERE other.title <> m.title
    RETURN DISTINCT d.name AS director, a.name AS actor, other.title AS other
    ORDER BY actor
'));
```

2. Variable-Length Paths

Use `[*min..max]` to traverse a variable number of hops.

Everyone reachable from an actor within 1–3 ACTED_IN hops

```
SELECT
    json_extract(value, '$.reach') AS reachable_node
FROM json_each(cypher('
    MATCH (a:Actor {name: "Tom Hardy"})-[:ACTED_IN*1..2]->(m)
    RETURN DISTINCT m.title AS reach
'));
```

The pattern `[:ACTED_IN*1..2]` matches one or two consecutive ACTED_IN edges, so it finds movies Tom Hardy acted in directly, and then movies those movies lead to (if there were further ACTED_IN from Movie nodes).

Shortest path between two actors (Bacon-number style)

Variable-length paths with any relationship type:

```
SELECT cypher('
    MATCH path = (a:Actor {name: "Leonardo DiCaprio"})-[*1..4]-(b:Actor {name:
"Mark Rylance"})
    RETURN length(path) AS hops
    ORDER BY hops
    LIMIT 1
');
```

3. OPTIONAL MATCH

`OPTIONAL MATCH` returns `null` for missing parts of the pattern instead of dropping the row entirely. This is equivalent to a SQL LEFT JOIN.

List all movies with their director (null if not yet recorded)

```
SELECT
    json_extract(value, '$.title')    AS title,
    json_extract(value, '$.year')     AS year,
    json_extract(value, '$.director') AS director
FROM json_each(cypher('
    MATCH (m:Movie)
    OPTIONAL MATCH (d:Director)-[:DIRECTED]->(m)
    RETURN m.title AS title, m.year AS year, d.name AS director
    ORDER BY m.year
'));
```

Output:

```
title             year  director
----------------  ----  -----------------
Memento           2000  Christopher Nolan
The Dark Knight   2008  Christopher Nolan
Inception         2010  Christopher Nolan
Interstellar      2014  Christopher Nolan
Dunkirk           2017  Christopher Nolan
```

Count cast size, including movies with no recorded cast

```
SELECT
    json_extract(value, '$.title') AS title,
    json_extract(value, '$.cast')  AS cast_size
FROM json_each(cypher('
    MATCH (m:Movie)
    OPTIONAL MATCH (a:Actor)-[:ACTED_IN]->(m)
    RETURN m.title AS title, count(a) AS cast
    ORDER BY cast DESC
'));
```

Output:

```
title             cast_size
----------------  ---------
Dunkirk           3
The Dark Knight   3
Inception         3
Memento           1
Interstellar      0
```

`Interstellar` has 0 because no ACTED_IN relationships were created for it.

4. WITH Clause for Pipelining

`WITH` passes the results of one query stage as input to the next, similar to a SQL CTE. It lets you filter and reshape data mid-query.

Find actors who appeared in more than one film, then get their earliest role

```
SELECT
    json_extract(value, '$.actor')        AS actor,
    json_extract(value, '$.film_count')   AS films,
    json_extract(value, '$.first_movie')  AS first_movie
FROM json_each(cypher('
    MATCH (a:Actor)-[:ACTED_IN]->(m:Movie)
    WITH a, count(m) AS film_count, min(m.year) AS earliest_year
    WHERE film_count > 1
    MATCH (a)-[:ACTED_IN]->(first:Movie {year: earliest_year})
    RETURN a.name AS actor, film_count, first.title AS first_movie
    ORDER BY film_count DESC
'));
```

Output:

```
actor           films  first_movie
--------------  -----  ---------------
Tom Hardy       3      The Dark Knight
Cillian Murphy  3      The Dark Knight
```

Ranked movies by rating, then filter to top tier

```
SELECT
    json_extract(value, '$.title')  AS title,
    json_extract(value, '$.rating') AS rating,
    json_extract(value, '$.tier')   AS tier
FROM json_each(cypher('
    MATCH (m:Movie)
    WITH m.title AS title, m.rating AS rating
    ORDER BY rating DESC
    WITH title, rating,
         CASE
             WHEN rating >= 9.0 THEN "Masterpiece"
             WHEN rating >= 8.5 THEN "Excellent"
             ELSE "Good"
         END AS tier
    RETURN title, rating, tier
'));
```

5. UNWIND for List Processing

`UNWIND` flattens a list into individual rows — useful for batch creation and list comprehensions.

Create multiple nodes from an inline list

```
SELECT cypher('
    UNWIND ["Action", "Thriller", "Sci-Fi", "Drama"] AS genre_name
    CREATE (g:Genre {name: genre_name})
');
```

Batch-tag movies using UNWIND

```
SELECT cypher('
    UNWIND [
        {movie: "Inception",    genre: "Sci-Fi"},
        {movie: "Inception",    genre: "Action"},
        {movie: "The Dark Knight", genre: "Action"},
        {movie: "The Dark Knight", genre: "Drama"},
        {movie: "Memento",      genre: "Thriller"},
        {movie: "Dunkirk",      genre: "Drama"}
    ] AS row
    MATCH (m:Movie {title: row.movie}), (g:Genre {name: row.genre})
    CREATE (m)-[:IN_GENRE]->(g)
');
```

Expand a collected list back to rows

```
SELECT
    json_extract(value, '$.genre')  AS genre,
    json_extract(value, '$.movie')  AS movie
FROM json_each(cypher('
    MATCH (m:Movie)-[:IN_GENRE]->(g:Genre)
    WITH g.name AS genre, collect(m.title) AS movies
    UNWIND movies AS movie
    RETURN genre, movie
    ORDER BY genre, movie
'));
```

6. Aggregation

Count, sum, average, collect

```
SELECT
    json_extract(value, '$.genre')   AS genre,
    json_extract(value, '$.count')   AS film_count,
    json_extract(value, '$.avg_rating') AS avg_rating,
    json_extract(value, '$.titles')  AS titles
FROM json_each(cypher('
    MATCH (m:Movie)-[:IN_GENRE]->(g:Genre)
    RETURN g.name AS genre,
           count(m) AS count,
           round(avg(m.rating), 2) AS avg_rating,
           collect(m.title) AS titles
    ORDER BY count DESC
'));
```

Output:

```
genre    film_count  avg_rating  titles
-------  ----------  ----------  ------------------------------------
Action   2           8.9         ["Inception","The Dark Knight"]
Drama    2           8.45        ["The Dark Knight","Dunkirk"]
Sci-Fi   1           8.8         ["Inception"]
Thriller 1           8.4         ["Memento"]
```

Top-N with ORDER BY and LIMIT

```
SELECT
    json_extract(value, '$.actor')  AS actor,
    json_extract(value, '$.films')  AS film_count
FROM json_each(cypher('
    MATCH (a:Actor)-[:ACTED_IN]->(m:Movie)
    RETURN a.name AS actor, count(m) AS films
    ORDER BY films DESC
    LIMIT 3
'));
```

7. CASE Expressions

CASE works both inline and in return clauses.

Label movie quality tier

```
SELECT
    json_extract(value, '$.title')  AS title,
    json_extract(value, '$.rating') AS rating,
    json_extract(value, '$.label')  AS quality
FROM json_each(cypher('
    MATCH (m:Movie)
    RETURN m.title AS title, m.rating AS rating,
           CASE
               WHEN m.rating >= 9.0 THEN "Masterpiece"
               WHEN m.rating >= 8.5 THEN "Excellent"
               WHEN m.rating >= 8.0 THEN "Very Good"
               ELSE "Good"
           END AS label
    ORDER BY m.rating DESC
'));
```

Output:

```
title             rating  quality
----------------  ------  -----------
The Dark Knight   9.0     Masterpiece
Inception         8.8     Excellent
Interstellar      8.6     Excellent
Memento           8.4     Very Good
Dunkirk           7.9     Good
```

Conditional aggregation

```
SELECT
    json_extract(value, '$.actor')          AS actor,
    json_extract(value, '$.high_rated')     AS high_rated_count,
    json_extract(value, '$.lower_rated')    AS lower_rated_count
FROM json_each(cypher('
    MATCH (a:Actor)-[:ACTED_IN]->(m:Movie)
    RETURN a.name AS actor,
           count(CASE WHEN m.rating >= 8.5 THEN 1 END) AS high_rated,
           count(CASE WHEN m.rating < 8.5  THEN 1 END) AS lower_rated
    ORDER BY actor
'));
```

8. UNION Queries

`UNION ALL` combines result sets, keeping duplicates. `UNION` deduplicates.

List actors and directors in a unified "people" result

```
SELECT
    json_extract(value, '$.name') AS name,
    json_extract(value, '$.role') AS role
FROM json_each(cypher('
    MATCH (a:Actor)
    RETURN a.name AS name, "Actor" AS role
    UNION ALL
    MATCH (d:Director)
    RETURN d.name AS name, "Director" AS role
    ORDER BY name
'));
```

Combine movies that were either highly rated OR recent

```
SELECT
    json_extract(value, '$.title')  AS title,
    json_extract(value, '$.reason') AS reason
FROM json_each(cypher('
    MATCH (m:Movie) WHERE m.rating >= 9.0
    RETURN m.title AS title, "Top rated" AS reason
    UNION
    MATCH (m:Movie) WHERE m.year >= 2014
    RETURN m.title AS title, "Recent release" AS reason
    ORDER BY title
'));
```

9. Working with Results via json_each()

The `cypher()` function returns a JSON array. Use SQLite's `json_each()` and `json_extract()` to integrate results with the rest of your schema.

Join Cypher results with a regular SQL table

Suppose you have a standard `reviews` table tracking critic scores:

```
CREATE TABLE IF NOT EXISTS reviews (
    title TEXT PRIMARY KEY,
    critic_score REAL,
    review_count INTEGER
);

INSERT OR IGNORE INTO reviews VALUES ('Inception',       95, 290);
INSERT OR IGNORE INTO reviews VALUES ('The Dark Knight', 94, 285);
INSERT OR IGNORE INTO reviews VALUES ('Interstellar',    72, 259);
INSERT OR IGNORE INTO reviews VALUES ('Memento',         93, 180);
INSERT OR IGNORE INTO reviews VALUES ('Dunkirk',         92, 259);
```

Now join graph data with SQL data:

```
WITH movie_ratings AS (
    SELECT
        json_extract(value, '$.title')   AS title,
        json_extract(value, '$.audience') AS audience_score
    FROM json_each(cypher('
        MATCH (m:Movie)
        RETURN m.title AS title, m.rating AS audience
    '))
)
SELECT
    mr.title,
    mr.audience_score,
    r.critic_score,
    r.review_count,
    ROUND(ABS(mr.audience_score * 10 - r.critic_score), 1) AS gap
FROM movie_ratings mr
JOIN reviews r ON r.title = mr.title
ORDER BY gap DESC;
```

Output:

```
title          audience_score  critic_score  review_count  gap
-------------  --------------  ------------  ------------  ----
Interstellar   8.6             72            259           14.0
The Dark Knight 9.0            94            285           4.0
Inception      8.8             95            290           7.0
Dunkirk        7.9             92            259           13.0
Memento        8.4             93            180           9.0
```

Create a SQL view over a Cypher query

```
CREATE VIEW IF NOT EXISTS actor_filmography AS
SELECT
    json_extract(value, '$.actor')  AS actor,
    json_extract(value, '$.movie')  AS movie,
    json_extract(value, '$.year')   AS year,
    json_extract(value, '$.role')   AS role
FROM json_each(cypher('
    MATCH (a:Actor)-[r:ACTED_IN]->(m:Movie)
    RETURN a.name AS actor, m.title AS movie, m.year AS year, r.role AS role
    ORDER BY m.year
'));
```

Use it like any table:

```
SELECT * FROM actor_filmography WHERE actor = 'Tom Hardy' ORDER BY year;
```

Output:

```
actor      movie             year  role
---------  ----------------  ----  -------
Tom Hardy  The Dark Knight   2008  Bane
Tom Hardy  Inception         2010  Eames
Tom Hardy  Dunkirk           2017  Farrier
```

Next Steps

- Graph Algorithms (SQL) — Run PageRank, community detection, and path finding from SQL
- SQL Interface Reference — Full `cypher()` documentation and schema tables
- Cypher Functions Reference — All built-in string, math, list, and aggregate functions

Building a Knowledge Graph

This tutorial builds a research publication knowledge graph in Python. The domain includes researchers, academic papers, and research topics, connected by authorship, citation, and topic membership. You will model the schema, populate it with parameterized writes, query it with Cypher, apply graph algorithms to discover influential papers and research clusters, and maintain the graph over time.

What You Will Build

A knowledge graph with:

Node labels

- `Researcher` — scientists and academics
- `Paper` — published works
- `Topic` — research areas

Relationship types

- `AUTHORED` — Researcher authored Paper
- `CITES` — Paper cites Paper
- `IN_TOPIC` — Paper belongs to a Topic
- `COLLABORATES` — Researcher co-authored with Researcher (derived)

What You Will Learn

- Design a multi-type node schema
- Use parameterized queries for all writes
- Find co-authors, citation chains, and research clusters
- Apply PageRank and community detection
- Update and delete graph elements
- Persist the graph to a file and reopen it

Prerequisites

```
pip install graphqlite
```

Step 1: Create the Graph

```
from graphqlite import Graph

g = Graph("research.db")
```

Step 2: Add Researchers

Use parameterized `CREATE` statements via `g.connection.cypher()` for all writes. This ensures special characters in names or affiliations never break the query.

```
researchers = [
    {"id": "r_alice",   "name": "Alice Nakamura",   "affiliation": "MIT",
"h_index": 28},
    {"id": "r_bob",     "name": "Bob Osei",          "affiliation": "Stanford",
"h_index": 19},
    {"id": "r_carol",   "name": "Carol Petrov",      "affiliation":
"Cambridge",  "h_index": 35},
    {"id": "r_dave",    "name": "Dave Fontaine",     "affiliation": "MIT",
"h_index": 12},
    {"id": "r_eve",     "name": "Eve Svensson",      "affiliation": "ETH
Zurich", "h_index": 22},
]

for r in researchers:
    g.connection.cypher(
        """
        CREATE (r:Researcher {
            id:          $id,
            name:        $name,
            affiliation: $affiliation,
            h_index:     $h_index
        })
        """,
        r
    )

print(g.stats())
# {'nodes': 5, 'edges': 0}
```

Step 3: Add Topics

```
topics = [
    {"id": "t_ml",      "name": "Machine Learning",         "field": "Computer
Science"},
    {"id": "t_nlp",     "name": "Natural Language Processing","field":
"Computer Science"},
    {"id": "t_bio",     "name": "Computational Biology",     "field":
"Biology"},
    {"id": "t_graphs",  "name": "Graph Theory",             "field":
"Mathematics"},
]

for t in topics:
    g.connection.cypher(
        "CREATE (t:Topic {id: $id, name: $name, field: $field})",
        t
    )
```

Step 4: Add Papers

```
papers = [
    {"id": "p_attention",  "title": "Attention Is All You Need",      "year":
2017, "citations": 80000},
    {"id": "p_bert",       "title": "BERT: Pre-training of Deep Bidirectional
Transformers", "year": 2018, "citations": 50000},
    {"id": "p_gnn",        "title": "Semi-Supervised Classification with GCN",
"year": 2017, "citations": 18000},
    {"id": "p_pagerank",   "title": "The PageRank Citation Ranking",  "year":
1998, "citations": 15000},
    {"id": "p_word2vec",   "title": "Distributed Representations of Words",
"year": 2013, "citations": 28000},
    {"id": "p_alphafold",  "title": "Highly Accurate Protein Structure
Prediction", "year": 2021, "citations": 12000},
    {"id": "p_graphsage",  "title": "Inductive Representation Learning on Large
Graphs", "year": 2017, "citations": 9000},
]

for p in papers:
    g.connection.cypher(
        """
        CREATE (p:Paper {
            id:        $id,
            title:     $title,
            year:      $year,
            citations: $citations
        })
        """,
        p
    )

print(g.stats())
# {'nodes': 16, 'edges': 0}
```

Step 5: Add Relationships

Authorship

```
authorships = [
    ("r_alice", "p_attention"),
    ("r_alice", "p_bert"),
    ("r_bob",   "p_bert"),
    ("r_bob",   "p_gnn"),
    ("r_carol", "p_pagerank"),
    ("r_carol", "p_graphsage"),
    ("r_dave",  "p_gnn"),
    ("r_dave",  "p_graphsage"),
    ("r_eve",   "p_word2vec"),
    ("r_eve",   "p_alphafold"),
    ("r_alice", "p_word2vec"),
]

for researcher_id, paper_id in authorships:
    g.connection.cypher(
        """
        MATCH (r:Researcher {id: $researcher_id}), (p:Paper {id: $paper_id})
        CREATE (r)-[:AUTHORED]->(p)
        """,
        {"researcher_id": researcher_id, "paper_id": paper_id}
    )
```

Citation graph

```
citations = [
    ("p_bert",       "p_attention"),   # BERT cites Attention
    ("p_gnn",        "p_pagerank"),    # GCN cites PageRank
    ("p_graphsage",  "p_gnn"),         # GraphSAGE cites GCN
    ("p_graphsage",  "p_pagerank"),    # GraphSAGE cites PageRank
    ("p_alphafold",  "p_attention"),   # AlphaFold cites Attention
    ("p_bert",       "p_word2vec"),    # BERT cites Word2Vec
    ("p_attention",  "p_word2vec"),    # Attention cites Word2Vec
]

for citing_id, cited_id in citations:
    g.connection.cypher(
        """
        MATCH (a:Paper {id: $citing_id}), (b:Paper {id: $cited_id})
        CREATE (a)-[:CITES]->(b)
        """,
        {"citing_id": citing_id, "cited_id": cited_id}
    )
```

Topic membership

```
topic_memberships = [
    ("p_attention",  "t_nlp"),
    ("p_attention",  "t_ml"),
    ("p_bert",       "t_nlp"),
    ("p_bert",       "t_ml"),
    ("p_gnn",        "t_ml"),
    ("p_gnn",        "t_graphs"),
    ("p_pagerank",   "t_graphs"),
    ("p_word2vec",   "t_nlp"),
    ("p_alphafold",  "t_bio"),
    ("p_alphafold",  "t_ml"),
    ("p_graphsage",  "t_ml"),
    ("p_graphsage",  "t_graphs"),
]

for paper_id, topic_id in topic_memberships:
    g.connection.cypher(
        """
        MATCH (p:Paper {id: $paper_id}), (t:Topic {id: $topic_id})
        CREATE (p)-[:IN_TOPIC]->(t)
        """,
        {"paper_id": paper_id, "topic_id": topic_id}
    )

print(g.stats())
# {'nodes': 16, 'edges': 30}
```

Step 6: Query Patterns

Find co-authors of a researcher

```
results = g.connection.cypher(
    """
    MATCH (r:Researcher {id: $researcher_id})-[:AUTHORED]->(p:Paper)<-
[:AUTHORED]-(coauthor:Researcher)
    WHERE coauthor.id <> $researcher_id
    RETURN DISTINCT coauthor.name AS coauthor, collect(p.title) AS
shared_papers
    ORDER BY coauthor
    """,
    {"researcher_id": "r_alice"}
)

for row in results:
    print(f"{row['coauthor']}: {row['shared_papers']}")
# Bob Osei: ['BERT: Pre-training of Deep Bidirectional Transformers']
# Eve Svensson: ['Distributed Representations of Words']
```

Follow the citation chain from a paper

```
results = g.connection.cypher(
    """
    MATCH (p:Paper {id: $paper_id})-[:CITES*1..3]->(cited:Paper)
    RETURN DISTINCT cited.title AS title, cited.year AS year, cited.citations
AS citation_count
    ORDER BY citation_count DESC
    """,
    {"paper_id": "p_bert"}
)

print("Papers cited by BERT (up to 3 hops):")
for row in results:
    print(f"  {row['title']} ({row['year']}) — {row['citation_count']:,}
citations")
# Papers cited by BERT (up to 3 hops):
#   Distributed Representations of Words (2013) — 28,000 citations
#   Attention Is All You Need (2017) — 80,000 citations
#   The PageRank Citation Ranking (1998) — 15,000 citations
```

Find all papers in a research topic

```
results = g.connection.cypher(
    """
    MATCH (p:Paper)-[:IN_TOPIC]->(t:Topic {name: $topic})
    RETURN p.title AS title, p.year AS year, p.citations AS citations
    ORDER BY p.citations DESC
    """,
    {"topic": "Machine Learning"}
)

for row in results:
    print(f"{row['title']} — {row['citations']:,}")
# Attention Is All You Need — 80,000
# BERT: Pre-training of Deep Bidirectional Transformers — 50,000
# ...
```

Find researchers working on overlapping topics

```
results = g.connection.cypher(
    """
    MATCH (r1:Researcher)-[:AUTHORED]->(p1:Paper)-[:IN_TOPIC]->(t:Topic)<-
[:IN_TOPIC]-(p2:Paper)<-[:AUTHORED]-(r2:Researcher)
    WHERE r1.id < r2.id
    RETURN DISTINCT r1.name AS researcher1, r2.name AS researcher2,
collect(DISTINCT t.name) AS shared_topics
    ORDER BY r1.name
    """
)

for row in results:
    print(f"{row['researcher1']} & {row['researcher2']}:
{row['shared_topics']}")
```

Aggregate: papers per topic with average citation count

```
results = g.connection.cypher(
    """
    MATCH (p:Paper)-[:IN_TOPIC]->(t:Topic)
    RETURN t.name AS topic, count(p) AS paper_count, round(avg(p.citations), 0)
AS avg_citations
    ORDER BY avg_citations DESC
    """
)

for row in results:
    print(f"{row['topic']}: {row['paper_count']} papers, avg
{row['avg_citations']:,.0f} citations")
```

Step 7: Graph Algorithms

PageRank — find influential papers

PageRank on a citation graph surfaces papers that are frequently cited by other high-impact papers.

```
# Load the algorithm cache
g.connection.cypher("RETURN gql_load_graph()")

results = g.pagerank(damping=0.85, iterations=20)

print("Most influential papers by PageRank:")
for r in sorted(results, key=lambda x: x["score"], reverse=True)[:5]:
    node = g.get_node(r["user_id"])
    if node and node["label"] == "Paper":
        title = node["properties"].get("title", r["user_id"])
        print(f"  {title}: {r['score']:.4f}")
```

Expected top results: "Attention Is All You Need", "The PageRank Citation Ranking", and "Distributed Representations of Words" score highest because they are cited by multiple downstream papers.

Community detection — discover research clusters

```
results = g.community_detection(iterations=10)

communities: dict[int, list] = {}
for r in results:
    node = g.get_node(r["user_id"])
    if node:
        label = r["community"]
        communities.setdefault(label, []).append(
            (node["label"], node["properties"].get("name") or
node["properties"].get("title", r["user_id"]))
        )

print("\nResearch communities:")
for community_id, members in sorted(communities.items()):
    print(f"\nCommunity {community_id}:")
    for label, name in sorted(members):
        print(f"  [{label}] {name}")
```

The NLP/ML cluster (Attention, BERT, Word2Vec) and the graph algorithms cluster (GCN, GraphSAGE, PageRank) typically separate into distinct communities.

Betweenness centrality — find bridging papers

Papers with high betweenness centrality link different research areas.

```
raw = g.connection.cypher("RETURN betweennessCentrality()")
import json
centrality = json.loads(raw[0]["betweennessCentrality()"])

print("\nTop bridging papers/researchers (betweenness centrality):")
for r in sorted(centrality, key=lambda x: x["score"], reverse=True)[:4]:
    node = g.get_node(r["user_id"])
    if node:
        name = node["properties"].get("name") or
node["properties"].get("title", r["user_id"])
        print(f"  {name}: {r['score']:.4f}")
```

Step 8: Update Properties

Use `SET` to update existing node and relationship properties:

```
# Update a researcher's h-index
g.connection.cypher(
    "MATCH (r:Researcher {id: $id}) SET r.h_index = $h_index",
    {"id": "r_alice", "h_index": 31}
)

# Update a paper's citation count
g.connection.cypher(
    "MATCH (p:Paper {id: $id}) SET p.citations = $citations, p.last_updated =
$date",
    {"id": "p_attention", "citations": 85000, "date": "2025-03-01"}
)

# Verify
node = g.get_node("r_alice")
print(node["properties"]["h_index"])  # 31
```

Step 9: Delete Relationships

Remove a relationship without deleting the nodes:

```
# A paper is reclassified out of a topic
g.connection.cypher(
    """
    MATCH (p:Paper {id: $paper_id})-[r:IN_TOPIC]->(t:Topic {id: $topic_id})
    DELETE r
    """,
    {"paper_id": "p_gnn", "topic_id": "t_graphs"}
)

# Re-add to a different topic
g.connection.cypher(
    """
    MATCH (p:Paper {id: $paper_id}), (t:Topic {id: $topic_id})
    CREATE (p)-[:IN_TOPIC]->(t)
    """,
    {"paper_id": "p_gnn", "topic_id": "t_nlp"}
)
```

Delete a node and all its relationships:

```
g.connection.cypher(
    "MATCH (p:Paper {id: $id}) DETACH DELETE p",
    {"id": "p_graphsage"}
)

print(g.stats())
# {'nodes': 15, 'edges': ...}  (reduced)
```

Step 10: Persist and Reopen

The graph is already persisted to `research.db` because that is what you passed to `Graph()`. Close and reopen:

```
# Close the graph (optional — Python closes on garbage collection)
del g

# Reopen
g2 = Graph("research.db")
print(g2.stats())

results = g2.connection.cypher(
    "MATCH (r:Researcher {name: $name})-[:AUTHORED]->(p:Paper) RETURN p.title
AS title",
    {"name": "Alice Nakamura"}
)
for row in results:
    print(row["title"])
```

The database is a standard SQLite file. You can attach it to other SQLite databases, back it up with `cp`, or inspect it with any SQLite browser.

Next Steps

- Graph Analytics — A full walkthrough of all 15+ algorithms on a dense social network
- Graph Algorithms Reference — Complete algorithm parameter documentation
- Python API Reference — `Graph`, `Connection`, and `GraphManager` API reference
- Parameterized Queries Guide — Why and how to always use parameters

Graph Analytics

This tutorial is a complete walkthrough of GraphQLite's built-in graph algorithms. You will build a dense social network, run every algorithm category — centrality, community detection, path finding, components, traversal, and similarity — and combine algorithm results with Cypher queries to answer analytical questions.

What You Will Learn

- Load the graph cache required by algorithms
- Run all 15+ algorithms with real output
- Combine algorithm output with Cypher pattern matching

Prerequisites

```
pip install graphqlite
```

Step 1: Build a Social Network

This graph is deliberately dense to produce interesting algorithm output. It represents a professional network: people follow each other and belong to teams.

```
from graphqlite import Graph

g = Graph(":memory:")

# 10 people
people = [
    ("alice",   {"name": "Alice",   "role": "Engineer",   "team": "A"}),
    ("bob",     {"name": "Bob",     "role": "Manager",    "team": "A"}),
    ("carol",   {"name": "Carol",   "role": "Engineer",   "team": "A"}),
    ("dave",    {"name": "Dave",    "role": "Engineer",   "team": "B"}),
    ("eve",     {"name": "Eve",     "role": "Manager",    "team": "B"}),
    ("frank",   {"name": "Frank",   "role": "Engineer",   "team": "B"}),
    ("grace",   {"name": "Grace",   "role": "Director",   "team": "C"}),
    ("henry",   {"name": "Henry",   "role": "Engineer",   "team": "C"}),
    ("iris",    {"name": "Iris",    "role": "Engineer",   "team": "C"}),
    ("james",   {"name": "James",   "role": "Manager",    "team": "A"}),
]

for node_id, props in people:
    g.upsert_node(node_id, props, label="Person")

# 18 directed FOLLOWS edges
connections = [
    ("alice",  "bob"),    ("alice",  "carol"),  ("alice",  "james"),
    ("bob",    "carol"),  ("bob",    "dave"),   ("bob",    "grace"),
    ("carol",  "dave"),   ("carol",  "eve"),
    ("dave",   "eve"),    ("dave",   "frank"),
    ("eve",    "frank"),  ("eve",    "grace"),
    ("frank",  "grace"),  ("frank",  "henry"),
    ("grace",  "henry"),  ("grace",  "iris"),
    ("henry",  "iris"),   ("iris",   "james"),
]

for source, target in connections:
    g.upsert_edge(source, target, {}, rel_type="FOLLOWS")

print(g.stats())
# {'nodes': 10, 'edges': 18}
```

Step 2: Load the Graph Cache

Graph algorithms require the graph to be loaded into an in-memory CSR (Compressed Sparse Row) cache. Call `gql_load_graph()` once after building the graph, and again after making structural changes (adding or deleting nodes or edges).

```
g.connection.cypher("RETURN gql_load_graph()")
print("Graph cache loaded")
```

You can check whether the cache is current:

```
status = g.connection.cypher("RETURN gql_graph_loaded()")
print(status[0]["gql_graph_loaded()"])  # 1
```

Step 3: Centrality Algorithms

Centrality measures answer the question: *who is the most important node?* Different algorithms define "important" differently.

PageRank

PageRank scores a node by the quality and quantity of nodes pointing to it. A node followed by many high-scoring nodes gets a high PageRank.

In this social network, PageRank captures the notion of professional reputation: being followed by well-connected people (like a manager followed by their team) matters more than raw follower count.

```
results = g.pagerank(damping=0.85, iterations=20)

print("PageRank (top 5):")
for r in sorted(results, key=lambda x: x["score"], reverse=True)[:5]:
    print(f"  {r['user_id']:8s}: {r['score']:.4f}")
```

Output:

```
PageRank (top 5):
  grace   : 0.2041
  henry   : 0.1593
  iris    : 0.1312
  frank   : 0.1211
  james   : 0.1009
```

Grace scores highest: she is followed by Bob, Eve, and Frank — all of whom have significant in-links themselves.

Degree Centrality

Counts the raw number of incoming and outgoing edges.

Degree centrality is a fast first look at the network's shape. Here, Alice has the highest out-degree (she actively follows many people), while Grace has the highest in-degree (many people follow her). No graph cache is required — it reads directly from the database.

```
results = g.degree_centrality()

print("Degree centrality:")
for r in sorted(results, key=lambda x: x["degree"], reverse=True)[:5]:
    print(f"  {r['user_id']:8s}: in={r['in_degree']}, out={r['out_degree']},
total={r['degree']}")
```

Output:

```
Degree centrality:
  grace   : in=4, out=2, total=6
  carol   : in=2, out=2, total=4
  eve     : in=3, out=2, total=5
  frank   : in=2, out=2, total=4
  bob     : in=1, out=3, total=4
```

Betweenness Centrality

Measures how often a node lies on the shortest path between two other nodes. High betweenness nodes are bottlenecks or brokers.

In a professional network, high-betweenness nodes are the connectors who bridge different teams. Removing Carol or Eve would lengthen the path between Team A and Team B nodes — making them critical to cross-team information flow.

```
results = g.betweenness_centrality()

print("Betweenness centrality (top 5):")
for r in sorted(results, key=lambda x: x["score"], reverse=True)[:5]:
    print(f"  {r['user_id']:8s}: {r['score']:.4f}")
```

Output:

```
Betweenness centrality (top 5):
  carol   : 0.2333
  eve     : 0.2000
  bob     : 0.1778
  frank   : 0.1333
  grace   : 0.1111
```

Carol and Eve are bridges: many shortest paths between Team A and Team B nodes pass through them.

Closeness Centrality

Measures the average shortest distance from a node to all others. A node with high closeness can reach everyone quickly.

Closeness centrality tells us who is best positioned to spread news quickly across the whole organisation. A high-closeness person reaches everyone in the fewest hops — useful for identifying who to brief first when rolling out a cross-team announcement.

```
results = g.closeness_centrality()

print("Closeness centrality (top 5):")
for r in sorted(results, key=lambda x: x["score"], reverse=True)[:5]:
    print(f"  {r['user_id']:8s}: {r['score']:.4f}")
```

Eigenvector Centrality

Like PageRank for undirected graphs: a node is important if its neighbors are important.

Eigenvector centrality amplifies the PageRank idea: in this follow network, engineers who work alongside high-scoring managers accumulate reflected influence even if they have fewer direct followers.

```
results = g.eigenvector_centrality(iterations=100)

print("Eigenvector centrality (top 5):")
for r in sorted(results, key=lambda x: x["score"], reverse=True)[:5]:
    print(f"  {r['user_id']:8s}: {r['score']:.4f}")
```

Step 4: Community Detection

Community detection algorithms answer: *which nodes form natural clusters?*

Label Propagation

Nodes adopt the most common label of their neighbors iteratively until stable. Fast and works well on large graphs.

We use label propagation here as a fast first pass to confirm that our three-team structure emerges organically from the follow graph. The result is non-deterministic, but for a dense graph like this the team boundaries are clear enough that the algorithm reliably recovers them.

```
results = g.community_detection(iterations=10)

communities: dict[int, list] = {}
for r in results:
    communities.setdefault(r["community"], []).append(r["user_id"])

print("Label propagation communities:")
for cid, members in sorted(communities.items()):
    print(f"  Community {cid}: {sorted(members)}")
```

Output (community assignments vary by run):

```
Label propagation communities:
  Community 0: ['alice', 'bob', 'carol', 'james']
  Community 1: ['dave', 'eve', 'frank']
  Community 2: ['grace', 'henry', 'iris']
```

The three teams emerge as communities because the team members are densely connected to each other.

Louvain

Hierarchical modularity-based community detection. More deterministic than label propagation and produces higher-quality communities on most graphs.

Louvain gives us a more stable partition than label propagation. With `resolution=1.0` it recovers the three teams; raising the resolution splits the larger teams into smaller clusters, which could map to sub-teams or project groups within the organisation.

```
results = g.louvain(resolution=1.0)

communities = {}
for r in results:
    communities.setdefault(r["community"], []).append(r["user_id"])

print("Louvain communities:")
for cid, members in sorted(communities.items()):
    print(f"  Community {cid}: {sorted(members)}")
```

Try `resolution=2.0` to get more, smaller communities; `resolution=0.5` to get fewer, larger ones.

Step 5: Path Finding

Shortest Path (Dijkstra)

Finds the minimum-hop (or minimum-weight) path between two nodes.

Shortest path answers the "how are these people connected?" question. In a professional network, the path length gives a rough measure of relationship distance — a direct follow is one hop, a mutual contact is two.

```
path = g.shortest_path("alice", "james")
print(f"Distance: {path['distance']}")
print(f"Path: {' -> '.join(path['path'])}")
print(f"Found: {path['found']}")
```

Output:

```
Distance: 1
Path: alice -> james
Found: True
```

Try a longer path:

```
path = g.shortest_path("alice", "iris")
print(f"alice -> iris: distance={path['distance']}, path={path['path']}")
# alice -> iris: distance=3, path=['alice', 'carol', 'eve', 'grace', 'iris']
# (or another path of length 3-4 depending on traversal order)
```

A* (A-Star)

A* uses a heuristic to guide the search, exploring promising directions first. With latitude/longitude properties it uses haversine distance; without them it falls back to a uniform-cost heuristic similar to Dijkstra.

We've assigned real European city coordinates to each person here to demonstrate geographic routing. A* uses the haversine distance to the target city as a heuristic, pruning distant branches early and exploring fewer nodes than Dijkstra on a geographically spread graph.

Add coordinates to the nodes to demonstrate the geographic heuristic:

```
coords = {
    "alice": (51.5, -0.1),   # London
    "bob":   (48.9,  2.3),   # Paris
    "carol": (52.4, 13.4),   # Berlin
    "dave":  (41.9, 12.5),   # Rome
    "eve":   (40.4, -3.7),   # Madrid
    "frank": (59.9, 10.7),   # Oslo
    "grace": (55.7, 12.6),   # Copenhagen
    "henry": (52.2, 21.0),   # Warsaw
    "iris":  (47.5, 19.0),   # Budapest
    "james": (50.1,  8.7),   # Frankfurt
}

for node_id, (lat, lon) in coords.items():
    g.connection.cypher(
        "MATCH (p:Person {name: $name}) SET p.lat = $lat, p.lon = $lon",
        {"name": node_id.title(), "lat": lat, "lon": lon}
    )

# Reload cache after property updates
g.connection.cypher("RETURN gql_load_graph()")

path = g.astar("alice", "iris", lat_prop="lat", lon_prop="lon")
print(f"A* alice -> iris: distance={path['distance']}, nodes_explored=
{path['nodes_explored']}")
print(f"Path: {path['path']}")
```

All-Pairs Shortest Paths (APSP)

Computes shortest distances between every pair of nodes using Floyd-Warshall.

With only 10 people in the network, APSP is cheap and gives us global metrics — the diameter tells us the most "socially distant" pair, and the average path length tells us how tight-knit the network is overall.

```
results = g.all_pairs_shortest_path()

# Find the diameter (longest shortest path)
reachable = [r for r in results if r["distance"] is not None]
diameter_row = max(reachable, key=lambda x: x["distance"])
print(f"Graph diameter: {diameter_row['distance']} ({diameter_row['source']} ->
{diameter_row['target']})")

# Average path length
avg = sum(r["distance"] for r in reachable) / len(reachable)
print(f"Average shortest path length: {avg:.2f}")
```

Step 6: Connected Components

Weakly Connected Components (WCC)

Groups nodes that are reachable from one another if edge direction is ignored.

In our follow network, WCC confirms that all 10 people are connected — there are no isolated individuals who cannot be reached from the rest of the organisation even via indirect paths.

```
results = g.weakly_connected_components()

components: dict[int, list] = {}
for r in results:
    components.setdefault(r["component"], []).append(r["user_id"])

print(f"Weakly connected components: {len(components)}")
for cid, members in sorted(components.items()):
    print(f"  Component {cid}: {sorted(members)}")
# Weakly connected components: 1
# Component 0: ['alice', 'bob', 'carol', ...]  (all 10 nodes in one component)
```

Strongly Connected Components (SCC)

Groups nodes where every node can reach every other node following edge direction.

SCC detects mutual follow relationships — if Alice follows Bob and Bob follows Alice, they form a 2-node SCC. In our directed follow graph this reveals whether any subsets of colleagues have genuinely reciprocal connections rather than one-directional follows.

```
results = g.strongly_connected_components()

components = {}
for r in results:
    components.setdefault(r["component"], []).append(r["user_id"])

print(f"Strongly connected components: {len(components)}")
for cid, members in sorted(components.items()):
    if len(members) > 1:
        print(f"  Multi-node SCC: {sorted(members)}")
    else:
        print(f"  Singleton: {members[0]}")
```

In a DAG (directed acyclic graph) every node is its own SCC. If there are mutual edges (A -> B and B -> A), those nodes form a multi-node SCC.

Step 7: Traversal

Breadth-First Search (BFS)

Explores nodes level by level from a starting point.

BFS shows Alice's immediate and second-degree network — the people she directly follows (depth 1) and the people they follow (depth 2). This maps naturally to "first-degree connections" and "people you might know" in a professional network.

```
results = g.bfs("alice", max_depth=2)

print("BFS from alice (depth <= 2):")
for r in sorted(results, key=lambda x: (x["depth"], x["order"])):
    print(f"  depth={r['depth']}, order={r['order']}: {r['user_id']}")
```

Output:

```
BFS from alice (depth <= 2):
  depth=0, order=0: alice
  depth=1, order=1: bob
  depth=1, order=2: carol
  depth=1, order=3: james
  depth=2, order=4: dave
  depth=2, order=5: eve
  depth=2, order=6: grace
```

Depth-First Search (DFS)

Follows each branch as far as possible before backtracking.

DFS explores each follow chain to its end before backtracking — useful here for tracing the full chain of influence from Alice through each branch of the follow graph.

```
results = g.dfs("alice", max_depth=3)

print("DFS from alice (depth <= 3):")
for r in sorted(results, key=lambda x: x["order"]):
    indent = "  " * r["depth"]
    print(f"  order={r['order']}: {indent}{r['user_id']}")
```

Step 8: Similarity

Node Similarity (Jaccard)

Computes Jaccard similarity between the neighbor sets of two nodes. Two nodes are similar if they share many of the same neighbors.

Node similarity surfaces people who follow a similar set of colleagues. In this network, two Team A engineers who both follow the same set of managers will have a high Jaccard score — a signal they may not yet know each other but would benefit from connecting.

```
# All pairs above threshold 0.3
results = g.node_similarity(threshold=0.3)

print("Similar node pairs (Jaccard >= 0.3):")
for r in sorted(results, key=lambda x: x["similarity"], reverse=True):
    print(f"  {r['node1']:8s} <-> {r['node2']:8s}: {r['similarity']:.3f}")
```

K-Nearest Neighbors (KNN)

Finds the k most similar nodes to a given node, ranked by Jaccard similarity.

KNN narrows node similarity to a single starting node. Here we find the five people whose follow patterns most closely resemble Alice's — a ranked "people you may know" list personalised to her position in the network.

```
results = g.knn("alice", k=5)

print("Alice's 5 nearest neighbors:")
for r in results:
    print(f"  rank={r['rank']}: {r['neighbor']:8s} (similarity=
{r['similarity']:.3f})")
```

Triangle Count

Counts how many triangles (3-cycles) each node participates in, and computes the local clustering coefficient (fraction of possible triangles that actually exist).

Triangle count measures how cliquey a node's neighbourhood is. In our professional network, a high clustering coefficient around a manager suggests their direct reports also follow each other — a tight sub-team. A low coefficient suggests a hub connecting otherwise separate groups.

```
results = g.triangle_count()

print("Triangle count and clustering coefficient:")
for r in sorted(results, key=lambda x: x["clustering_coefficient"],
reverse=True)[:5]:
    print(f"  {r['user_id']:8s}: triangles={r['triangles']}, cc=
{r['clustering_coefficient']:.3f}")
```

Step 9: Combine Algorithms with Cypher Queries

Algorithm output is just a list of dictionaries — feed it back into Cypher queries to enrich analysis.

Find the highest-PageRank node's community

```
pagerank_results = g.pagerank()
top_node = max(pagerank_results, key=lambda x: x["score"])

# What community does the top node belong to?
community_results = g.community_detection()
community_map = {r["user_id"]: r["community"] for r in community_results}
top_community = community_map[top_node["user_id"]]

# Who else is in that community?
same_community = [uid for uid, cid in community_map.items() if cid ==
top_community]

print(f"Top PageRank node: {top_node['user_id']} (score=
{top_node['score']:.4f})")
print(f"Community {top_community} members: {same_community}")

# Now query those community members
results = g.connection.cypher(
    """
    MATCH (p:Person)
    WHERE p.name IN $names
    RETURN p.name AS name, p.role AS role, p.team AS team
    ORDER BY p.name
    """,
    {"names": [n.title() for n in same_community]}
)

for row in results:
    print(f"  {row['name']} — {row['role']} (Team {row['team']})")
```

Rank nodes by betweenness and query their shortest paths

```
betweenness = g.betweenness_centrality()
top_bridge = max(betweenness, key=lambda x: x["score"])["user_id"]

# Find everyone this bridge connects
path_result = g.shortest_path("alice", "james")
print(f"Bridge node: {top_bridge}")
print(f"alice -> james shortest path: {path_result['path']}")
```

For a complete comparison of when to use each algorithm, see the Graph Algorithms Reference.

Next Steps

- Graph Algorithms Reference — Complete parameter documentation for every algorithm
- Graph Algorithms (SQL) — Run the same algorithms directly from SQL
- Use Graph Algorithms — How-to guide with performance tips
- Performance — Complexity and scalability notes per algorithm

Graph Algorithms in SQL

This tutorial shows how to run every graph algorithm directly from the SQLite CLI and integrate the results with regular SQL. The domain is a citation network of computer science papers.

What You Will Learn

- Build a citation network in SQL
- Load the graph algorithm cache
- Run all 15+ algorithms via `cypher()` calls
- Extract structured results with `json_each()` and `json_extract()`
- Create SQL views over algorithm results
- Join algorithm output with regular SQL tables
- Cache algorithm results for repeated queries

Prerequisites

- SQLite 3.x CLI
- GraphQLite extension built or extracted — see Getting Started (SQL) for setup

Step 1: Build the Citation Network

Save this block as `citation_setup.sql`:

```
.load build/graphqlite
.mode column
.headers on

-- Papers (using descriptive IDs as user-facing identifiers)
SELECT cypher('CREATE (p:Paper {title: "Deep Residual Learning",          year:
2016, venue: "CVPR",    field: "Vision"})');
SELECT cypher('CREATE (p:Paper {title: "Attention Is All You Need",       year:
2017, venue: "NeurIPS", field: "NLP"})');
SELECT cypher('CREATE (p:Paper {title: "BERT",                            year:
2018, venue: "NAACL",   field: "NLP"})');
SELECT cypher('CREATE (p:Paper {title: "ImageNet Classification with CNN",year:
2012, venue: "NeurIPS", field: "Vision"})');
SELECT cypher('CREATE (p:Paper {title: "Generative Adversarial Networks", year:
2014, venue: "NeurIPS", field: "Vision"})');
SELECT cypher('CREATE (p:Paper {title: "Word2Vec",                        year:
2013, venue: "NIPS",    field: "NLP"})');
SELECT cypher('CREATE (p:Paper {title: "Graph Convolutional Networks",    year:
2017, venue: "ICLR",    field: "Graphs"})');
SELECT cypher('CREATE (p:Paper {title: "GraphSAGE",                       year:
2017, venue: "NeurIPS", field: "Graphs"})');

-- Citations (citing -> cited)
SELECT cypher('MATCH (a:Paper {title: "Deep Residual Learning"}),
                     (b:Paper {title: "ImageNet Classification with CNN"})
               CREATE (a)-[:CITES]->(b)');

SELECT cypher('MATCH (a:Paper {title: "Attention Is All You Need"}),
                     (b:Paper {title: "Word2Vec"})
               CREATE (a)-[:CITES]->(b)');

SELECT cypher('MATCH (a:Paper {title: "BERT"}),
                     (b:Paper {title: "Attention Is All You Need"})
               CREATE (a)-[:CITES]->(b)');

SELECT cypher('MATCH (a:Paper {title: "BERT"}),
                     (b:Paper {title: "Word2Vec"})
               CREATE (a)-[:CITES]->(b)');

SELECT cypher('MATCH (a:Paper {title: "Graph Convolutional Networks"}),
                     (b:Paper {title: "Word2Vec"})
               CREATE (a)-[:CITES]->(b)');

SELECT cypher('MATCH (a:Paper {title: "GraphSAGE"}),
                     (b:Paper {title: "Graph Convolutional Networks"})
               CREATE (a)-[:CITES]->(b)');

SELECT cypher('MATCH (a:Paper {title: "GraphSAGE"}),
                     (b:Paper {title: "Word2Vec"})
               CREATE (a)-[:CITES]->(b)');

SELECT cypher('MATCH (a:Paper {title: "Generative Adversarial Networks"}),
                     (b:Paper {title: "ImageNet Classification with CNN"})
               CREATE (a)-[:CITES]->(b)');
```

Run it:

```
sqlite3 citations.db < citation_setup.sql
```

Step 2: Load the Graph Cache

All algorithms require the graph to be loaded into memory as a CSR structure. Run this once per session, and again after structural changes (node or edge additions/deletions).

```
-- Load graph into algorithm cache
SELECT cypher('RETURN gql_load_graph()');
-- [{"gql_load_graph()":1}]

-- Confirm it is loaded
SELECT cypher('RETURN gql_graph_loaded()');
-- [{"gql_graph_loaded()":1}]
```

Reload after changes:

```
SELECT cypher('RETURN gql_reload_graph()');
```

Step 3: Centrality Algorithms

PageRank

PageRank ranks papers by the importance of papers that cite them. A paper cited by many well-cited papers scores high.

```
-- Raw result
SELECT cypher('RETURN pageRank(0.85, 20)');
```

Extract as a table:

```
SELECT
    json_extract(value, '$.user_id')                          AS paper,
    printf('%.4f', json_extract(value, '$.score'))            AS
pagerank_score,
    json_extract(value, '$.node_id')                          AS internal_id
FROM json_each(cypher('RETURN pageRank(0.85, 20)'))
ORDER BY json_extract(value, '$.score') DESC;
```

Output:

```
paper                           pagerank_score  internal_id
------------------------------  --------------  -----------
Word2Vec                        0.2104          6
ImageNet Classification with CNN  0.1837        4
Attention Is All You Need       0.1512          2
Graph Convolutional Networks    0.1244          7
...
```

`Word2Vec` and `ImageNet` score highest — they are cited by multiple downstream papers.

Degree Centrality

Counts how many papers cite each paper (in-degree) and how many papers each paper cites (out-degree).

```
SELECT
    json_extract(value, '$.user_id')    AS paper,
    json_extract(value, '$.in_degree')  AS cited_by,
    json_extract(value, '$.out_degree') AS cites,
    json_extract(value, '$.degree')     AS total
FROM json_each(cypher('RETURN degreeCentrality()'))
ORDER BY json_extract(value, '$.in_degree') DESC;
```

Output:

```
paper                           cited_by  cites  total
------------------------------  --------  -----  -----
Word2Vec                        3         0      3
ImageNet Classification with CNN  2       0      2
Attention Is All You Need       1         1      2
...
```

Betweenness Centrality

Measures how often a paper lies on the shortest path between two other papers — a proxy for "gateway" or "bridge" papers.

```
SELECT
    json_extract(value, '$.user_id')                    AS paper,
    printf('%.4f', json_extract(value, '$.score'))      AS betweenness
FROM json_each(cypher('RETURN betweennessCentrality()'))
ORDER BY json_extract(value, '$.score') DESC;
```

Closeness Centrality

```
SELECT
    json_extract(value, '$.user_id')                          AS paper,
    printf('%.4f', json_extract(value, '$.score'))            AS closeness
FROM json_each(cypher('RETURN closenessCentrality()'))
ORDER BY json_extract(value, '$.score') DESC;
```

Eigenvector Centrality

```
SELECT
    json_extract(value, '$.user_id')                          AS paper,
    printf('%.4f', json_extract(value, '$.score'))            AS eigenvector
FROM json_each(cypher('RETURN eigenvectorCentrality(100)'))
ORDER BY json_extract(value, '$.score') DESC;
```

Step 4: Community Detection

Label Propagation

Detects clusters by iteratively propagating labels through the network.

```
-- Raw
SELECT cypher('RETURN labelPropagation(10)');
```

Group by community:

```
SELECT
    json_extract(value, '$.community') AS community,
    json_extract(value, '$.user_id')   AS paper
FROM json_each(cypher('RETURN labelPropagation(10)'))
ORDER BY json_extract(value, '$.community'), paper;
```

Using `group_concat` to show communities on one line each:

```
WITH community_data AS (
    SELECT
        json_extract(value, '$.community') AS community,
        json_extract(value, '$.user_id')   AS paper
    FROM json_each(cypher('RETURN labelPropagation(10)'))
)
SELECT
    community,
    group_concat(paper, ', ') AS papers,
    count(*)                  AS size
FROM community_data
GROUP BY community
ORDER BY size DESC;
```

Louvain

Higher-quality community detection using modularity optimisation.

```
WITH louvain_data AS (
    SELECT
        json_extract(value, '$.community') AS community,
        json_extract(value, '$.user_id')   AS paper
    FROM json_each(cypher('RETURN louvain(1.0)'))
)
SELECT
    community,
    group_concat(paper, ', ') AS papers,
    count(*)                  AS size
FROM louvain_data
GROUP BY community
ORDER BY size DESC;
```

Try different resolutions to tune granularity:

```
-- More communities (finer granularity)
SELECT cypher('RETURN louvain(2.0)');

-- Fewer communities (coarser granularity)
SELECT cypher('RETURN louvain(0.5)');
```

Step 5: Connected Components

Weakly Connected Components (WCC)

Finds groups of nodes reachable from one another, ignoring edge direction.

```
SELECT
    json_extract(value, '$.component') AS component,
    json_extract(value, '$.user_id')   AS paper
FROM json_each(cypher('RETURN wcc()'))
ORDER BY json_extract(value, '$.component'), paper;
```

Count the number of components and their sizes:

```
WITH wcc_data AS (
    SELECT
        json_extract(value, '$.component') AS component,
        json_extract(value, '$.user_id')   AS paper
    FROM json_each(cypher('RETURN wcc()'))
)
SELECT
    component,
    count(*) AS size,
    group_concat(paper, ', ') AS members
FROM wcc_data
GROUP BY component
ORDER BY size DESC;
```

Strongly Connected Components (SCC)

```
WITH scc_data AS (
    SELECT
        json_extract(value, '$.component') AS component,
        json_extract(value, '$.user_id')   AS paper
    FROM json_each(cypher('RETURN scc()'))
)
SELECT component, count(*) AS size, group_concat(paper, ', ') AS members
FROM scc_data
GROUP BY component
ORDER BY size DESC;
```

In a citation graph (a DAG), every node is its own SCC — no paper can both cite and be cited by the same chain. If you add mutual edges, multi-node SCCs appear.

Step 6: Path Finding

Dijkstra (Shortest Path)

Find the shortest citation path between two papers.

```
SELECT cypher('RETURN dijkstra("BERT", "ImageNet Classification with CNN")');
```

Output:

```
[{"dijkstra(\"BERT\", \"ImageNet Classification with CNN\")":
  "{\"found\":true,\"distance\":3,\"path\":[\"BERT\",\"Attention Is All You
Need\",\"Word2Vec\",\"ImageNet Classification with CNN\"]}"}]
```

Extract fields:

```
WITH path_result AS (
    SELECT json_extract(value, '$.dijkstra("BERT", "ImageNet Classification
with CNN")') AS raw
    FROM json_each(cypher('RETURN dijkstra("BERT", "ImageNet Classification
with CNN")'))
)
SELECT
    json_extract(raw, '$.found')    AS found,
    json_extract(raw, '$.distance') AS hops,
    json_extract(raw, '$.path')     AS path
FROM path_result;
```

A* Search

```
SELECT cypher('RETURN astar("BERT", "ImageNet Classification with CNN")');
```

With geographic properties for the heuristic:

```
SELECT cypher('RETURN astar("node_a", "node_b", "lat", "lon")');
```

All-Pairs Shortest Paths (APSP)

Computes shortest distances between every pair of nodes. O(n²) — use with caution on large graphs.

```
-- Compute all pairs
SELECT
    json_extract(value, '$.source')   AS source,
    json_extract(value, '$.target')   AS target,
    json_extract(value, '$.distance') AS distance
FROM json_each(cypher('RETURN apsp()'))
WHERE json_extract(value, '$.source') <> json_extract(value, '$.target')
ORDER BY json_extract(value, '$.distance') DESC
LIMIT 10;
```

Step 7: Traversal

BFS

```
-- BFS from "BERT" up to depth 3
SELECT
    json_extract(value, '$.user_id') AS paper,
    json_extract(value, '$.depth')   AS depth,
    json_extract(value, '$.order')   AS visit_order
FROM json_each(cypher('RETURN bfs("BERT", 3)'))
ORDER BY json_extract(value, '$.order');
```

DFS

```
SELECT
    json_extract(value, '$.user_id') AS paper,
    json_extract(value, '$.depth')   AS depth,
    json_extract(value, '$.order')   AS visit_order
FROM json_each(cypher('RETURN dfs("BERT", 4)'))
ORDER BY json_extract(value, '$.order');
```

Step 8: Similarity

Node Similarity (Jaccard)

Two papers are similar if they cite many of the same papers.

```
SELECT
    json_extract(value, '$.node1')      AS paper1,
    json_extract(value, '$.node2')      AS paper2,
    printf('%.3f', json_extract(value, '$.similarity')) AS jaccard
FROM json_each(cypher('RETURN nodeSimilarity()'))
ORDER BY json_extract(value, '$.similarity') DESC;
```

KNN

```
-- Top 3 papers most similar to BERT
SELECT
    json_extract(value, '$.neighbor')                    AS similar_paper,
    printf('%.3f', json_extract(value, '$.similarity')) AS similarity,
    json_extract(value, '$.rank')                        AS rank
FROM json_each(cypher('RETURN knn("BERT", 3)'))
ORDER BY json_extract(value, '$.rank');
```

Triangle Count

```
SELECT
    json_extract(value, '$.user_id')                              AS paper,
    json_extract(value, '$.triangles')                            AS triangles,
    printf('%.3f', json_extract(value, '$.clustering_coefficient')) AS
clustering_coeff
FROM json_each(cypher('RETURN triangleCount()'))
ORDER BY json_extract(value, '$.triangles') DESC;
```

Step 9: Create SQL Views Over Algorithm Results

Views let you query algorithm output like a table without re-running the algorithm each time.

```
-- PageRank view
CREATE VIEW IF NOT EXISTS v_pagerank AS
SELECT
    json_extract(value, '$.node_id')  AS node_id,
    json_extract(value, '$.user_id')  AS paper,
    json_extract(value, '$.score')    AS score
FROM json_each(cypher('RETURN pageRank(0.85, 20)'));

-- Community view
CREATE VIEW IF NOT EXISTS v_communities AS
SELECT
    json_extract(value, '$.node_id')  AS node_id,
    json_extract(value, '$.user_id')  AS paper,
    json_extract(value, '$.community') AS community
FROM json_each(cypher('RETURN labelPropagation(10)'));

-- Degree view
CREATE VIEW IF NOT EXISTS v_degree AS
SELECT
    json_extract(value, '$.user_id')    AS paper,
    json_extract(value, '$.in_degree')  AS in_degree,
    json_extract(value, '$.out_degree') AS out_degree,
    json_extract(value, '$.degree')     AS degree
FROM json_each(cypher('RETURN degreeCentrality()'));
```

Query the views:

```
SELECT paper, score FROM v_pagerank ORDER BY score DESC;

SELECT community, count(*) AS size FROM v_communities GROUP BY community;

SELECT p.paper, p.score AS pagerank, d.in_degree AS cited_by
FROM v_pagerank p
JOIN v_degree d ON d.paper = p.paper
ORDER BY p.score DESC;
```

Step 10: Combine Algorithm Output with Regular SQL Tables

Create a regular metadata table for papers:

```
CREATE TABLE IF NOT EXISTS paper_metadata (
    title        TEXT PRIMARY KEY,
    authors      TEXT,
    impact_factor REAL
);

INSERT OR IGNORE INTO paper_metadata VALUES
    ('Deep Residual Learning',          'He, Zhang, Ren, Sun', 9.8),
    ('Attention Is All You Need',       'Vaswani et al.',      9.5),
    ('BERT',                            'Devlin et al.',       9.2),
    ('ImageNet Classification with CNN','Krizhevsky et al.',   9.7),
    ('Generative Adversarial Networks', 'Goodfellow et al.',   9.6),
    ('Word2Vec',                        'Mikolov et al.',      9.0),
    ('Graph Convolutional Networks',    'Kipf, Welling',       8.8),
    ('GraphSAGE',                       'Hamilton et al.',     8.5);
```

Join PageRank with metadata:

```
WITH rankings AS (
    SELECT
        json_extract(value, '$.user_id') AS title,
        json_extract(value, '$.score')   AS pagerank
    FROM json_each(cypher('RETURN pageRank(0.85, 20)'))
)
SELECT
    r.title,
    pm.authors,
    printf('%.4f', r.pagerank)    AS pagerank_score,
    pm.impact_factor,
    ROUND(r.pagerank * 10 + pm.impact_factor, 2) AS combined_score
FROM rankings r
JOIN paper_metadata pm ON pm.title = r.title
ORDER BY combined_score DESC;
```

Add community labels to metadata:

```
WITH comm AS (
    SELECT
        json_extract(value, '$.user_id')   AS title,
        json_extract(value, '$.community') AS community
    FROM json_each(cypher('RETURN louvain(1.0)'))
)
SELECT
    pm.title,
    pm.authors,
    c.community AS research_cluster
FROM paper_metadata pm
JOIN comm c ON c.title = pm.title
ORDER BY c.community, pm.title;
```

Step 11: Cache Algorithm Results

For expensive algorithms on large graphs, cache results in a real table:

```
-- Cache PageRank results
DROP TABLE IF EXISTS pagerank_cache;
CREATE TABLE pagerank_cache AS
SELECT
    json_extract(value, '$.node_id')  AS node_id,
    json_extract(value, '$.user_id')  AS paper,
    json_extract(value, '$.score')    AS score,
    datetime('now')                   AS computed_at
FROM json_each(cypher('RETURN pageRank(0.85, 20)'));

CREATE INDEX IF NOT EXISTS idx_pagerank_score ON pagerank_cache(score DESC);

-- Fast repeated queries
SELECT paper, score FROM pagerank_cache ORDER BY score DESC LIMIT 5;
```

Export results to CSV for visualisation:

```
.mode csv
.output pagerank_results.csv
SELECT paper, score FROM pagerank_cache ORDER BY score DESC;
.output stdout
.mode column
```

Next Steps

- Graph Algorithms Reference — Full parameter documentation for every algorithm
- SQL Interface Reference — `cypher()` function, `json_each()` patterns, and schema tables
- Performance — Algorithm complexity and scaling guidance

Building a GraphRAG System

This tutorial builds a complete Graph Retrieval-Augmented Generation (GraphRAG) system by combining GraphQLite with sqlite-vec for vector search and sentence-transformers for text embeddings.

GraphRAG enriches standard vector retrieval with graph traversal — finding not just semantically similar passages, but also the entities they mention and the graph neighbourhood around those entities. The result is richer, more contextual input for language models.

> **Note**: This tutorial requires additional dependencies beyond GraphQLite itself. The complete working example with HotpotQA multi-hop reasoning is in `examples/llm-graphrag/`.

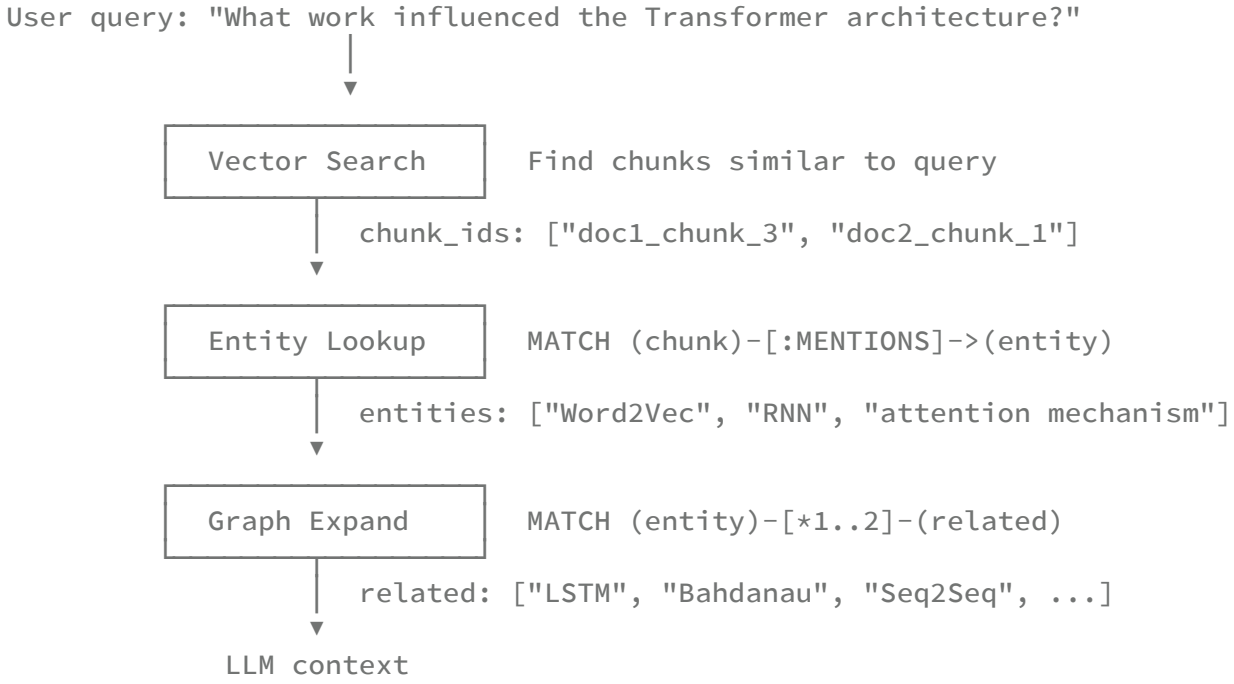
What is GraphRAG?

Traditional RAG:

1. Embed the query
2. Find the most similar document chunks by vector distance
3. Send those chunks to the LLM

GraphRAG adds: 4. Extract entities mentioned in the retrieved chunks 5. Traverse the knowledge graph to find related entities 6. Include the graph neighbourhood as additional context

This matters when answers require connecting information across multiple documents — "Who co-authored papers with the researcher who invented Word2Vec?" requires knowing both the entity graph (researchers, papers) and the text content of specific chunks.



Prerequisites

```
pip install graphqlite sentence-transformers sqlite-vec spacy
python -m spacy download en_core_web_sm
```

Verify:

```
import graphqlite, sqlite_vec, spacy
from sentence_transformers import SentenceTransformer
print("All dependencies available")
```

Step 1: Document Ingestion Helpers

Define the data structures and chunking logic:

```
from __future__ import annotations
from dataclasses import dataclass
from typing import List

@dataclass
class Chunk:
    chunk_id: str
    doc_id:   str
    text:     str

def chunk_text(text: str, doc_id: str, chunk_size: int = 200, overlap: int =
40) -> List[Chunk]:
    """Split text into overlapping word-based chunks."""
    words = text.split()
    chunks = []
    start = 0
    index = 0
    while start < len(words):
        end = min(start + chunk_size, len(words))
        chunk_words = words[start:end]
        chunks.append(Chunk(
            chunk_id=f"{doc_id}_chunk_{index}",
            doc_id=doc_id,
            text=" ".join(chunk_words),
        ))
        start += chunk_size - overlap
        index += 1
    return chunks
```

Step 2: Entity Extraction

Use spaCy to extract named entities and create co-occurrence relationships:

```
import spacy

nlp = spacy.load("en_core_web_sm")

def extract_entities(text: str) -> List[dict]:
    """Return named entities with their type."""
    doc = nlp(text)
    seen = set()
    entities = []
    for ent in doc.ents:
        key = (ent.text.strip(), ent.label_)
        if key not in seen:
            seen.add(key)
            entities.append({"text": ent.text.strip(), "label": ent.label_})
    return entities

def entity_node_id(name: str) -> str:
    """Normalise entity name to a stable node ID."""
    return "ent_" + name.lower().replace(" ", "_").replace(".",
"").replace(",", "")
```

Step 3: Build the Knowledge Graph

```
import sqlite3
import sqlite_vec
from sentence_transformers import SentenceTransformer
from graphqlite import Graph

EMBEDDING_DIM = 384
model = SentenceTransformer("all-MiniLM-L6-v2")

def setup(db_path: str):
    """Initialise GraphQLite and the vector table."""
    g = Graph(db_path)
    conn = sqlite3.connect(db_path)
    sqlite_vec.load(conn)

    conn.execute(f"""
        CREATE VIRTUAL TABLE IF NOT EXISTS chunk_embeddings USING vec0(
            chunk_id  TEXT PRIMARY KEY,
            embedding FLOAT[{EMBEDDING_DIM}]
        )
    """)
    conn.commit()
    return g, conn


def ingest_document(g: Graph, conn: sqlite3.Connection, doc_id: str, text:
str):
    """
    Chunk a document, extract entities, store graph nodes and edges,
    and compute embeddings.
    """
    chunks = chunk_text(text, doc_id=doc_id)

    # Embed all chunks in one batch
    texts = [c.text for c in chunks]
    embeddings = model.encode(texts, show_progress_bar=False)

    for chunk, embedding in zip(chunks, embeddings):
        # Store chunk as a graph node (truncated text for storage efficiency)
        g.upsert_node(
            chunk.chunk_id,
            {"doc_id": doc_id, "text": chunk.text[:1000]},
            label="Chunk"
        )

        # Store embedding
        conn.execute(
            "INSERT OR REPLACE INTO chunk_embeddings (chunk_id, embedding)
VALUES (?, ?)",
            [chunk.chunk_id, embedding.tobytes()]
        )

        # Extract entities from this chunk
        entities = extract_entities(chunk.text)
        for ent in entities:
```

```
            ent_id = entity_node_id(ent["text"])
            g.upsert_node(ent_id, {"name": ent["text"], "type": ent["label"]},
label="Entity")
            g.upsert_edge(chunk.chunk_id, ent_id, {}, rel_type="MENTIONS")

        # Entity co-occurrence: connect entities that appear in the same chunk
        for i, ent_a in enumerate(entities):
            for ent_b in entities[i + 1:]:
                id_a = entity_node_id(ent_a["text"])
                id_b = entity_node_id(ent_b["text"])
                g.upsert_edge(id_a, id_b, {"source_chunk": chunk.chunk_id},
rel_type="CO_OCCURS")

    conn.commit()
    print(f"Ingested {doc_id}: {len(chunks)} chunks, graph now has {g.stats()
['nodes']} nodes")
```

Step 4: Vector Search

```
def vector_search(conn: sqlite3.Connection, query: str, k: int = 5) ->
List[str]:
    """Return the k most semantically similar chunk IDs."""
    query_embedding = model.encode([query])[0]
    rows = conn.execute(
        """
        SELECT chunk_id
        FROM chunk_embeddings
        WHERE embedding MATCH ?
          AND k = ?
        """,
        [query_embedding.tobytes(), k]
    ).fetchall()
    return [row[0] for row in rows]
```

Step 5: GraphRAG Retrieval

```
def graphrag_retrieve(
    g: Graph,
    conn: sqlite3.Connection,
    query: str,
    k_chunks: int = 5,
    expand_hops: int = 2,
) -> dict:
    """
    Hybrid retrieval:
    1. Vector search for semantically similar chunks
    2. Find entities mentioned in those chunks (graph lookup)
    3. Expand to related entities via graph traversal
    4. Return chunks + entity context
    """

    # 1. Vector search
    chunk_ids = vector_search(conn, query, k=k_chunks)
    if not chunk_ids:
        return {"chunks": [], "entities": [], "related_entities": [],
"graph_paths": []}

    # 2. Entity lookup via graph — parameterized query
    entities: set[str] = set()
    for chunk_id in chunk_ids:
        rows = g.connection.cypher(
            "MATCH (c:Chunk {id: $chunk_id})-[:MENTIONS]->(e:Entity) RETURN
e.name AS name",
            {"chunk_id": chunk_id}
        )
        for row in rows:
            entities.add(row["name"])

    # 3. Graph expansion — find related entities
    related_entities: set[str] = set()
    for entity_name in entities:
        ent_id = entity_node_id(entity_name)
        rows = g.connection.cypher(
            """
            MATCH (e:Entity {id: $ent_id})-[*1..$hops]-(related:Entity)
            WHERE related.id <> $ent_id
            RETURN DISTINCT related.name AS name
            """,
            {"ent_id": ent_id, "hops": expand_hops}
        )
        for row in rows:
            related_entities.add(row["name"])

    # 4. Retrieve chunk texts
    chunk_texts = []
    for chunk_id in chunk_ids:
        node = g.get_node(chunk_id)
        if node:
            chunk_texts.append({
                "chunk_id": chunk_id,
```

```
            "doc_id":   node["properties"].get("doc_id", ""),
            "text":     node["properties"].get("text", ""),
        })

return {
    "chunks":           chunk_texts,
    "entities":         sorted(entities),
    "related_entities": sorted(related_entities - entities),
}
```

Step 6: Graph Algorithms for Retrieval Enhancement

Graph algorithms improve retrieval quality in several ways.

PageRank for entity importance

Entities that are heavily co-mentioned across documents are likely central to the corpus topics. Use PageRank to weight entity importance during retrieval.

```
def get_important_entities(g: Graph, top_k: int = 20) -> List[str]:
    """Return top-k entity node IDs by PageRank."""
    g.connection.cypher("RETURN gql_load_graph()")
    results = g.pagerank(damping=0.85, iterations=20)

    important = []
    for r in sorted(results, key=lambda x: x["score"], reverse=True):
        node = g.get_node(r["user_id"])
        if node and node["label"] == "Entity":
            important.append(r["user_id"])
            if len(important) >= top_k:
                break
    return important


def graphrag_retrieve_ranked(
    g: Graph,
    conn: sqlite3.Connection,
    query: str,
    k_chunks: int = 5,
) -> dict:
    """Retrieval that boosts chunks mentioning high-PageRank entities."""
    base = graphrag_retrieve(g, conn, query, k_chunks=k_chunks)
    important_entities = set(get_important_entities(g, top_k=20))

    # Score chunks by how many important entities they mention
    scored_chunks = []
    for chunk in base["chunks"]:
        rows = g.connection.cypher(
            "MATCH (c:Chunk {id: $chunk_id})-[:MENTIONS]->(e:Entity) RETURN
e.id AS eid",
            {"chunk_id": chunk["chunk_id"]}
        )
        ent_ids = {r["eid"] for r in rows}
        boost = len(ent_ids & important_entities)
        scored_chunks.append({**chunk, "importance_boost": boost})

    scored_chunks.sort(key=lambda x: x["importance_boost"], reverse=True)
    base["chunks"] = scored_chunks
    return base
```

Community detection for topic-aware retrieval

Detect research clusters to route queries to the right part of the graph:

```python
def get_entity_communities(g: Graph) -> dict[str, int]:
    """Return a mapping of entity node_id -> community ID."""
    g.connection.cypher("RETURN gql_load_graph()")
    results = g.community_detection(iterations=10)
    return {r["user_id"]: r["community"] for r in results}


def retrieve_by_community(g: Graph, entity_name: str) -> List[str]:
    """Find all entity names in the same community as a given entity."""
    communities = get_entity_communities(g)
    ent_id = entity_node_id(entity_name)
    if ent_id not in communities:
        return []
    target_community = communities[ent_id]
    return [
        uid for uid, cid in communities.items()
        if cid == target_community and uid != ent_id
    ]
```

Step 7: Complete Pipeline

```
# ---- Initialise ----
g, conn = setup("graphrag.db")

# ---- Ingest Documents ----
documents = [
    {
        "id": "vaswani2017",
        "text": (
            "Attention Is All You Need. Vaswani et al., Google Brain, 2017. "
            "We propose the Transformer, a model architecture eschewing
recurrence "
            "and instead relying entirely on an attention mechanism to draw
global "
            "dependencies between input and output. The Transformer allows for
"
            "significantly more parallelization than recurrent architectures. "
            "Our model is trained on the WMT 2014 English-German and English-
French "
            "translation tasks and achieves state-of-the-art results."
        )
    },
    {
        "id": "mikolov2013",
        "text": (
            "Distributed Representations of Words and Phrases and their
Compositionality. "
            "Mikolov et al., Google, 2013. "
            "We present several extensions of the original Word2Vec model that
improve "
            "quality of the vectors and training speed. We show that
subsampling of "
            "frequent words during training results in significant speedup."
        )
    },
    {
        "id": "devlin2018",
        "text": (
            "BERT: Pre-training of Deep Bidirectional Transformers for Language
Understanding. "
            "Devlin, Chang, Lee, Toutanova, Google AI Language, 2018. "
            "We introduce BERT, a new language representation model. BERT
stands for "
            "Bidirectional Encoder Representations from Transformers. BERT is
designed "
            "to pre-train deep bidirectional representations from unlabeled
text by "
            "jointly conditioning on both left and right context in all
layers."
        )
    },
]

for doc in documents:
    ingest_document(g, conn, doc["id"], doc["text"])
```

```
print(f"\nFinal graph: {g.stats()}")

# ---- Retrieve ----
query = "What mechanisms replaced recurrence in language models?"
context = graphrag_retrieve(g, conn, query, k_chunks=3, expand_hops=2)

print(f"\nQuery: {query}")
print(f"Retrieved {len(context['chunks'])} chunks")
print(f"Direct entities: {context['entities']}")
print(f"Related entities: {context['related_entities']}")

print("\nChunk texts:")
for chunk in context["chunks"]:
    print(f"  [{chunk['doc_id']}] {chunk['text'][:120]}...")

# ---- Build prompt ----
def build_prompt(query: str, context: dict) -> str:
    chunk_text = "\n\n".join(
        f"[{c['doc_id']}] {c['text']}" for c in context["chunks"]
    )
    entity_context = ", ".join(context["entities"])
    related_context = ", ".join(context["related_entities"])

    return f"""Answer the following question using only the provided context.

Question: {query}

Relevant text passages:
{chunk_text}

Key entities mentioned: {entity_context}
Related entities from knowledge graph: {related_context}

Answer:"""

prompt = build_prompt(query, context)
print("\n--- Prompt ---")
print(prompt[:600], "...")
```

Step 8: Working with the Example Project

The `examples/llm-graphrag/` directory contains a complete production-grade implementation:

- Ingests the HotpotQA multi-hop reasoning dataset
- Uses Ollama for local LLM inference (no API keys required)
- Demonstrates multi-hop question answering where the answer requires connecting information across multiple documents

```
cd examples/llm-graphrag

# Install dependencies (uses uv)
uv sync

# Ingest the HotpotQA dataset
uv run python ingest.py

# Interactive query mode
uv run python rag.py
```

The ingest script builds a graph of ~10,000 Wikipedia article chunks with entity co-occurrence relationships. The rag script accepts questions and returns answers with citations.

For a deeper discussion of when to use GraphRAG vs plain RAG, graph schema design trade-offs, and embedding model selection, see the Architecture explanation.

Next Steps

- Graph Analytics — Deep dive into PageRank, community detection, and other algorithms used here
- Graph Algorithms Reference — Full algorithm parameter documentation
- Python API Reference — `Graph`, `Connection`, and `GraphManager` API
- Use with Other Extensions — Loading sqlite-vec alongside GraphQLite

Installation

GraphQLite is a SQLite extension that adds Cypher graph query support. It can be used from Python, Rust, the command line, or any language that can load SQLite extensions.

Python (Recommended)

Install from PyPI:

```
pip install graphqlite
```

This installs pre-built binaries for:

- macOS (arm64, x86_64)
- Linux (x86_64, aarch64)
- Windows (x86_64)

Optional Dependencies

The Leiden community detection algorithm requires `graspologic`:

```
pip install graphqlite[leiden]
```

For rustworkx-powered graph export:

```
pip install graphqlite[rustworkx]
```

Install everything:

```
pip install graphqlite[leiden,rustworkx]
```

Rust

Add to your `Cargo.toml`:

```
[dependencies]
graphqlite = "0.3"
```

The `bundled-extension` feature is enabled by default. It bundles the compiled extension directly into your binary so no separate `.so` / `.dylib` file is needed at runtime:

```
[dependencies]
graphqlite = { version = "0.3", features = ["bundled-extension"] }
```

To use an external extension at runtime instead, disable the default features:

```
[dependencies]
graphqlite = { version = "0.3", default-features = false }
```

From Source

Building from source produces the raw extension file you can load into any SQLite environment.

Prerequisites

Tool	Minimum Version	Purpose
GCC or Clang	9+	C compiler
Bison	3.0+	Parser generator
Flex	2.6+	Lexer generator
SQLite development headers	3.35+	SQLite API
CUnit	2.1+	Unit tests (optional)

macOS

```
brew install bison flex sqlite cunit
export PATH="$(brew --prefix bison)/bin:$PATH"
git clone https://github.com/colliery-io/graphqlite
cd graphqlite
make extension
```

Homebrew installs a newer Bison than the one shipped with macOS. The `PATH` export ensures the build system picks up the correct version.

Linux (Debian/Ubuntu)

```
sudo apt-get install build-essential bison flex libsqlite3-dev libcunit1-dev
git clone https://github.com/colliery-io/graphqlite
cd graphqlite
make extension
```

Windows (MSYS2)

```
pacman -S mingw-w64-x86_64-gcc mingw-w64-x86_64-sqlite3 bison flex make
git clone https://github.com/colliery-io/graphqlite
cd graphqlite
make extension
```

Debug vs. Release Build

The default build includes debug symbols and assertions. For a release build:

```
make extension RELEASE=1
```

Release builds enable `-O2` optimization and strip assertions. Use them for production deployments.

Output Files

After building, the extension appears in `build/`:

Platform	File
macOS	`build/graphqlite.dylib`
Linux	`build/graphqlite.so`
Windows	`build/graphqlite.dll`

See Building from Source for the full list of build targets and how to run tests.

Verifying Installation

Python

```python
import graphqlite

# Check version
print(graphqlite.__version__)

# Quick smoke test
from graphqlite import Graph

g = Graph(":memory:")
g.upsert_node("alice", {"name": "Alice", "age": 30}, label="Person")
g.upsert_node("bob", {"name": "Bob", "age": 25}, label="Person")
g.upsert_edge("alice", "bob", {"since": 2020}, rel_type="KNOWS")

print(g.stats())          # {'nodes': 2, 'edges': 1}
print(g.query("MATCH (n:Person) RETURN n.name ORDER BY n.name"))
```

Rust

```rust
use graphqlite::Connection;

fn main() -> graphqlite::Result<()> {
    let conn = Connection::open_in_memory()?;
    conn.cypher("CREATE (n:Person {name: 'Alice'})")?;
    let rows = conn.cypher("MATCH (n:Person) RETURN n.name")?;
    for row in &rows {
        println!("{}", row.get::<String>("n.name")?);
    }
    Ok(())
}
```

SQL (raw SQLite)

```sql
sqlite3
.load /path/to/build/graphqlite
SELECT cypher('RETURN 1 + 1 AS result');
SELECT cypher('CREATE (n:Test {value: 42})');
SELECT cypher('MATCH (n:Test) RETURN n.value');
```

Extension Path Configuration

When GraphQLite cannot find the extension automatically, specify the path explicitly.

Environment Variable

Set `GRAPHQLITE_EXTENSION_PATH` before running your application:

```
export GRAPHQLITE_EXTENSION_PATH=/path/to/build/graphqlite.dylib
python my_app.py
```

This works for Python, the CLI (`gqlite`), and any program that reads environment variables before loading the extension.

Python: Explicit Path

```
from graphqlite import connect, Graph, GraphManager

# connect()
conn = connect("graph.db", extension_path="/path/to/graphqlite.dylib")

# Graph()
g = Graph("graph.db", extension_path="/path/to/graphqlite.dylib")

# GraphManager / graphs()
from graphqlite import graphs
with graphs("./data", extension_path="/path/to/graphqlite.dylib") as gm:
    social = gm.open_or_create("social")
```

Finding the Bundled Path

When using the Python package, GraphQLite ships a bundled extension. Get its path:

```
import graphqlite

path = graphqlite.loadable_path()
print(path)  # e.g. /usr/local/lib/python3.12/site-
packages/graphqlite/graphqlite.dylib
```

You can pass this path to other tools or languages that need to load the extension manually.

Platform Notes

macOS

- The extension is a Mach-O shared library with the `.dylib` suffix.
- If you see `Library not loaded` errors, ensure you are using Homebrew's SQLite rather than the system SQLite:

```
export DYLD_LIBRARY_PATH="$(brew --prefix sqlite)/lib:$DYLD_LIBRARY_PATH"
```

- Apple Silicon (M1/M2/M3) and Intel builds are separate; PyPI ships a universal wheel containing both.

Linux

- The extension is an ELF shared object with the `.so` suffix.
- Make sure `libsqlite3` is installed at runtime (not just `libsqlite3-dev`).
- On musl-based systems (Alpine Linux), build from source.

Windows

- The extension is a DLL with the `.dll` suffix.
- Load it with the full path: `.load C:/path/to/graphqlite`.
- MSYS2 or WSL are the supported build environments.

Troubleshooting

Extension not found (Python)

```
FileNotFoundError: GraphQLite extension not found
```

Either the package was installed without wheels for your platform, or the environment variable points to the wrong file. Try:

```
import graphqlite
print(graphqlite.loadable_path())  # See where Python looks
```

Then set `GRAPHQLITE_EXTENSION_PATH` or pass `extension_path` explicitly.

Python sqlite3 extension support disabled

Some Python distributions compile `sqlite3` without extension loading support (e.g., certain Docker images):

```
AttributeError: 'sqlite3.Connection' object has no attribute
'enable_load_extension'
```

Rebuild Python from source with `--enable-loadable-sqlite-extensions`, or switch to a distribution that includes it (standard CPython builds from python.org do support it).

Version mismatch

If you see unexpected query errors after upgrading, ensure the Python package and the loaded extension are from the same version:

```
import graphqlite
print(graphqlite.__version__)   # Python package version
```

When building from source for use with the Rust crate, run `make install-bundled` after `make extension` to replace the bundled binary in the Rust crate's source tree.

SQLite version too old

GraphQLite requires SQLite 3.35 or later for JSON function support. Check your SQLite version:

```
import sqlite3
print(sqlite3.sqlite_version)  # Should be 3.35.0 or higher
```

Working with Multiple Graphs

GraphQLite supports managing and querying across multiple graph databases. This is useful for:

- **Separation of concerns**: Keep different data domains in separate graphs.
- **Access control**: Different graphs can have different file-level permissions.
- **Performance**: Smaller, focused graphs are faster to query and analyze.
- **Cross-domain queries**: Join relationships that span different datasets.

Python: GraphManager

The `GraphManager` class (accessed via the `graphs()` factory function) manages multiple graph databases stored as separate SQLite files in a directory.

Creating and Opening Graphs

```
from graphqlite import graphs

with graphs("./data") as gm:
    # Create new graphs (raises FileExistsError if already exists)
    social = gm.create("social")
    products = gm.create("products")

    # Populate them
    social.upsert_node("alice", {"name": "Alice", "age": 30, "user_id": "u1"},
"Person")
    social.upsert_node("bob", {"name": "Bob", "age": 25, "user_id": "u2"},
"Person")
    social.upsert_edge("alice", "bob", {"since": 2020}, "KNOWS")

    products.upsert_node("phone", {"name": "iPhone 15", "price": 999},
"Product")
    products.upsert_node("laptop", {"name": "MacBook Pro", "price": 1999},
"Product")

    print(gm.list())       # ['products', 'social']
    print(len(gm))         # 2
    print("social" in gm)  # True
```

All connections are closed automatically when the `with` block exits.

Opening Existing Graphs

```
from graphqlite import graphs

with graphs("./data") as gm:
    # Open an existing graph (raises FileNotFoundError if missing)
    social = gm.open("social")

    # Open, creating if it doesn't exist
    cache = gm.open_or_create("cache")

    for row in social.query("MATCH (n:Person) RETURN n.name ORDER BY n.name"):
        print(row["n.name"])
```

Listing and Dropping Graphs

```
from graphqlite import graphs

with graphs("./data") as gm:
    # List all graph names in the directory
    for name in gm.list():
        g = gm.open(name)
        print(f"{name}: {g.stats()}")

    # Delete a graph and its database file permanently
    gm.drop("cache")
```

`drop()` deletes the `.db` file on disk. There is no undo.

Cross-Graph Queries

GraphQLite can query across multiple graphs in a single Cypher statement using the `FROM` clause.

The FROM Clause

Attach one or more graphs and reference them by name in `MATCH` patterns:

```
from graphqlite import graphs

with graphs("./data") as gm:
    social = gm.open_or_create("social")
    social.upsert_node("alice", {"name": "Alice", "user_id": "u1"}, "Person")

    purchases = gm.open_or_create("purchases")
    purchases.upsert_node("order1", {"user_id": "u1", "total": 99.99, "item":
"Phone"}, "Order")

    # GraphManager commits open graphs before running cross-graph queries
    result = gm.query(
        """
        MATCH (p:Person) FROM social
        WHERE p.user_id = 'u1'
        RETURN p.name, graph(p) AS source
        """,
        graphs=["social"]
    )

    for row in result:
        print(f"{row['p.name']} is from graph: {row['source']}")
```

The `graphs` parameter tells `gm.query()` which databases to attach before running the query.

The graph() Function

`graph(node)` returns the name of the graph that the node comes from. Use it to identify results in multi-graph queries:

```
result = gm.query(
    """
    MATCH (n) FROM social
    RETURN n.name, graph(n) AS source_graph
    """,
    graphs=["social"]
)

for row in result:
    print(f"{row['n.name']} lives in {row['source_graph']}")
```

Cross-Graph Queries with Parameters

Pass parameters to cross-graph queries the same way as single-graph queries:

```
result = gm.query(
    "MATCH (n:Person {user_id: $uid}) FROM social RETURN n.name",
    graphs=["social"],
    params={"uid": "u1"}
)
```

Raw SQL Cross-Graph Queries

For low-level access, `query_sql()` attaches the named graphs and runs raw SQL. The attached graph's tables are prefixed with the graph name:

```
# Count nodes in the social graph
result = gm.query_sql(
    "SELECT COUNT(*) AS node_count FROM social.nodes",
    graphs=["social"]
)
print(f"Social graph has {result[0][0]} nodes")

# Join across graphs with raw SQL
result = gm.query_sql(
    """
    SELECT s.user_id, COUNT(p.rowid) AS order_count
    FROM social.node_props_text s
    JOIN purchases.node_props_text p ON s.value = p.value
    WHERE s.key = 'user_id' AND p.key = 'user_id'
    GROUP BY s.user_id
    """,
    graphs=["social", "purchases"]
)
```

`query_sql()` is useful for analytics that go beyond what Cypher exposes, such as aggregations across multiple graph schemas at once.

Important: Commit Before Cross-Graph Queries

`GraphManager` automatically commits all open graph connections before running cross-graph queries with `query()` or `query_sql()`. If you are using the underlying connections directly, commit first:

```
social.connection.execute("COMMIT")
result = gm.query("MATCH (n) FROM social RETURN n", graphs=["social"])
```

Rust: GraphManager

The Rust API mirrors the Python API closely.

```
use graphqlite::graphs;

fn main() -> graphqlite::Result<()> {
    let mut gm = graphs("./data")?;

    // Create graphs
    {
        let social = gm.create("social")?;
        social.query("CREATE (n:Person {name: 'Alice', user_id: 'u1'})")?;
        social.query("CREATE (n:Person {name: 'Bob', user_id: 'u2'})")?;
        social.query(
            "MATCH (a:Person {name: 'Alice'}), (b:Person {name: 'Bob'}) \
             CREATE (a)-[:KNOWS {since: 2020}]->(b)"
        )?;
    }

    {
        let products = gm.create("products")?;
        products.query("CREATE (n:Product {name: 'Phone', sku: 'p1', price:
999})")?;
    }

    // List all graphs
    for name in gm.list()? {
        println!("Graph: {}", name);
    }

    // Open an existing graph
    let social = gm.open_graph("social")?;
    let stats = social.stats()?;
    println!("Social: {} nodes, {} edges", stats.nodes, stats.edges);

    // Cross-graph query using FROM clause
    let result = gm.query(
        "MATCH (n:Person) FROM social RETURN n.name AS name ORDER BY n.name",
        &["social"],
    )?;

    for row in &result {
        println!("Person: {}", row.get::<String>("name")?);
    }

    // Raw SQL cross-graph query
    let counts = gm.query_sql(
        "SELECT COUNT(*) FROM social.nodes",
        &["social"],
    )?;

    // Open or create (idempotent)
    let _cache = gm.open_or_create("cache")?;

    // Drop a graph
    gm.drop("products")?;

    // Error handling for missing graphs
    match gm.open_graph("nonexistent") {
        Err(graphqlite::Error::GraphNotFound { name, available }) => {
```

```
                println!("'{}' not found. Available: {:?}", name, available);
            }
            _ => {}
        }

        Ok(())
    }
```

Rust GraphManager Methods

Method	Description
`gm.create("name")`	Create a new graph; errors if it already exists
`gm.open_graph("name")`	Open an existing graph; errors if missing
`gm.open_or_create("name")`	Open or create idempotently
`gm.list()?`	Returns `Vec<String>` of graph names
`gm.exists("name")`	Returns `bool`
`gm.drop("name")?`	Delete graph and its file
`gm.query(cypher, graphs)?`	Cross-graph Cypher query
`gm.query_sql(sql, graphs)?`	Cross-graph raw SQL query

Using ATTACH Directly

For complete control, attach databases manually using SQLite's `ATTACH` mechanism:

```
import sqlite3
import graphqlite

# Build each graph
conn1 = sqlite3.connect("social.db")
graphqlite.load(conn1)
conn1.execute("SELECT cypher('CREATE (n:Person {name: \"Alice\"})')")
conn1.commit()
conn1.close()

conn2 = sqlite3.connect("products.db")
graphqlite.load(conn2)
conn2.execute("SELECT cypher('CREATE (n:Product {name: \"Phone\"})')")
conn2.commit()
conn2.close()

# Query across both
coordinator = sqlite3.connect(":memory:")
graphqlite.load(coordinator)
coordinator.execute("ATTACH DATABASE 'social.db' AS social")
coordinator.execute("ATTACH DATABASE 'products.db' AS products")

result = coordinator.execute(
    "SELECT cypher('MATCH (n:Person) FROM social RETURN n.name')"
).fetchall()
print(result)
```

Best Practices

1. **Use the context manager.** `with graphs(...) as gm:` ensures all connections are closed and any pending transactions are flushed.
2. **Commit before cross-graph queries.** GraphManager handles this automatically, but manual connections do not.
3. **Use valid SQL identifiers for graph names.** Graph names become SQLite database aliases (`ATTACH ... AS name`). Use lowercase letters, digits, and underscores only — no hyphens or spaces.
4. **Keep graphs focused.** Design each graph around a single domain or service boundary. Cross-graph queries are read-only for the attached graphs.
5. **Use the same extension version.** All attached graphs should be queried with the same version of the GraphQLite extension to avoid schema incompatibilities.

Limitations

- Cross-graph `FROM` clause queries are read-only for attached graphs.
- The `FROM` clause is only supported inside `MATCH` patterns.
- SQLite supports a maximum of approximately 10 simultaneously attached databases.
- Graph names must be valid SQL identifiers (alphanumeric and underscores).

Using the gqlite CLI

`gqlite` is an interactive Cypher shell for querying GraphQLite databases from the command line. It supports interactive mode, multi-line statements, dot commands, and script execution.

Building

Build the CLI from source:

```
make graphqlite
```

For a release build with optimizations:

```
make graphqlite RELEASE=1
```

The binary is placed at `build/gqlite`.

See Building from Source for prerequisites.

Command Line Options

```
Usage: build/gqlite [OPTIONS] [DATABASE_FILE]
```

Option	Description
`-h, --help`	Show help message and exit
`-v, --verbose`	Enable verbose debug output (shows query execution details)
`-i, --init`	Initialize a fresh database (overwrites existing)
`DATABASE_FILE`	Path to the SQLite database file (default: `graphqlite.db`)

Examples

```
# Open the default database (graphqlite.db in the current directory)
./build/gqlite

# Open a specific file
./build/gqlite social.db

# Initialize a fresh database, discarding any existing content
./build/gqlite -i social.db

# Verbose mode — shows row counts and execution details
./build/gqlite -v social.db
```

Interactive Mode

Start `gqlite` without piping stdin to enter interactive mode:

```
GraphQLite Interactive Shell
Type .help for help, .quit to exit
Queries must end with semicolon (;)

graphqlite>
```

Statement Termination

All Cypher statements must end with a semicolon (`;`). `gqlite` buffers input across multiple lines until it sees a `;` .

```
graphqlite> CREATE (a:Person {name: "Alice", age: 30});
Query executed successfully
  Nodes created: 1
  Properties set: 2

graphqlite> MATCH (n:Person) RETURN n.name, n.age;
n.name    n.age
----------
Alice     30
```

Multi-Line Statements

Press Enter to continue a statement on the next line. A `...>` prompt indicates continuation:

```
graphqlite> MATCH (a:Person {name: "Alice"}), (b:Person {name: "Bob"})
       ...>   CREATE (a)-[:KNOWS {since: 2021}]->(b);
Query executed successfully
  Relationships created: 1
```

Dot Commands

Dot commands control the shell itself. They do not end with a semicolon.

Command	Description
`.help`	Show all available commands
`.schema`	Display the full database schema
`.tables`	List all tables in the database
`.stats`	Show graph statistics (node count, edge count, labels, types)
`.quit`	Exit the shell
`.exit`	Alias for `.quit`

Example: .stats

```
graphqlite> .stats

Database Statistics:
====================
  Nodes           : 3
  Edges           : 2
  Node Labels     : 1
  Property Keys   : 2
  Edge Types      : KNOWS
```

Example: .schema

```
graphqlite> .schema
CREATE TABLE nodes (rowid INTEGER PRIMARY KEY, user_id TEXT UNIQUE, label
TEXT);
CREATE TABLE edges (rowid INTEGER PRIMARY KEY, source_id INTEGER, target_id
INTEGER, ...);
...
```

Script Mode

Pipe a file or inline text to `gqlite` to run a script non-interactively:

```
# Execute a script file
./build/gqlite social.db < setup.cypher

# Inline heredoc
./build/gqlite social.db <<'EOF'
CREATE (alice:Person {name: "Alice", age: 30});
CREATE (bob:Person {name: "Bob", age: 25});
MATCH (a:Person {name: "Alice"}), (b:Person {name: "Bob"})
  CREATE (a)-[:KNOWS {since: 2020}]->(b);
MATCH (a:Person)-[:KNOWS]->(b:Person) RETURN a.name, b.name;
EOF

# Inline echo
echo 'MATCH (n) RETURN n.name;' | ./build/gqlite social.db
```

Script Format

Scripts use the same semicolon-terminated syntax as the interactive shell. Use `--` for comments:

```
-- setup.cypher
-- Create people
CREATE (alice:Person {name: "Alice", age: 30, city: "London"});
CREATE (bob:Person {name: "Bob", age: 25, city: "Paris"});
CREATE (carol:Person {name: "Carol", age: 35, city: "Berlin"});

-- Create relationships
MATCH (a:Person {name: "Alice"}), (b:Person {name: "Bob"})
  CREATE (a)-[:KNOWS {since: 2018}]->(b);

MATCH (b:Person {name: "Bob"}), (c:Person {name: "Carol"})
  CREATE (b)-[:KNOWS {since: 2021}]->(c);

-- Query friend-of-friend
MATCH (a:Person {name: "Alice"})-[:KNOWS]->()-[:KNOWS]->(fof)
RETURN fof.name AS friend_of_friend;
```

Worked Example: Build and Query a Graph

The following session creates a small knowledge graph, queries it, and inspects statistics.

Step 1: Initialize the Database

```
./build/gqlite -i company.db
```

```
GraphQLite Interactive Shell
Type .help for help, .quit to exit
Queries must end with semicolon (;)

graphqlite>
```

Step 2: Add People

```
graphqlite> CREATE (alice:Person {name: "Alice", title: "Engineer"});
Query executed successfully
  Nodes created: 1
  Properties set: 2

graphqlite> CREATE (bob:Person {name: "Bob", title: "Manager"});
Query executed successfully
  Nodes created: 1
  Properties set: 2

graphqlite> CREATE (carol:Person {name: "Carol", title: "Engineer"});
Query executed successfully
  Nodes created: 1
  Properties set: 2
```

Step 3: Add a Project Node

```
graphqlite> CREATE (proj:Project {name: "GraphQLite", status: "active"});
Query executed successfully
  Nodes created: 1
  Properties set: 2
```

Step 4: Create Relationships

```
graphqlite> MATCH (alice:Person {name: "Alice"}), (proj:Project {name:
"GraphQLite"})
       ...>   CREATE (alice)-[:WORKS_ON {since: 2023}]->(proj);
Query executed successfully
  Relationships created: 1

graphqlite> MATCH (carol:Person {name: "Carol"}), (proj:Project {name:
"GraphQLite"})
       ...>   CREATE (carol)-[:WORKS_ON {since: 2024}]->(proj);
Query executed successfully
  Relationships created: 1

graphqlite> MATCH (bob:Person {name: "Bob"}), (alice:Person {name: "Alice"})
       ...>   CREATE (bob)-[:MANAGES]->(alice);
Query executed successfully
  Relationships created: 1
```

Step 5: Query the Graph

```
graphqlite> MATCH (mgr:Person)-[:MANAGES]->(eng:Person)-[:WORKS_ON]->
(p:Project)
       ...>   RETURN mgr.name AS manager, eng.name AS engineer, p.name AS
project;
manager    engineer    project
-----------------------------------
Bob        Alice       GraphQLite
```

Step 6: Check Statistics

```
graphqlite> .stats

Database Statistics:
====================
  Nodes           : 4
  Edges           : 3
  Node Labels     : 2
  Property Keys   : 6
  Edge Types      : MANAGES, WORKS_ON

graphqlite> .quit
Goodbye!
```

Tips

- Use `.tables` to see all underlying SQLite tables, which is useful for debugging the graph schema.
- In verbose mode (`-v`), each query prints timing and row count information.
- The CLI loads the bundled extension automatically; no separate `GRAPHQLITE_EXTENSION_PATH` configuration is needed.
- For large imports, use Bulk Import from Python rather than piping thousands of `CREATE` statements through the CLI.

Using Graph Algorithms

GraphQLite includes 18 built-in graph algorithms covering centrality, community detection, path finding, connectivity, traversal, and similarity. All algorithms run inside the SQLite process — no external graph engine is required.

Setup

All examples in this guide use the same sample graph:

```
from graphqlite import Graph

g = Graph(":memory:")

# Nodes
for name, role in [
    ("alice", "engineer"), ("bob", "manager"), ("carol", "engineer"),
    ("dave", "analyst"), ("eve", "engineer"), ("frank", "manager"),
]:
    g.upsert_node(name, {"name": name.capitalize(), "role": role}, "Person")

# Edges
for src, dst, weight in [
    ("alice", "bob", 1), ("alice", "carol", 2), ("bob", "dave", 1),
    ("carol", "dave", 3), ("dave", "eve", 1), ("eve", "frank", 2),
    ("frank", "alice", 4), ("bob", "eve", 1),
]:
    g.upsert_edge(src, dst, {"weight": weight}, "KNOWS")
```

Graph Cache Management

Before running algorithms on large graphs, load the graph into the in-memory cache. This avoids redundant disk reads and speeds up repeated algorithm calls significantly.

Python

```
# Load into cache
g.load_graph()

# Check whether the cache is populated
print(g.graph_loaded())  # True

# Run algorithms — all use the cached graph
pr = g.pagerank()
cc = g.community_detection()

# Reload cache after data changes
g.upsert_node("grace", {"name": "Grace"}, "Person")
g.reload_graph()

# Free memory when done
g.unload_graph()
print(g.graph_loaded())  # False
```

SQL

```
-- Load cache
SELECT gql_load_graph();

-- Check status
SELECT gql_graph_loaded();  -- 1 or 0

-- Reload after changes
SELECT gql_reload_graph();

-- Unload
SELECT gql_unload_graph();
```

> For small graphs (under ~10 000 nodes) the cache provides modest gains. For larger graphs, always call `load_graph()` before running multiple algorithms.

Centrality Algorithms

Centrality algorithms measure the relative importance of nodes in the graph.

PageRank

Assigns scores based on the number and quality of incoming edges. Nodes linked from highly-scored nodes receive higher scores.

When to use: Ranking nodes by overall influence or authority (web pages, citations, recommendation networks).

Python:

```
results = g.pagerank(damping=0.85, iterations=20)
# [{"node_id": "alice", "score": 0.21}, ...]

top5 = sorted(results, key=lambda r: r["score"], reverse=True)[:5]
for r in top5:
    print(f"{r['node_id']}: {r['score']:.4f}")
```

Rust:

```
let results = g.pagerank(0.85, 20)?;
for r in &results {
    println!("{}: {:.4}", r.node_id, r.score);
}
```

SQL:

```
SELECT
    json_extract(value, '$.node_id') AS node,
    json_extract(value, '$.score')   AS score
FROM json_each(cypher('RETURN pageRank(0.85, 20)'))
ORDER BY score DESC
LIMIT 5;
```

Parameters: `damping` (default 0.85), `iterations` (default 20).

Degree Centrality

Counts in-degree, out-degree, and total degree for every node.

When to use: Quick identification of hubs and leaf nodes; baseline for other analyses.

Python:

```
results = g.degree_centrality()
# [{"node_id": "alice", "in_degree": 1, "out_degree": 2, "degree": 3}, ...]

# Find the highest out-degree node
hub = max(results, key=lambda r: r["out_degree"])
print(f"Top hub: {hub['node_id']} with {hub['out_degree']} outgoing edges")
```

Rust:

```
let results = g.degree_centrality()?;
for r in &results {
    println!("{}: in={}, out={}, total={}", r.node_id, r.in_degree,
r.out_degree, r.degree);
}
```

SQL:

```
SELECT
    json_extract(value, '$.node_id')    AS node,
    json_extract(value, '$.out_degree') AS out_degree
FROM json_each(cypher('RETURN degreeCentrality()'))
ORDER BY out_degree DESC;
```

Betweenness Centrality

Measures how often a node appears on shortest paths between other node pairs.

When to use: Identifying bottlenecks, bridges, or brokers in a network (infrastructure nodes, information brokers).

Python:

```
results = g.betweenness_centrality()
# [{"node_id": "dave", "score": 0.45}, ...]

for r in sorted(results, key=lambda r: r["score"], reverse=True):
    print(f"{r['node_id']}: {r['score']:.4f}")
```

Rust:

```
let results = g.betweenness_centrality()?;
for r in &results {
    println!("{}: {:.4}", r.node_id, r.score);
}
```

SQL:

```
SELECT
    json_extract(value, '$.node_id') AS node,
    json_extract(value, '$.score')   AS score
FROM json_each(cypher('RETURN betweennessCentrality()'))
ORDER BY score DESC;
```

> Betweenness is O(n * m) and can be slow on graphs with hundreds of thousands of edges.

Closeness Centrality

Measures how quickly a node can reach all other nodes (inverse of average shortest path length).

When to use: Finding nodes that can spread information fastest, or nodes most central to communication.

Python:

```
results = g.closeness_centrality()
# [{"node_id": "alice", "score": 0.71}, ...]
```

Rust:

```
let results = g.closeness_centrality()?;
```

SQL:

```
SELECT
    json_extract(value, '$.node_id') AS node,
    json_extract(value, '$.score')   AS score
FROM json_each(cypher('RETURN closenessCentrality()'))
ORDER BY score DESC;
```

Eigenvector Centrality

Assigns scores iteratively: a node is important if it is connected to other important nodes.

When to use: Influence scoring in social networks; pages linked from authoritative sources.

Python:

```
results = g.eigenvector_centrality(iterations=100)
# [{"node_id": "alice", "score": 0.55}, ...]
```

Rust:

```
let results = g.eigenvector_centrality(100)?;
```

SQL:

```
SELECT
    json_extract(value, '$.node_id') AS node,
    json_extract(value, '$.score')   AS score
FROM json_each(cypher('RETURN eigenvectorCentrality(100)'))
ORDER BY score DESC;
```

Parameters: `iterations` (default 100) — the algorithm stops when scores converge or iterations are exhausted.

Community Detection

Community detection partitions nodes into clusters based on edge density.

Label Propagation (community_detection)

Nodes iteratively adopt the label most common among their neighbors. Fast and approximate.

When to use: Quick community discovery on large graphs; when exact modularity optimization is not required.

Python:

```
results = g.community_detection(iterations=10)
# [{"node_id": "alice", "community": 1}, ...]

# Group nodes by community
from collections import defaultdict
communities = defaultdict(list)
for r in results:
    communities[r["community"]].append(r["node_id"])

for cid, members in communities.items():
    print(f"Community {cid}: {members}")
```

Rust:

```
let results = g.community_detection(10)?;
for r in &results {
    println!("{} -> community {}", r.node_id, r.community);
}
```

SQL:

```
SELECT
    json_extract(value, '$.node_id')   AS node,
    json_extract(value, '$.community') AS community
FROM json_each(cypher('RETURN labelPropagation(10)'))
ORDER BY community;
```

Louvain

Hierarchical community detection that optimizes modularity. Produces higher-quality communities than label propagation at the cost of more computation.

When to use: When community quality matters more than speed; medium-sized graphs (under ~100 000 nodes).

Python:

```
results = g.louvain(resolution=1.0)
# [{"node_id": "alice", "community": 0}, ...]

# Higher resolution = more, smaller communities
fine_grained = g.louvain(resolution=2.0)
```

Rust:

```
let results = g.louvain(1.0)?;
```

SQL:

```
SELECT
    json_extract(value, '$.node_id')   AS node,
    json_extract(value, '$.community') AS community
FROM json_each(cypher('RETURN louvain(1.0)'))
ORDER BY community;
```

Parameters: `resolution` (default 1.0) — increase to find more communities; decrease to merge communities.

Leiden Communities

An improved variant of Louvain that guarantees well-connected communities and avoids the resolution limit problem.

When to use: When Louvain produces communities that seem internally disconnected, or for publication-quality community detection.

Requires: `pip install graspologic` (or `pip install graphqlite[leiden]`).

Python:

```
# Install: pip install graphqlite[leiden]
results = g.leiden_communities(resolution=1.0, random_seed=42)
# [{"node_id": "alice", "community": 0}, ...]
```

Leiden is not available via the Cypher `RETURN` interface; use the Python or Rust Graph API.

Path Finding

Shortest Path (Dijkstra)

Finds the minimum-weight path between two nodes.

When to use: Navigation, network routing, social distance ("degrees of separation").

Python:

```
result = g.shortest_path("alice", "frank")
# {"distance": 3, "path": ["alice", "bob", "dave", "eve", "frank"], "found":
True}

if result["found"]:
    print(f"Distance: {result['distance']}")
    print(f"Path: {' -> '.join(result['path'])}")
else:
    print("No path found")

# With weighted edges
result = g.shortest_path("alice", "frank", weight_property="weight")
```

Rust:

```
let result = g.shortest_path("alice", "frank", None)?;  // None = unweighted
if result.found {
    println!("Distance: {:?}", result.distance);
    println!("Path: {:?}", result.path);
}

// Weighted
let result = g.shortest_path("alice", "frank", Some("weight"))?;
```

SQL:

```
SELECT cypher('RETURN dijkstra(''alice'', ''frank'')');
```

A* Search

Shortest path with a heuristic to guide the search. When node latitude/longitude properties are available, uses haversine distance as the heuristic, which can dramatically reduce nodes explored.

When to use: Geographic routing, maps, spatial graphs where coordinates are available.

Python:

```
# With geographic coordinates
result = g.astar("city_a", "city_b", lat_prop="latitude", lon_prop="longitude")
# {"found": True, "distance": 412.5, "path": [...], "nodes_explored": 18}

print(f"Explored {result['nodes_explored']} nodes (vs full BFS)")

# Without coordinates (falls back to uniform heuristic)
result = g.astar("alice", "frank")
```

Rust:

```
let result = g.astar("city_a", "city_b", Some("latitude"), Some("longitude"))?;
println!("Explored {} nodes", result.nodes_explored);
```

SQL:

```
SELECT cypher('RETURN astar(''city_a'', ''city_b'', ''latitude'',
''longitude'')');
```

All-Pairs Shortest Path

Computes shortest distances between every pair of nodes using Floyd-Warshall.

When to use: Building distance matrices, computing graph diameter, small dense graphs.

Python:

```
results = g.all_pairs_shortest_path()
# [{"source": "alice", "target": "carol", "distance": 1.0}, ...]

# Build a distance matrix
import numpy as np
nodes = list({r["source"] for r in results})
n = len(nodes)
idx = {name: i for i, name in enumerate(nodes)}
D = np.full((n, n), float("inf"))
for r in results:
    D[idx[r["source"]], idx[r["target"]]] = r["distance"]

print(f"Graph diameter: {D[D < float('inf')].max()}")
```

Rust:

```
let results = g.apsp()?;
for r in &results {
    println!("{} -> {}: {}", r.source, r.target, r.distance);
}
```

SQL:

```
SELECT
    json_extract(value, '$.source')   AS src,
    json_extract(value, '$.target')   AS tgt,
    json_extract(value, '$.distance') AS dist
FROM json_each(cypher('RETURN apsp()'))
ORDER BY dist;
```

All-pairs shortest path is O(n²) in space and time. Avoid on graphs with more than a few thousand nodes.

Connected Components

Weakly Connected Components

Groups nodes that are reachable from each other when edge direction is ignored.

When to use: Finding isolated subgraphs, checking overall graph connectivity, data quality checks.

Python:

```
results = g.weakly_connected_components()
# [{"node_id": "alice", "component": 0}, ...]

# Count components
components = {r["component"] for r in results}
print(f"Graph has {len(components)} weakly connected component(s)")

# Find isolated nodes (singletons)
from collections import Counter
counts = Counter(r["component"] for r in results)
singletons = [cid for cid, cnt in counts.items() if cnt == 1]
print(f"{len(singletons)} isolated node(s)")
```

Rust:

```
let results = g.wcc()?;
```

SQL:

```
SELECT
    json_extract(value, '$.component') AS component,
    COUNT(*) AS size
FROM json_each(cypher('RETURN wcc()'))
GROUP BY component
ORDER BY size DESC;
```

Strongly Connected Components

Groups nodes where every node can reach every other node following edge direction.

When to use: Detecting cycles, finding strongly coupled subsystems, directed network analysis.

Python:

```
results = g.strongly_connected_components()
# [{"node_id": "alice", "component": 0}, ...]

from collections import Counter
counts = Counter(r["component"] for r in results)
largest = max(counts, key=counts.get)
print(f"Largest SCC has {counts[largest]} nodes")
```

Rust:

```
let results = g.scc()?;
```

SQL:

```
SELECT
    json_extract(value, '$.component') AS component,
    COUNT(*) AS size
FROM json_each(cypher('RETURN scc()'))
GROUP BY component
ORDER BY size DESC;
```

Traversal

Breadth-First Search (BFS)

Explores nodes level by level outward from a starting node.

When to use: Finding all nodes within N hops, shortest-hop paths, social network analysis.

Python:

```
results = g.bfs("alice", max_depth=2)
# [{"user_id": "alice", "depth": 0, "order": 0},
#  {"user_id": "bob",   "depth": 1, "order": 1},
#  {"user_id": "carol", "depth": 1, "order": 2}, ...]

# Print reachable nodes within 2 hops
for r in results:
    print(f"  {'  ' * r['depth']}{r['user_id']} (depth {r['depth']})")
```

Rust:

```
let results = g.bfs("alice", Some(2))?;
for r in &results {
    println!("depth {}: {}", r.depth, r.user_id);
}
```

SQL:

```
SELECT
    json_extract(value, '$.user_id') AS node,
    json_extract(value, '$.depth')   AS depth
FROM json_each(cypher('RETURN bfs(''alice'', 2)'))
ORDER BY depth, json_extract(value, '$.order');
```

Depth-First Search (DFS)

Explores as deep as possible along each branch before backtracking.

When to use: Cycle detection, topological ordering exploration, tree-like structures.

Python:

```
results = g.dfs("alice", max_depth=3)
# [{"user_id": "alice", "depth": 0, "order": 0}, ...]

for r in sorted(results, key=lambda r: r["order"]):
    indent = "  " * r["depth"]
    print(f"{indent}{r['user_id']}")
```

Rust:

```
let results = g.dfs("alice", None)?;  // None = unlimited depth
for r in &results {
    println!("order {}: {} (depth {})", r.order, r.user_id, r.depth);
}
```

SQL:

```
SELECT
    json_extract(value, '$.user_id') AS node,
    json_extract(value, '$.depth')   AS depth,
    json_extract(value, '$.order')   AS visit_order
FROM json_each(cypher('RETURN dfs(''alice'', 5)'))
ORDER BY visit_order;
```

Similarity

Node Similarity (Jaccard)

Computes Jaccard similarity between the neighborhoods of nodes: `|intersection| / |union|`.

When to use: Collaborative filtering, finding structurally similar entities, de-duplication.

Python:

```
# All pairs above a threshold
results = g.node_similarity(threshold=0.3)
# [{"node1": "alice", "node2": "carol", "similarity": 0.5}, ...]

# Between two specific nodes
result = g.node_similarity(node1_id="alice", node2_id="bob")

# Top-10 most similar pairs
results = g.node_similarity(top_k=10)

for r in sorted(results, key=lambda r: r["similarity"], reverse=True):
    print(f"{r['node1']} <-> {r['node2']}: {r['similarity']:.3f}")
```

Rust:

```
// threshold=0.3, top_k=0 (all pairs)
let results = g.node_similarity(None, None, 0.3, 0)?;
for r in &results {
    println!("{} <-> {}: {:.3}", r.node1, r.node2, r.similarity);
}
```

SQL:

```
SELECT
    json_extract(value, '$.node1')      AS n1,
    json_extract(value, '$.node2')      AS n2,
    json_extract(value, '$.similarity') AS sim
FROM json_each(cypher('RETURN nodeSimilarity()'))
WHERE json_extract(value, '$.similarity') > 0.3
ORDER BY sim DESC;
```

K-Nearest Neighbors (KNN)

Finds the k most similar nodes to a given node based on Jaccard neighborhood similarity.

When to use: Recommendation systems ("users like you also connected to..."), suggestion features.

Python:

```
results = g.knn("alice", k=3)
# [{"neighbor": "carol", "similarity": 0.5, "rank": 1},
#  {"neighbor": "dave",  "similarity": 0.33, "rank": 2}, ...]

print("Alice's most similar neighbors:")
for r in results:
    print(f"  #{r['rank']}: {r['neighbor']} (similarity
{r['similarity']:.3f})")
```

Rust:

```
let results = g.knn("alice", 3)?;
for r in &results {
    println!("#{}: {} ({:.3})", r.rank, r.neighbor, r.similarity);
}
```

SQL:

```
SELECT
    json_extract(value, '$.neighbor')   AS neighbor,
    json_extract(value, '$.similarity') AS similarity,
    json_extract(value, '$.rank')       AS rank
FROM json_each(cypher('RETURN knn(''alice'', 5)'))
ORDER BY rank;
```

Triangle Count

Counts the number of triangles each node participates in and computes the local clustering coefficient.

When to use: Measuring graph density and cliquishness; social network cohesion analysis.

Python:

```
results = g.triangle_count()
# [{"node_id": "alice", "triangles": 2, "clustering_coefficient": 0.67}, ...]

# Nodes with high clustering (tight local clusters)
high_cluster = [r for r in results if r["clustering_coefficient"] > 0.5]
total_triangles = sum(r["triangles"] for r in results) // 3  # each triangle
counted 3 times
print(f"Total triangles in graph: {total_triangles}")
```

Rust:

```
let results = g.triangle_count()?;
for r in &results {
    println!("{}: {} triangles, clustering={:.3}",
        r.node_id, r.triangles, r.clustering_coefficient);
}
```

SQL:

```
SELECT
    json_extract(value, '$.node_id')               AS node,
    json_extract(value, '$.triangles')             AS triangles,
    json_extract(value, '$.clustering_coefficient') AS clustering
FROM json_each(cypher('RETURN triangleCount()'))
ORDER BY triangles DESC;
```

Algorithm Selection Guide

Goal	Recommended Algorithm
Rank nodes by influence	PageRank
Find hubs and leaf nodes	Degree Centrality
Find brokers / bridges	Betweenness Centrality
Find information spreaders	Closeness Centrality
Fast community discovery	Label Propagation
High-quality communities	Louvain or Leiden
Shortest path (unweighted)	Dijkstra / `shortest_path`
Shortest path (geographic)	A* with lat/lon properties
Full distance matrix	APSP (small graphs only)
Check connectivity	Weakly Connected Components
Find cycles	Strongly Connected Components
Find N-hop neighbors	BFS
Explore deep paths	DFS
Similar-neighbor pairs	Node Similarity
"People you may know"	KNN
Measure clustering	Triangle Count

Performance Tips

1. **Load the cache first.** Call `g.load_graph()` before running multiple algorithms. This populates an in-memory representation that avoids re-reading the database for each algorithm call.

2. **Reload after writes.** After inserting or updating nodes and edges, call `g.reload_graph()` so algorithms see the new data.

3. **Avoid APSP on large graphs.** `all_pairs_shortest_path()` is $O(n^2)$ in both time and memory. It is practical up to roughly 5 000 nodes; beyond that, use `shortest_path()` for specific pairs.

4. **Use `max_depth` for BFS/DFS.** Without a depth limit, traversal may visit the entire graph. Always pass a `max_depth` when you only need local neighborhood information.

5. **Betweenness scales as O(n * m).** On graphs with millions of edges this can take minutes. For approximate betweenness, consider sampling a subset of source nodes.

6. **Leiden requires graspologic.** Install it with `pip install graphqlite[leiden]`. If graspologic is not installed, `leiden_communities()` raises an `ImportError`.

Handling Special Characters

Certain characters in property values or identifiers require special treatment in Cypher queries. This guide covers the three main categories — property values, relationship types, and property names — and the right approach for each.

Property Values

The Problem

When property values are interpolated directly into a Cypher string, control characters and punctuation can break parsing or produce silently wrong results:

```
# This will cause a syntax error or corrupt the query
g.query("CREATE (n:Note {text: 'It's a lovely day'})")  # SyntaxError:
unmatched quote

# This may parse but produce no results
g.query("CREATE (n:Note {text: 'Line1\nLine2'})")      # newline inside string
literal
```

Characters that need special handling:

Character	Risk
' single quote	Terminates the string literal early
\ backslash	Starts an escape sequence
\n newline	Splits the literal across lines; breaks parsing
\r carriage return	Same as newline
\t tab	Less common; can cause issues in some parsers
" double quote	Less common in Cypher but still problematic

Solution 1: Parameterized Queries (Recommended)

Parameters bypass the Cypher string parser entirely. The value is passed as JSON outside the query text, so no escaping is required:

```
from graphqlite import connect

conn = connect(":memory:")

# Single quotes, newlines, backslashes — all handled automatically
conn.cypher(
    "CREATE (n:Note {title: $title, content: $content})",
    {
        "title": "Alice's Report",
        "content": "Line 1\nLine 2\nBackslash: \\ done.",
    }
)

# Retrieve and verify
results = conn.cypher(
    "MATCH (n:Note {title: $title}) RETURN n.content",
    {"title": "Alice's Report"}
)
print(results[0]["n.content"])
# Line 1
# Line 2
# Backslash: \ done.
```

See Parameterized Queries for the full guide.

Solution 2: The Graph API

The high-level `Graph` API handles escaping internally for `upsert_node` and `upsert_edge`. Pass raw Python strings; no escaping is needed:

```
from graphqlite import Graph

g = Graph(":memory:")

g.upsert_node("note1", {
    "title": "Alice's Report",
    "content": "Line 1\nLine 2\nBackslash: \\",
    "author": 'Bob said "hello"',
}, label="Note")

node = g.get_node("note1")
print(node["properties"]["content"])  # Line 1\nLine 2\nBackslash: \
```

Solution 3: escape_string() for Manual Queries

If you must build a Cypher string by hand, use the `escape_string()` utility from the graphqlite package:

```
from graphqlite import Graph, escape_string

g = Graph(":memory:")

raw_text = "Alice's note:\nLine 1\nLine 2"
safe_text = escape_string(raw_text)  # Escapes quotes, newlines, backslashes

g.query(f"CREATE (n:Note {{content: '{safe_text}'}})")
```

`escape_string()` applies these transformations in order:

1. `\` → `\\` (backslashes first, so they are not double-escaped)
2. `'` → `\'` (single quotes)
3. `\n` → (newlines replaced with a space)
4. `\r` → (carriage returns replaced with a space)
5. `\t` → (tabs replaced with a space)

If preserving newlines in stored values is important, use parameterized queries instead — `escape_string()` converts them to spaces.

Relationship Types

Relationship type names must be valid identifiers. The `sanitize_rel_type()` utility converts arbitrary strings into safe type names:

```
from graphqlite import sanitize_rel_type

print(sanitize_rel_type("has-friend"))   # HAS_FRIEND
print(sanitize_rel_type("works with"))  # WORKS_WITH
print(sanitize_rel_type("type/1"))      # TYPE_1
```

The function:

- Converts to uppercase
- Replaces hyphens, spaces, and slashes with underscores
- Strips other non-alphanumeric characters

Use it whenever relationship types come from user input or external data:

```
from graphqlite import Graph, sanitize_rel_type

g = Graph(":memory:")

user_provided_type = "works-with"
safe_type = sanitize_rel_type(user_provided_type)  # WORKS_WITH

g.upsert_edge("alice", "bob", {"project": "Apollo"}, rel_type=safe_type)
```

Property Names and Identifiers

Backtick Quoting

Property names that conflict with Cypher keywords or contain special characters can be quoted with backticks:

```
# "type", "end", "order" are reserved Cypher keywords
g.connection.cypher("MATCH (n) WHERE n.`type` = 'A' RETURN n.`order`")

# Property names with spaces or hyphens
g.connection.cypher("CREATE (n:Item {`item-code`: 'XYZ-001', `display name`:
'Widget'})")
```

Using CYPHER_RESERVED

The `CYPHER_RESERVED` set contains all reserved Cypher keywords. Check before using a string as a label or property name:

```
from graphqlite import CYPHER_RESERVED

def safe_label(name: str) -> str:
    if name.upper() in CYPHER_RESERVED:
        return f"`{name}`"
    return name

label = safe_label("order")  # "`order`"
label = safe_label("Person") # "Person"

g.connection.cypher(f"CREATE (n:{label} {{id: 'o1'}})")
```

Common Pitfalls

Symptom: MATCH returns nothing after CREATE

Cause: Newlines or carriage returns in property values broke the inline Cypher string during creation. The node was stored but its properties are corrupt or missing.

Fix: Use parameterized queries for any value that may contain whitespace control characters.

```
# Broken
name_with_newline = "Alice\nMitchell"
g.query(f"CREATE (n:Person {{name: '{name_with_newline}'}})")

# Fixed
g.connection.cypher("CREATE (n:Person {name: $name})", {"name":
name_with_newline})
```

Symptom: SyntaxError on CREATE

Cause: Unescaped single quotes in the value.

```
# Broken
g.query("CREATE (n:Quote {text: 'It's a test'})")   # SyntaxError

# Fixed: use parameters
g.connection.cypher("CREATE (n:Quote {text: $text})", {"text": "It's a test"})

# Or: escape manually (less preferred)
g.query("CREATE (n:Quote {text: 'It\\'s a test'})")
```

Symptom: Relationship type with hyphens not found

Cause: Hyphens are not valid in unquoted Cypher identifiers. `CREATE (a)-[:has-friend]->(b)` is parsed as `has` minus `friend`.

Fix: Use underscores or sanitize the type name:

```
from graphqlite import sanitize_rel_type

# Broken
g.query("CREATE (a:Person {name: 'A'})-[:has-friend]->(b:Person {name: 'B'})")

# Fixed: use sanitized type
rel_type = sanitize_rel_type("has-friend")  # HAS_FRIEND
g.upsert_edge("alice", "bob", {}, rel_type=rel_type)
```

Symptom: Property access on reserved keyword property name

Cause: `n.type` is parsed as `n` followed by the keyword `type`, not as property access.

Fix: Quote with backticks.

```
# Broken
results = g.query("MATCH (n) WHERE n.type = 'Product' RETURN n")

# Fixed
results = g.query("MATCH (n) WHERE n.`type` = 'Product' RETURN n")
```

Best Practices

1. **Always use parameterized queries for user-supplied data.** This is the only safe approach for arbitrary values.
2. **Use the Graph API (`upsert_node` , `upsert_edge`) for CRUD operations.** It handles escaping automatically.
3. **Call `sanitize_rel_type()` for dynamic relationship types.** Any type name derived from external input needs sanitization.
4. **Backtick-quote property names that are reserved words.** Check against `CYPHER_RESERVED` when property names come from a schema or API response.
5. **Validate and strip control characters at ingestion time** if your data comes from sources that may embed nulls or other non-printable characters.

Using GraphQLite with Other SQLite Extensions

GraphQLite is a standard SQLite extension and can share a connection with other SQLite extensions, including `sqlite-vec` (vector search), `sqlite-fts5` (full-text search), and others.

Loading GraphQLite into an Existing Connection

If you already have a `sqlite3.Connection`, use `graphqlite.load()` to add GraphQLite functions to it:

```
import sqlite3
import graphqlite

conn = sqlite3.connect("combined.db")
graphqlite.load(conn)

# Both the graph functions and the raw database are now available on conn
conn.execute("SELECT cypher('CREATE (n:Page {title: \"Home\"})')")
conn.execute("SELECT * FROM nodes")
```

Wrapping an Existing Connection

`graphqlite.wrap()` loads GraphQLite into an existing `sqlite3.Connection` and returns a `Connection` object that exposes the `cypher()` method:

```
import sqlite3
import graphqlite

raw_conn = sqlite3.connect("combined.db")
conn = graphqlite.wrap(raw_conn)

# Use the graphqlite Connection API
conn.cypher("CREATE (n:Page {title: 'Home'})")
results = conn.cypher("MATCH (n:Page) RETURN n.title")

# Access the underlying sqlite3.Connection for raw SQL
raw_conn.execute("SELECT COUNT(*) FROM nodes").fetchone()
```

Loading Multiple Extensions

Load GraphQLite first (it creates the graph schema tables), then load other extensions:

```
import sqlite3
import graphqlite

conn = sqlite3.connect("combined.db")

# 1. Load GraphQLite (creates schema tables)
graphqlite.load(conn)

# 2. Load other extensions
conn.enable_load_extension(True)
conn.load_extension("/path/to/other_extension")
conn.enable_load_extension(False)
```

This order matters: GraphQLite creates `nodes`, `edges`, and related tables on first load. If another extension has conflicting table names, loading GraphQLite first lets you detect the conflict early.

Example: GraphQLite + sqlite-vec

Combine graph traversal with vector similarity search. This pattern is the foundation of GraphRAG systems: find semantically similar documents with vectors, then expand to related content via graph edges.

```
import sqlite3
import graphqlite
import sqlite_vec
import json

# Create and configure the connection
conn = sqlite3.connect("knowledge.db")
graphqlite.load(conn)
sqlite_vec.load(conn)

# Create graph nodes (documents)
conn.execute("SELECT cypher('CREATE (n:Document {doc_id: \"doc1\", title:
\"Introduction to Graphs\"})')")
conn.execute("SELECT cypher('CREATE (n:Document {doc_id: \"doc2\", title:
\"Graph Algorithms\"})')")
conn.execute("SELECT cypher('CREATE (n:Document {doc_id: \"doc3\", title:
\"Vector Search\"})')")

# Link related documents in the graph
conn.execute("""
    SELECT cypher('
        MATCH (a:Document {doc_id: "doc1"}), (b:Document {doc_id: "doc2"})
        CREATE (a)-[:RELATED_TO {strength: 0.9}]->(b)
    ')
""")
conn.execute("""
    SELECT cypher('
        MATCH (b:Document {doc_id: "doc2"}), (c:Document {doc_id: "doc3"})
        CREATE (b)-[:RELATED_TO {strength: 0.7}]->(c)
    ')
""")

# Create a vector table for document embeddings
conn.execute("""
    CREATE VIRTUAL TABLE IF NOT EXISTS doc_embeddings
    USING vec0(
        doc_id TEXT PRIMARY KEY,
        embedding FLOAT[4]
    )
""")

# Insert mock embeddings (replace with real model output)
embeddings = {
    "doc1": [0.1, 0.2, 0.9, 0.3],
    "doc2": [0.1, 0.3, 0.8, 0.4],
    "doc3": [0.9, 0.1, 0.1, 0.8],
}
for doc_id, emb in embeddings.items():
    conn.execute(
        "INSERT INTO doc_embeddings(doc_id, embedding) VALUES (?, ?)",
        [doc_id, json.dumps(emb)]
    )
conn.commit()

# --- Query: vector search + graph expansion ---

# Step 1: Find the most similar document to a query vector
```

```
query_embedding = json.dumps([0.1, 0.25, 0.85, 0.35])
similar = conn.execute("""
    SELECT doc_id
    FROM doc_embeddings
    WHERE embedding MATCH ?
      AND k = 2
    ORDER BY distance
""", [query_embedding]).fetchall()

print("Similar documents:", [row[0] for row in similar])

# Step 2: Expand to graph neighbors for each similar document
for (doc_id,) in similar:
    related = conn.execute(f"""
        SELECT cypher('
            MATCH (d:Document {{doc_id: "{doc_id}"}})-[:RELATED_TO]->
(other:Document)
            RETURN other.doc_id AS id, other.title AS title
        ')
    """).fetchall()
    if related:
        print(f"  {doc_id} is related to:")
        for (row_json,) in related:
            row = json.loads(row_json)
            print(f"    - {row['id']}: {row['title']}")
```

Sharing Connections: In-Memory Databases

In-memory SQLite databases are private to a single connection. All extensions that need to share data must use the same `conn` object:

```
# Correct: one connection, multiple extensions
conn = sqlite3.connect(":memory:")
graphqlite.load(conn)
sqlite_vec.load(conn)
# Both extensions operate on the same in-memory database

# Wrong: two separate connections, two separate databases
conn1 = sqlite3.connect(":memory:")
conn2 = sqlite3.connect(":memory:")
# conn1 and conn2 cannot see each other's data
```

For file-based databases, this restriction does not apply — separate connections can open the same file, subject to SQLite locking rules.

Extension Loading Order

In general:

1. **GraphQLite first.** It creates the graph schema (`nodes` , `edges` , property tables) on first load. Other extensions that reference these tables will find them already present.
2. **Other extensions next.** Load them with `enable_load_extension(True)` / `load_extension()` / `enable_load_extension(False)` .
3. **Commit between loads** if any extension creates tables, to ensure schema visibility:

```
graphqlite.load(conn)
conn.commit()

conn.enable_load_extension(True)
conn.load_extension("my_extension")
conn.enable_load_extension(False)
conn.commit()
```

Using GraphQLite with the Python Graph API Alongside Other Extensions

When you use `graphqlite.Graph` or `graphqlite.connect()` , access the underlying `sqlite3.Connection` to load additional extensions:

```
import graphqlite
import sqlite_vec

g = graphqlite.Graph("knowledge.db")

# Access the raw sqlite3.Connection
raw_conn = g.connection.sqlite_connection
sqlite_vec.load(raw_conn)

# Now use both APIs on the same connection
g.upsert_node("doc1", {"title": "Introduction"}, "Document")
raw_conn.execute("CREATE VIRTUAL TABLE IF NOT EXISTS vecs USING vec0(embedding
FLOAT[4])")
```

Troubleshooting

Extension conflicts

If two extensions register functions with the same name, the later-loaded extension wins. Check for collisions between GraphQLite's functions (`cypher` , `regexp` , `gql_load_graph` , etc.) and the functions registered by your other extension.

Missing tables after loading

Ensure GraphQLite was loaded before any query that references graph tables. The schema is created lazily on first load; if the connection is closed and reopened, `graphqlite.load()` must be called again.

Transaction isolation

Some extensions use their own transaction management. If you encounter "table is locked" errors, commit between extension operations:

```
graphqlite.load(conn)
conn.execute("SELECT cypher('CREATE (n:Test {v: 1})')")
conn.commit()   # Flush GraphQLite writes

# Now safe to use the other extension
conn.execute("INSERT INTO vec_table VALUES (?)", ["..."])
conn.commit()
```

Parameterized Queries

Parameterized queries pass values separately from the Cypher query text. They are the recommended approach for any query that incorporates user input, external data, or strings that may contain special characters.

Why Use Parameters

- **Security.** Parameters prevent Cypher injection — the same class of attack as SQL injection. A value like `' OR 1=1 --` in a parameter is treated as a literal string, not query syntax.
- **Correctness.** Values with single quotes, backslashes, newlines, or Unicode characters work without manual escaping.
- **Clarity.** Query logic and data stay separate, making queries easier to read and reuse.

Named Parameters with $

GraphQLite uses `$name` syntax for named parameters. Parameter names map to keys in the dictionary (Python) or JSON object (SQL) you pass alongside the query:

```
MATCH (n:Person {name: $name}) WHERE n.age > $min_age RETURN n
```

Parameters can appear anywhere a literal value is valid: in property predicates, `WHERE` clauses, `SET` assignments, and `CREATE` property maps.

Python: Connection.cypher()

Pass a dictionary as the second argument to `Connection.cypher()`:

```
from graphqlite import connect

conn = connect(":memory:")

# CREATE with parameters
conn.cypher(
    "CREATE (n:Person {name: $name, age: $age, city: $city})",
    {"name": "Alice", "age": 30, "city": "London"}
)

# MATCH with parameters
results = conn.cypher(
    "MATCH (n:Person) WHERE n.age >= $min_age AND n.city = $city RETURN n.name,
n.age",
    {"min_age": 25, "city": "London"}
)
for row in results:
    print(f"{row['n.name']}, age {row['n.age']}")

# SET with parameters
conn.cypher(
    "MATCH (n:Person {name: $name}) SET n.status = $status",
    {"name": "Alice", "status": "active"}
)
```

Python: Graph.query() with Parameters

`Graph.query()` accepts an optional `params` argument:

```
from graphqlite import Graph

g = Graph(":memory:")
g.upsert_node("alice", {"name": "Alice", "age": 30}, "Person")
g.upsert_node("bob",   {"name": "Bob",   "age": 25}, "Person")

results = g.query(
    "MATCH (n:Person) WHERE n.age >= $min_age RETURN n.name ORDER BY n.name",
    params={"min_age": 26}
)
for row in results:
    print(row["n.name"])  # Alice
```

SQL Interface

Pass the parameters as a JSON string in the second argument to `cypher()`:

```
-- Single parameter
SELECT cypher(
    'MATCH (n:Person {name: $name}) RETURN n.age',
    '{"name": "Alice"}'
);

-- Multiple parameters
SELECT cypher(
    'MATCH (n:Person) WHERE n.age >= $min AND n.age <= $max RETURN n.name',
    '{"min": 25, "max": 35}'
);

-- CREATE with parameters
SELECT cypher(
    'CREATE (n:Event {title: $title, year: $year})',
    '{"title": "Graph Summit", "year": 2025}'
);
```

In Python with a raw `sqlite3` connection:

```
import sqlite3, json, graphqlite

conn = sqlite3.connect(":memory:")
graphqlite.load(conn)

params = json.dumps({"name": "Alice", "age": 30})
conn.execute("SELECT cypher('CREATE (n:Person {name: $name, age: $age})', ?)",
[params])
conn.commit()

params = json.dumps({"min_age": 25})
rows = conn.execute(
    "SELECT cypher('MATCH (n:Person) WHERE n.age >= $min_age RETURN n.name',
?)",
    [params]
).fetchall()
```

Rust

In Rust, embed parameter values directly into the query string using `format!`. Full parameterized binding is planned for a future release.

```rust
use graphqlite::Connection;

fn main() -> graphqlite::Result<()> {
    let conn = Connection::open_in_memory()?;

    // Safe integer embedding
    let min_age: i32 = 25;
    let results = conn.cypher(&format!(
        "MATCH (n:Person) WHERE n.age >= {} RETURN n.name AS name",
        min_age
    ))?;

    for row in &results {
        println!("{}", row.get::<String>("name")?);
    }

    // For strings, pass via JSON through the SQL cypher() function
    let name = "Alice";
    let params = serde_json::json!({"name": name, "age": 30});
    conn.execute_sql(
        "SELECT cypher('CREATE (n:Person {name: $name, age: $age})', ?)",
        &[&params.to_string()],
    )?;

    Ok(())
}
```

Supported Parameter Types

Parameters map to JSON types, which GraphQLite converts to Cypher-compatible values:

JSON Type	Cypher Type	Python Example	Rust Type
String	String	`"hello"`	`String`, `&str`
Integer	Integer	`42`	`i32`, `i64`
Float	Float	`3.14`	`f64`
Boolean	Boolean	`True` / `False`	`bool`
Null	Null	`None`	`Option<T>`
Array	List	`[1, 2, 3]`	`Vec<T>`
Object	Map	`{"k": "v"}`	`serde_json::Value`

```
conn.cypher(
    "CREATE (n:Record {label: $label, count: $count, ratio: $ratio, active:
$active, tags: $tags})",
    {
        "label": "alpha",
        "count": 100,
        "ratio": 0.75,
        "active": True,
        "tags": ["graph", "database", "cypher"],
    }
)
```

Common Patterns

User Input Safety

Always parameterize user-provided values:

```
def find_person(user_input: str):
    return conn.cypher(
        "MATCH (n:Person {name: $name}) RETURN n",
        {"name": user_input}   # Safe regardless of what user_input contains
    )

# These all work correctly and safely:
find_person("Alice")
find_person("O'Brien")
find_person("Robert'); DROP TABLE nodes;--")
```

Dynamic Filtering

Build the parameter dictionary dynamically; keep the query shape stable:

```
def search_people(name=None, min_age=None, city=None):
    conditions = []
    params = {}

    if name is not None:
        conditions.append("n.name = $name")
        params["name"] = name
    if min_age is not None:
        conditions.append("n.age >= $min_age")
        params["min_age"] = min_age
    if city is not None:
        conditions.append("n.city = $city")
        params["city"] = city

    where = f"WHERE {' AND '.join(conditions)}" if conditions else ""
    query = f"MATCH (n:Person) {where} RETURN n.name, n.age, n.city ORDER BY
n.name"

    return conn.cypher(query, params if params else None)
```

IN Clause with Lists

Pass a list parameter and use `IN`:

```
names = ["Alice", "Bob", "Carol"]
results = conn.cypher(
    "MATCH (n:Person) WHERE n.name IN $names RETURN n.name, n.age",
    {"names": names}
)
```

Batch Inserts

Loop over a dataset and reuse the same parameterized query:

```
people = [
    {"name": "Alice",  "age": 30, "city": "London"},
    {"name": "Bob",    "age": 25, "city": "Paris"},
    {"name": "Carol",  "age": 35, "city": "Berlin"},
    {"name": "Dave",   "age": 28, "city": "London"},
]

for person in people:
    conn.cypher(
        "CREATE (n:Person {name: $name, age: $age, city: $city})",
        person
    )
```

For very large datasets (thousands of nodes), the Bulk Import API is significantly faster.

Values with Special Characters

Parameters handle all special characters automatically — no need for `escape_string()`:

```
documents = [
    {"id": "d1", "text": "It's a lovely day.\nThe sun is shining."},
    {"id": "d2", "text": 'He said "hello" and left.'},
    {"id": "d3", "text": "Path: C:\\Users\\alice\\documents"},
]

for doc in documents:
    conn.cypher(
        "CREATE (n:Document {doc_id: $id, content: $text})",
        doc
    )
```

Optional / Nullable Values

Pass `None` for parameters that may be absent:

```
conn.cypher(
    "CREATE (n:Person {name: $name, nickname: $nickname})",
    {"name": "Alice", "nickname": None}  # nickname will be stored as null
)
```

Parameters vs. String Interpolation

Avoid building queries by string formatting or concatenation:

```
# Dangerous — susceptible to injection and escaping bugs
name = user_input
conn.cypher(f"MATCH (n {{name: '{name}'}}) RETURN n")

# Correct
conn.cypher("MATCH (n {name: $name}) RETURN n", {"name": name})
```

The only case where string formatting is appropriate is for **structural** parts of a query that cannot be parameterized, such as label names or property names. Even then, validate the value against an allowlist before interpolating it.

Bulk Importing Data

For loading large datasets into a GraphQLite graph, the bulk import API is 100–500x faster than issuing individual Cypher `CREATE` statements. This guide covers when to use it, how it works, and a complete worked example.

When to Use Bulk Import

Scenario	Recommended approach
Interactive graph building, <1 000 nodes	`upsert_node` / `upsert_edge` or Cypher `CREATE`
Importing CSV / JSON datasets, >1 000 nodes	`insert_graph_bulk` or `insert_nodes_bulk` + `insert_edges_bulk`
Incremental updates to an existing graph	`upsert_nodes_batch` / `upsert_edges_batch`
Connecting new edges to existing nodes	`insert_edges_bulk` with `resolve_node_ids`

The bulk API bypasses the Cypher parser and writes directly to the graph tables in a single transaction, eliminating per-query overhead.

Python Bulk API

insert_nodes_bulk

Insert a list of nodes in a single transaction. Returns an `id_map` dictionary mapping each external node ID to its internal SQLite rowid.

```
from graphqlite import Graph

g = Graph("company.db")

nodes = [
    ("emp_1",  {"name": "Alice", "role": "engineer", "age": 30}, "Employee"),
    ("emp_2",  {"name": "Bob",   "role": "manager",  "age": 40}, "Employee"),
    ("emp_3",  {"name": "Carol", "role": "engineer", "age": 28}, "Employee"),
    ("dept_1", {"name": "Engineering", "budget": 500000},
"Department"),
    ("dept_2", {"name": "Product",     "budget": 300000},
"Department"),
]

id_map = g.insert_nodes_bulk(nodes)
# id_map = {"emp_1": 1, "emp_2": 2, "emp_3": 3, "dept_1": 4, "dept_2": 5}
print(f"Inserted {len(id_map)} nodes")
```

Each entry in the `nodes` list is a tuple of `(external_id, properties, label)`:

Position	Type	Description
0	`str`	External identifier (any string)
1	`dict`	Dictionary of property key-value pairs
2	`str`	Node label

insert_edges_bulk

Insert edges after nodes have been loaded. Uses the `id_map` returned by `insert_nodes_bulk` to resolve external IDs to internal rowids:

```
edges = [
    ("emp_1", "dept_1", {"since": 2020}, "WORKS_IN"),
    ("emp_2", "dept_1", {"since": 2018}, "WORKS_IN"),
    ("emp_3", "dept_2", {"since": 2022}, "WORKS_IN"),
    ("emp_2", "emp_1",  {},              "MANAGES"),
    ("emp_2", "emp_3",  {},              "MANAGES"),
]

g.insert_edges_bulk(edges, id_map)
print(f"Inserted {len(edges)} edges")
```

Each entry in the `edges` list is a tuple of `(source_id, target_id, properties, rel_type)`:

Position	Type	Description
0	`str`	External ID of the source node
1	`str`	External ID of the target node
2	`dict`	Dictionary of property key-value pairs (may be empty `{}`)

Position	Type	Description
3	`str`	Relationship type

insert_graph_bulk

Insert nodes and edges together in a single call. Internally calls `insert_nodes_bulk` then `insert_edges_bulk`:

```
nodes = [
    ("a", {"name": "Alice", "age": 30}, "Person"),
    ("b", {"name": "Bob",   "age": 25}, "Person"),
    ("c", {"name": "Carol", "age": 35}, "Person"),
]

edges = [
    ("a", "b", {"since": 2019}, "KNOWS"),
    ("b", "c", {"since": 2021}, "KNOWS"),
]

result = g.insert_graph_bulk(nodes, edges)
print(f"Inserted {result.nodes_inserted} nodes, {result.edges_inserted} edges")
print(result.id_map)  # {"a": 1, "b": 2, "c": 3}
```

The id_map Pattern

The `id_map` bridges the gap between your external IDs (strings like `"emp_1"`) and the internal SQLite rowids that GraphQLite uses for edge resolution.

- `insert_nodes_bulk` returns `{"emp_1": 1, "emp_2": 2, ...}`.
- `insert_edges_bulk` uses this map to look up source and target rowids before writing.
- The external IDs are also stored as the `user_id` column on each node, so Cypher queries can still reference them by string: `MATCH (n {id: 'emp_1'})`.

If an external ID in an edge list is not present in `id_map`, `insert_edges_bulk` raises a `KeyError`. Validate your edge list against your node list before calling bulk insert.

Connecting New Edges to Existing Nodes

When importing edges that reference nodes already in the database (from a previous import), use `resolve_node_ids` to build the id_map without re-inserting the nodes:

```
existing_ids = ["emp_1", "emp_2", "emp_3"]
id_map = g.resolve_node_ids(existing_ids)
# id_map = {"emp_1": 1, "emp_2": 2, "emp_3": 3}

# Now add new edges referencing those existing nodes
new_edges = [
    ("emp_1", "emp_3", {"project": "Phoenix"}, "COLLABORATES"),
]
g.insert_edges_bulk(new_edges, id_map)
```

This is the correct pattern when you import nodes in one batch and edges in a separate batch, or when you are enriching an existing graph with new relationships.

Batch Upsert

For incremental updates — adding or updating nodes and edges that may already exist — use the batch upsert methods. These use Cypher MERGE semantics (update if exists, create if not) and are slower than bulk insert but handle conflicts gracefully.

> **Non-atomicity warning:** Batch upsert methods call `upsert_node` / `upsert_edge` in a loop. If an operation fails partway through, earlier operations will have already completed. For atomic batch inserts, use the bulk insert methods instead, or wrap the call in an explicit transaction via `g.connection.sqlite_connection`.

```
# Upsert multiple nodes
nodes_to_update = [
    ("emp_1", {"name": "Alice", "role": "senior engineer", "age": 31},
"Employee"),
    ("emp_4", {"name": "Dave",  "role": "analyst",         "age": 27},
"Employee"),
]
g.upsert_nodes_batch(nodes_to_update)

# Upsert multiple edges
edges_to_update = [
    ("emp_4", "dept_1", {"since": 2023}, "WORKS_IN"),
]
g.upsert_edges_batch(edges_to_update)
```

Each tuple in `upsert_nodes_batch` is `(node_id, properties, label)`. Each tuple in `upsert_edges_batch` is `(source_id, target_id, properties, rel_type)`.

Performance Comparison

Approximate timings for inserting 100 000 nodes + 200 000 edges on a modern laptop:

Method	Time (approx.)
Cypher `CREATE` (one per statement)	90–180 seconds
`upsert_node` / `upsert_edge` in a loop	30–60 seconds
`upsert_nodes_batch` / `upsert_edges_batch`	10–20 seconds
`insert_nodes_bulk` + `insert_edges_bulk`	0.5–2 seconds

Bulk insert achieves its speed by:

1. Writing all rows in a single SQLite transaction.
2. Bypassing the Cypher parser entirely.
3. Preparing INSERT statements once and reusing them.

Complete Example: Importing a CSV

This example imports a CSV of employees and a CSV of manager relationships:

```
import csv
from graphqlite import Graph

g = Graph("hr.db")

# --- Load employees.csv ---
# Columns: id, name, department, age, salary
nodes = []
dept_set = set()
with open("employees.csv") as f:
    for row in csv.DictReader(f):
        nodes.append((
            row["id"],
            {
                "name":       row["name"],
                "department": row["department"],
                "age":        int(row["age"]),
                "salary":     float(row["salary"]),
            },
            "Employee",
        ))
        dept_set.add(row["department"])

print(f"Loading {len(nodes)} employees ...")
id_map = g.insert_nodes_bulk(nodes)

# --- Load department nodes (deduplicated from employee data) ---
dept_nodes = [
    (f"dept_{d}", {"name": d}, "Department")
    for d in dept_set
]
dept_id_map = g.insert_nodes_bulk(dept_nodes)

# Combine id_maps for edge resolution
full_id_map = {**id_map, **dept_id_map}

# Add WORKS_IN edges from employees to departments
dept_edges = [
    (ext_id, f"dept_{props['department']}", {}, "WORKS_IN")
    for ext_id, props, _label in nodes
]
g.insert_edges_bulk(dept_edges, full_id_map)

# --- Load managers.csv ---
# Columns: employee_id, manager_id
edges = []
with open("managers.csv") as f:
    for row in csv.DictReader(f):
        edges.append((
            row["manager_id"],
            row["employee_id"],
            {},
            "MANAGES",
        ))

print(f"Loading {len(edges)} manager relationships ...")
g.insert_edges_bulk(edges, id_map)
```

```
print(g.stats())

# Query to verify
results = g.query("""
    MATCH (mgr:Employee)-[:MANAGES]->(emp:Employee)
    RETURN mgr.name AS manager, emp.name AS report
    ORDER BY mgr.name, emp.name
    LIMIT 10
""")
for row in results:
    print(f"  {row['manager']} manages {row['report']}")
```

Rust Bulk Import

The Rust `Graph` API exposes equivalent batch methods:

```rust
use graphqlite::Graph;

fn main() -> graphqlite::Result<()> {
    let g = Graph::open("company.db")?;

    // Bulk insert nodes
    let nodes = vec![
        ("emp_1", vec![("name", "Alice"), ("role", "engineer"), ("age", "30")],
"Employee"),
        ("emp_2", vec![("name", "Bob"),   ("role", "manager"),  ("age", "40")],
"Employee"),
        ("emp_3", vec![("name", "Carol"), ("role", "engineer"), ("age", "28")],
"Employee"),
    ];
    g.upsert_nodes_batch(nodes)?;

    // Bulk insert edges
    let edges = vec![
        ("emp_2", "emp_1", vec![("since", "2020")], "MANAGES"),
        ("emp_2", "emp_3", vec![("since", "2021")], "MANAGES"),
    ];
    g.upsert_edges_batch(edges)?;

    let stats = g.stats()?;
    println!("Nodes: {}, Edges: {}", stats.nodes, stats.edges);

    // Verify
    let results = g.query("MATCH (m:Employee)-[:MANAGES]->(e:Employee) RETURN
m.name, e.name")?;
    for row in &results {
        println!(
            "{} manages {}",
            row.get::<String>("m.name")?,
            row.get::<String>("e.name")?
        );
    }

    Ok(())
}
```

> The Rust API currently exposes `upsert_nodes_batch` and `upsert_edges_batch` (which use INSERT OR REPLACE). For maximum throughput on very large imports, call the Python bulk API via the Python bindings, or build the graph in Python and use it from Rust.

Tips

- **Wrap bulk inserts in a transaction** if you call `insert_nodes_bulk` and `insert_edges_bulk` separately, to ensure atomicity:

```
with g.connection.sqlite_connection:
    id_map = g.insert_nodes_bulk(nodes)
    g.insert_edges_bulk(edges, id_map)
```

- **Validate before inserting.** Check that all edge source/target IDs exist in your node list before calling `insert_edges_bulk`. Missing IDs raise a `KeyError` mid-insert, which can leave the database in a partial state.
- **Reload the graph cache** after a bulk import if you plan to run algorithms immediately:

```
g.insert_graph_bulk(nodes, edges)
g.reload_graph()
results = g.pagerank()
```

Building from Source

This guide covers building the GraphQLite SQLite extension and CLI from source, running tests, checking code quality, and installing the result for use with the Rust crate.

Prerequisites

Tool	Minimum Version	macOS	Linux (Debian/Ubuntu)	Windows (MSYS2)
GCC or Clang	9 / 11	Xcode CLI tools	`build-essential`	`mingw-w64-x86_64-gcc`
Bison	3.0+	`brew install bison`	`bison`	`bison`
Flex	2.6+	`brew install flex`	`flex`	`flex`
SQLite dev headers	3.35+	`brew install sqlite`	`libsqlite3-dev`	`mingw-w64-x86_64-sqlite3`
CUnit	2.1+	`brew install cunit`	`libcunit1-dev`	(optional)
make	4.0+	Xcode CLI tools	`make`	`make`

macOS

```
xcode-select --install
brew install bison flex sqlite cunit

# Homebrew Bison must precede the system Bison on PATH
export PATH="$(brew --prefix bison)/bin:$PATH"
```

Add the `PATH` export to your shell profile (`~/.zshrc` or `~/.bash_profile`) to make it persistent.

Linux (Debian/Ubuntu)

```
sudo apt-get update
sudo apt-get install build-essential bison flex libsqlite3-dev libcunit1-dev
```

Windows (MSYS2)

```
pacman -S mingw-w64-x86_64-gcc mingw-w64-x86_64-sqlite3 bison flex make
```

Run all commands from the MSYS2 MinGW 64-bit shell.

Clone and Build

```
git clone https://github.com/your-org/graphqlite
cd graphqlite
make extension
```

This produces:

Platform	Output file
macOS	`build/graphqlite.dylib`
Linux	`build/graphqlite.so`
Windows	`build/graphqlite.dll`

Debug vs. Release

The default build includes debug symbols and C assertions. For a production build:

```
make extension RELEASE=1
```

`RELEASE=1` adds `-O2` optimization and strips assertions. Always use release builds for benchmarking.

Build Targets

Run `make help` to see all available targets. The most commonly used ones are:

Core Targets

Target	Description
`make extension`	Build the SQLite extension (`.dylib` / `.so` / `.dll`)
`make extension RELEASE=1`	Build optimized release extension
`make graphqlite`	Build the `gqlite` interactive CLI
`make graphqlite RELEASE=1`	Build optimized release CLI
`make all`	Build everything (extension + CLI)

Test Targets

Target	Description
`make test_unit`	Run CUnit unit tests (770 tests)
`make test_functional`	Run SQL-based functional tests
`make test_python`	Run Python binding tests
`make test_rust`	Build and run Rust binding tests
`make test-all`	Run all test suites

Quality Targets

Target	Description
`make lint`	Strict C11 compliance check
`make coverage`	Build with coverage instrumentation and run unit tests
`make performance`	Run performance benchmarks
`make performance-quick`	Quick performance smoke test

Install Targets

Target	Description
`make install-bundled`	Copy extension into the Rust crate's source tree

Running Tests

Unit Tests

```
make test unit
```

Builds and runs the CUnit test suite. Output shows pass/fail counts per suite. All 770 tests must pass on a clean build.

Functional Tests

```
make test functional
```

Runs SQL script tests against the built extension. Each test file is a `.sql` script in `tests/functional/` that exercises a specific feature:

```
# Run a single functional test manually
sqlite3 :memory: < tests/functional/01_create_match.sql
```

Python Tests

```
make test python
```

Requires Python 3.8+ and `pip install graphqlite[dev]` (or install the dev dependencies from `requirements-dev.txt`).

Rust Tests

```
make test rust
```

This target first calls `make install-bundled` to copy the freshly-built extension into the Rust crate, then runs `cargo test`.

All Tests

```
make test-all
```

Runs unit, functional, CLI, and binding tests in sequence. All suites must pass before a release.

Linting

```
make lint
```

The lint target compiles all C source files with strict C11 flags:

```
-std=c11 -Wall -Wextra -Wpedantic -Werror -Wshadow -Wformat=2
```

Zero warnings are permitted (`-Werror`). Fix all lint warnings before submitting changes. The lint target does not link the extension; it only checks the source.

Coverage

```
make coverage
```

Builds the unit test runner with `--coverage` instrumentation (GCC `gcov` / `lcov`) and runs the tests. Coverage data appears in `build/coverage/` . Open `build/coverage/index.html` in a browser to browse line-by-line coverage.

Requirements: `lcov` must be installed (`brew install lcov` / `sudo apt-get install lcov`).

Performance Benchmarks

```
# Full benchmark suite
make performance

# Quick smoke test (faster, fewer iterations)
make performance-quick

# Extended benchmark with larger graphs
make performance-full
```

Benchmark results are printed to stdout. Run with `RELEASE=1` for meaningful numbers:

```
make performance RELEASE=1
```

Installing for Rust Bundled Builds

The Rust crate defaults to the `bundled-extension` feature, which embeds the extension binary at compile time. After building from source, copy the extension into the Rust crate:

```
make install-bundled
```

This copies `build/graphqlite.{dylib,so,dll}` into the correct location inside `bindings/rust/` so that `cargo build` picks it up. Run this after every source change when developing the Rust crate.

Platform-Specific Notes

macOS

- The system `bison` (in `/usr/bin`) is version 2.x, which is too old. Always install via Homebrew and prepend it to `PATH`.
- If you see `library not loaded: libsqlite3.dylib` when loading the built extension, set `DYLD_LIBRARY_PATH`:

  ```
  export DYLD_LIBRARY_PATH="$(brew --prefix sqlite)/lib:$DYLD_LIBRARY_PATH"
  ```

- On Apple Silicon (M1/M2), the Homebrew prefix is `/opt/homebrew` rather than `/usr/local`.

Linux

- The `libcunit1-dev` package is only needed for unit tests. The extension itself (`make extension`) does not require CUnit.
- On systems without `lcov`, `make coverage` will fail at the report generation step. Install lcov or skip coverage.
- On Alpine Linux (musl libc), replace `apt-get` with `apk add`:

  ```
  apk add build-base bison flex sqlite-dev
  ```

Windows (MSYS2)

- All commands must be run from the **MSYS2 MinGW 64-bit** shell, not the default MSYS shell. The MinGW shell sets the correct compiler paths.
- Python tests require a Windows Python 3.8+ installation accessible from the MSYS2 PATH.
- The extension output is `build/graphqlite.dll`.

Build Directory Layout

After a full build:

```
build/
├── graphqlite.dylib        # SQLite extension (macOS)
├── gqlite                  # Interactive CLI binary
├── test_runner             # CUnit test runner binary
├── parser/                 # Compiled parser objects
├── transform/              # Compiled transform objects
├── executor/               # Compiled executor objects
└── coverage/               # Coverage HTML report (after make coverage)
```

Troubleshooting

`bison: syntax error` during build

The system Bison is too old. Confirm the version:

```
bison --version
```

If it shows `2.x`, install Homebrew Bison (`brew install bison`) and prepend it to `PATH`.

`flex: command not found`

Install flex: `brew install flex` / `sudo apt-get install flex`.

`cannot open shared object file: libcunit.so`

CUnit is not installed or not on `LD_LIBRARY_PATH`. Install it (`libcunit1-dev`) or run `make extension` (which does not need CUnit) instead of `make test unit`.

Python tests fail with `No module named graphqlite`

Install the Python package in development mode:

```
pip install -e bindings/python/
```

or install it from PyPI and use `GRAPHQLITE_EXTENSION_PATH` to point to your locally built extension:

```
export GRAPHQLITE_EXTENSION_PATH=$(pwd)/build/graphqlite.dylib
make test python
```

Cypher Support Reference

GraphQLite implements a substantial subset of openCypher. This page is a quick-reference index; details are in the sub-pages.

Clauses

Clause	Status	Notes
`MATCH`	Supported	Node, relationship, variable-length, named path patterns
`OPTIONAL MATCH`	Supported	Left outer join semantics
`CREATE`	Supported	Nodes and relationships
`MERGE`	Supported	`ON CREATE SET`, `ON MATCH SET`
`SET`	Supported	Property assign, map replace (`=`), map merge (`+=`), label add
`REMOVE`	Supported	Property removal, label removal
`DELETE`	Supported	Nodes and relationships
`DETACH DELETE`	Supported	Cascading edge removal
`RETURN`	Supported	`AS`, `DISTINCT`, `ORDER BY`, `LIMIT`, `SKIP`, `*`
`WITH`	Supported	Aggregation, filtering, projection between clauses
`WHERE`	Supported	All predicates; pattern predicates
`UNWIND`	Supported	List expansion
`FOREACH`	Supported	Mutation inside list iteration
`UNION / UNION ALL`	Supported	
`LOAD CSV WITH HEADERS FROM`	Supported	Local file paths
`FROM`	Supported	Multi-graph queries (GraphQLite extension)
`CALL {}` subqueries	**Not supported**	
`CALL procedure`	**Not supported**	No procedure registry
`CREATE INDEX`	**Not supported**	Schema is managed automatically

Clause	Status	Notes
`CASE` in `SET`	**Not supported**	Use `CASE` in `RETURN` / `WITH` instead
Nested `FOREACH`	**Not supported**	
`EXPLAIN` / `PROFILE`	**Not supported**	

Functions

Category	Functions
String	`toUpper`, `toLower`, `trim`, `ltrim`, `rtrim`, `btrim`, `substring`, `replace`, `reverse`, `left`, `right`, `split`, `toString`, `size`, `isEmpty`, `char_length`, `character_length`
Math	`abs`, `ceil`, `floor`, `round`, `sqrt`, `sign`, `log`, `log10`, `exp`, `e`, `pi`, `rand`, `toInteger`, `toFloat`
Trigonometry	`sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `atan2`, `degrees`, `radians`, `cot`, `haversin`, `sinh`, `cosh`, `tanh`, `coth`, `isNaN`
List	`size`, `head`, `tail`, `last`, `range`, `collect`, `keys`, `reduce`, `[expr FOR x IN list [WHERE cond]]`
Aggregation	`count`, `sum`, `avg`, `min`, `max`, `collect`, `stdev`, `stdevp`
Entity	`id`, `elementId`, `labels`, `type`, `properties`, `startNode`, `endNode`, `nodes`, `relationships`, `length`
Type conversion	`toString`, `toInteger`, `toFloat`, `toBoolean`, `toStringOrNull`, `toIntegerOrNull`, `toFloatOrNull`, `toBooleanOrNull`, `valueType`
Temporal	`date`, `time`, `datetime`, `localdatetime`, `duration`, `datetime.fromepoch`, `datetime.fromepochmillis`, `duration.inDays`, `duration.inSeconds`, `date.truncate`
Spatial	`point`, `distance`, `point.withinBBox`
Predicate	`exists`, `coalesce`, `nullIf`
CASE	`CASE WHEN … THEN … END`, `CASE expr WHEN v THEN … END`
Graph algorithms	`pageRank`, `labelPropagation`, `louvain`, `dijkstra`, `astar`, `degreeCentrality`, `betweennessCentrality`, `closenessCentrality`, `eigenvectorCentrality`, `weaklyConnectedComponents`, `stronglyConnectedComponents`, `bfs`, `dfs`, `nodeSimilarity`, `knn`, `triangleCount`, `apsp`, `shortestPath`

Operators

Category	Operators
Arithmetic	`+`, `-`, `*`, `/`, `%`
Comparison	`=`, `<>`, `<`, `>`, `<=`, `>=`
Boolean	`AND`, `OR`, `NOT`, `XOR`
String	`STARTS WITH`, `ENDS WITH`, `CONTAINS`, `=~` (regex)
List	`IN`, `+` (concat), `[index]`, `[start..end]` (slice)
Null	`IS NULL`, `IS NOT NULL`
Property access	`.` (dot notation), `['key']` (subscript)

Not Supported

- `CALL {}` correlated subqueries
- `CALL procedure(...)` procedure invocations
- `CREATE INDEX ON :Label(prop)`
- `EXPLAIN` / `PROFILE`
- `CASE` expressions on the left-hand side of `SET`
- Nested `FOREACH`

Sub-pages

- Clauses — syntax, description, and examples for every clause
- Functions — signature, return type, and example for every function
- Operators — all operators with precedence table

Cypher Clauses

MATCH

Syntax

```
MATCH pattern [WHERE condition]
```

Reads nodes and relationships matching the given pattern. Binds variables to matched entities. Multiple comma-separated patterns in one `MATCH` form a cross-product constrained by shared variables. A `WHERE` clause immediately following `MATCH` filters before joining subsequent clauses.

Pattern forms

Pattern	Example
Node	`(n:Label)`
Relationship	`(a)-[r:TYPE]->(b)`
Undirected relationship	`(a)-[r:TYPE]-(b)`
Variable-length	`(a)-[*1..3]->(b)`
Named path	`p = (a)-[*]->(b)`
Multiple labels	`(n:Person:Employee)`
Property filter	`(n:Person {name: 'Alice'})`

Examples

```
MATCH (n:Person) RETURN n.name
```

```
MATCH (a:Person)-[:KNOWS]->(b:Person)
WHERE a.name = 'Alice'
RETURN b.name
```

```
MATCH p = (a)-[*1..3]->(b)
RETURN nodes(p)
```

OPTIONAL MATCH

Syntax

```
OPTIONAL MATCH pattern [WHERE condition]
```

Left outer join. Rows from preceding clauses are preserved even when no match exists. Unmatched variables are bound to `null`.

Example

```
MATCH (n:Person)
OPTIONAL MATCH (n)-[:HAS_PET]->(p:Pet)
RETURN n.name, p.name
```

CREATE

Syntax

```
CREATE pattern
```

Creates nodes and/or relationships. Variables introduced in `CREATE` are available in subsequent clauses.

Examples

```
CREATE (n:Person {name: 'Alice', age: 30})
```

```
MATCH (a:Person {name: 'Alice'}), (b:Person {name: 'Bob'})
CREATE (a)-[:KNOWS {since: 2020}]->(b)
```

MERGE

Syntax

```
MERGE pattern
[ON CREATE SET assignment [, assignment ...]]
[ON MATCH SET assignment [, assignment ...]]
```

Matches the pattern or creates it if no match exists. `ON CREATE SET` executes only on creation; `ON MATCH SET` executes only when the pattern already exists. Both subclauses are optional and independent.

Examples

```
MERGE (n:Person {name: 'Alice'})
ON CREATE SET n.created = datetime()
ON MATCH SET n.updated = datetime()
```

```
MERGE (a:Person {name: 'Bob'})-[:KNOWS]->(b:Person {name: 'Carol'})
```

SET

Syntax

```
SET item [, item ...]
```

Assignment forms

Form	Behavior
`n.prop = expr`	Set or overwrite one property
`n = {map}`	Replace all properties with the map; unlisted properties are removed
`n += {map}`	Merge map into existing properties; unlisted properties are kept
`n:Label`	Add a label to a node

Examples

```
MATCH (n:Person {name: 'Alice'}) SET n.age = 31
```

```
MATCH (n:Person {name: 'Alice'}) SET n = {name: 'Alice', age: 31}
```

```
MATCH (n:Person {name: 'Alice'}) SET n += {age: 31, city: 'NYC'}
```

```
MATCH (n:Person {name: 'Alice'}) SET n:Employee
```

REMOVE

Syntax

```
REMOVE item [, item ...]
```

Item forms

Form	Behavior
`n.prop`	Delete a property from a node or relationship
`n:Label`	Remove a label from a node

Examples

```
MATCH (n:Person {name: 'Alice'}) REMOVE n.age
```

```
MATCH (n:Employee) REMOVE n:Employee
```

DELETE

Syntax

```
DELETE expr [, expr ...]
```

Deletes nodes or relationships. Attempting to delete a node that still has relationships raises an error. Use `DETACH DELETE` to cascade.

Examples

```
MATCH (n:Temp) DELETE n
```

```
MATCH (a)-[r:OLD]->(b) DELETE r
```

DETACH DELETE

Syntax

```
DETACH DELETE expr [, expr ...]
```

Deletes a node and all its incident relationships in one operation.

Example

```
MATCH (n:Person {name: 'Alice'}) DETACH DELETE n
```

RETURN

Syntax

```
RETURN [DISTINCT] expr [AS alias] [, ...]
[ORDER BY expr [ASC|DESC] [, ...]]
[SKIP expr]
[LIMIT expr]
```

Projects query results into the result set. `*` expands to all in-scope variables.

Modifiers

Modifier	Description
`DISTINCT`	Remove duplicate rows from output
`AS alias`	Assign a column name
`ORDER BY expr [ASC|DESC]`	Sort; default direction is `ASC`
`SKIP n`	Skip the first `n` rows
`LIMIT n`	Return at most `n` rows
`*`	Return all variables in scope

Examples

```
MATCH (n:Person)
RETURN n.name AS name, n.age
ORDER BY n.age DESC
LIMIT 10
```

```
MATCH (n:Person) RETURN DISTINCT n.city
```

```
MATCH (n) RETURN *
```

WITH

Syntax

```
WITH [DISTINCT] expr [AS alias] [, ...]
[ORDER BY expr [ASC|DESC] [, ...]]
[SKIP expr]
[LIMIT expr]
[WHERE condition]
```

Pipelines intermediate results between query parts. Variables not listed in `WITH` go out of scope. Aggregation in `WITH` collapses rows; `WHERE` after `WITH` filters the aggregated result.

Examples

```
MATCH (n:Person)-[:KNOWS]->(m)
WITH n, count(m) AS friends
WHERE friends > 3
RETURN n.name, friends
```

```
MATCH (n:Person)
WITH n ORDER BY n.age LIMIT 5
MATCH (n)-[:KNOWS]->(m)
RETURN n.name, m.name
```

WHERE

Syntax

```
WHERE condition
```

Filters rows. Appears after `MATCH`, `OPTIONAL MATCH`, or `WITH`. Supports all comparison operators, boolean operators, string predicates, `IS NULL`, `IS NOT NULL`, `IN`, and pattern predicates.

Predicate forms

Predicate	Example
Comparison	`n.age > 25`
String	`n.name STARTS WITH 'Al'`
Regex	`n.name =~ 'Al.*'`
Null check	`n.email IS NOT NULL`
List membership	`n.role IN ['admin', 'mod']`

Predicate	Example
Pattern existence	`(n)-[:KNOWS]->(m)`
Negated pattern	`NOT (n)-[:BLOCKED]->(m)`
exists{}	`exists{(n)-[:KNOWS]->(m)}`

Examples

```
MATCH (n:Person)
WHERE n.age > 25 AND n.city = 'NYC'
RETURN n.name
```

```
MATCH (n:Person)
WHERE (n)-[:KNOWS]->(:Person {name: 'Bob'})
RETURN n.name
```

UNWIND

Syntax

```
UNWIND expr AS variable
```

Expands a list into one row per element. A `null` list produces zero rows. An empty list produces zero rows.

Examples

```
UNWIND [1, 2, 3] AS x RETURN x
```

```
MATCH (n:Person)
UNWIND labels(n) AS lbl
RETURN n.name, lbl
```

FOREACH

Syntax

```
FOREACH (variable IN list | update_clause [update_clause ...])
```

Executes mutation clauses (`CREATE` , `MERGE` , `SET` , `REMOVE` , `DELETE`) for each element of a list. Variables introduced inside `FOREACH` are scoped to the body and not available outside. Nested `FOREACH` is not supported.

Examples

```
FOREACH (name IN ['Alice', 'Bob', 'Carol'] |
  CREATE (:Person {name: name})
)
```

```
MATCH p = (a:Person)-[:KNOWS*]->(b:Person)
FOREACH (n IN nodes(p) | SET n.visited = true)
```

UNION / UNION ALL

Syntax

```
query
UNION [ALL]
query
```

Combines results from two queries. Column names and count must match. `UNION` deduplicates rows; `UNION ALL` preserves all rows including duplicates.

Example

```
MATCH (n:Person) RETURN n.name AS name
UNION
MATCH (n:Company) RETURN n.name AS name
```

LOAD CSV WITH HEADERS FROM

Syntax

```
LOAD CSV WITH HEADERS FROM 'file:///path/to/file.csv' AS row
[FIELDTERMINATOR char]
```

Reads a CSV file and binds each row as a map keyed by header names. All values are strings; use `toInteger()` , `toFloat()` , or `toBoolean()` as needed.

Example

```
LOAD CSV WITH HEADERS FROM 'file:///data/people.csv' AS row
CREATE (:Person {name: row.name, age: toInteger(row.age)})
```

FROM (Multi-graph)

Syntax

```
FROM 'db_path'
MATCH ...
```

GraphQLite extension clause. Queries a different SQLite database file. The target database must have the GraphQLite schema initialized.

Example

```
FROM 'social.db'
MATCH (n:Person)
RETURN n.name
```

Pattern Predicates in WHERE

A pattern used as a boolean expression inside `WHERE` evaluates to `true` if at least one match exists. Supports full pattern syntax including property filters, relationship type filters, and variable-length paths.

```
WHERE (a)-[:KNOWS]->(b)
WHERE NOT (a)-[:BLOCKED]->(b)
WHERE (a)-[:KNOWS*1..3]->(b)
WHERE exists{(a)-[:KNOWS]->(:Person {active: true})}
```

Pattern predicates can reference variables bound in preceding `MATCH` clauses.

Cypher Functions

Every function available in GraphQLite Cypher queries, organized by category.

String Functions

Signature	Returns	Description
`toUpper(s)`	String	Convert to uppercase
`toLower(s)`	String	Convert to lowercase
`trim(s)`	String	Remove leading and trailing whitespace
`ltrim(s)`	String	Remove leading whitespace
`rtrim(s)`	String	Remove trailing whitespace
`btrim(s)`	String	Remove leading and trailing whitespace (alias of `trim`)
`substring(s, start)`	String	Substring from `start` (0-based) to end
`substring(s, start, len)`	String	Substring of length `len` from `start`
`replace(s, search, replacement)`	String	Replace all occurrences of `search` with `replacement`
`reverse(s)`	String	Reverse characters
`left(s, n)`	String	First `n` characters
`right(s, n)`	String	Last `n` characters
`split(s, delimiter)`	List<String>	Split string into list of substrings
`toString(val)`	String	Convert any value to its string representation
`size(s)`	Integer	Number of characters in string
`isEmpty(s)`	Boolean	`true` if string has length zero or is `null`
`char_length(s)`	Integer	Number of characters (alias of `size`)
`character_length(s)`	Integer	Number of characters (alias of `size`)

Examples

```
RETURN toUpper('hello')           -- 'HELLO'
RETURN substring('abcdef', 2, 3)  -- 'cde'
RETURN split('a,b,c', ',')        -- ['a', 'b', 'c']
RETURN left('abcdef', 3)          -- 'abc'
```

Math Functions

Signature	Returns	Description
`abs(n)`	Number	Absolute value
`ceil(n)`	Integer	Ceiling (smallest integer >= n)
`floor(n)`	Integer	Floor (largest integer <= n)
`round(n)`	Integer	Round to nearest integer
`round(n, precision)`	Float	Round to `precision` decimal places
`sqrt(n)`	Float	Square root
`sign(n)`	Integer	-1, 0, or 1
`log(n)`	Float	Natural logarithm
`log10(n)`	Float	Base-10 logarithm
`exp(n)`	Float	e raised to the power n
`e()`	Float	Euler's number (2.718...)
`pi()`	Float	Pi (3.141...)
`rand()`	Float	Random float in [0, 1)
`toInteger(val)`	Integer	Convert to integer; `null` on failure
`toFloat(val)`	Float	Convert to float; `null` on failure

Examples

```
RETURN abs(-5)          -- 5
RETURN round(3.567, 2)  -- 3.57
RETURN sqrt(16)         -- 4.0
RETURN rand()           -- e.g. 0.7341...
```

Trigonometric Functions

Signature	Returns	Description
`sin(n)`	Float	Sine (radians)

Signature	Returns	Description
`cos(n)`	Float	Cosine (radians)
`tan(n)`	Float	Tangent (radians)
`asin(n)`	Float	Arcsine; result in radians
`acos(n)`	Float	Arccosine; result in radians
`atan(n)`	Float	Arctangent; result in radians
`atan2(y, x)`	Float	Two-argument arctangent
`degrees(n)`	Float	Radians to degrees
`radians(n)`	Float	Degrees to radians
`cot(n)`	Float	Cotangent
`haversin(n)`	Float	Half the versine of n
`sinh(n)`	Float	Hyperbolic sine
`cosh(n)`	Float	Hyperbolic cosine
`tanh(n)`	Float	Hyperbolic tangent
`coth(n)`	Float	Hyperbolic cotangent
`isNaN(n)`	Boolean	`true` if n is NaN

Examples

```
RETURN degrees(pi())   -- 180.0
RETURN atan2(1.0, 1.0) -- 0.7853...
```

List Functions

Signature	Returns	Description
`size(list)`	Integer	Number of elements
`head(list)`	Any	First element; `null` if empty
`tail(list)`	List	All elements except the first; empty list if input has 0 or 1 elements
`last(list)`	Any	Last element; `null` if empty
`range(start, end)`	List<Integer>	Inclusive integer range with step 1
`range(start, end, step)`	List<Integer>	Inclusive integer range with given step
`collect(expr)`	List	Aggregate: collect non-null values into a list

Signature	Returns	Description
`keys(node_or_map)`	List<String>	Property key names of a node, relationship, or map
`reduce(acc = init, x IN list | expr)`	Any	Fold list into single value
`[expr FOR x IN list]`	List	List comprehension without filter
`[expr FOR x IN list WHERE cond]`	List	List comprehension with filter

Examples

```
RETURN range(1, 5)                              -- [1, 2, 3, 4, 5]
RETURN range(0, 10, 2)                          -- [0, 2, 4, 6, 8, 10]
RETURN head([1, 2, 3])                          -- 1
RETURN tail([1, 2, 3])                          -- [2, 3]
RETURN reduce(total = 0, x IN [1,2,3] | total + x) -- 6
RETURN [x * 2 FOR x IN [1,2,3] WHERE x > 1]        -- [4, 6]
```

Aggregation Functions

Aggregation functions collapse multiple rows into one. They are valid in `RETURN` and `WITH`.

Signature	Returns	Description
`count(expr)`	Integer	Count of non-null values
`count(*)`	Integer	Count of rows
`sum(expr)`	Number	Sum of numeric values
`avg(expr)`	Float	Arithmetic mean of numeric values
`min(expr)`	Any	Minimum value
`max(expr)`	Any	Maximum value
`collect(expr)`	List	List of non-null values
`stdev(expr)`	Float	Sample standard deviation
`stdevp(expr)`	Float	Population standard deviation

Examples

```
MATCH (n:Person) RETURN count(n), avg(n.age), collect(n.name)
MATCH (n:Person) RETURN count(*) AS total
```

Entity Functions

Signature	Returns	Description
`id(entity)`	Integer	Internal numeric ID of a node or relationship
`elementId(entity)`	String	String form of internal ID
`labels(node)`	List<String>	All labels of a node
`type(rel)`	String	Relationship type name
`properties(entity)`	Map	All properties as a map
`startNode(rel)`	Node	Source node of a relationship
`endNode(rel)`	Node	Target node of a relationship
`nodes(path)`	List<Node>	Ordered list of nodes in a path
`relationships(path)`	List<Relationship>	Ordered list of relationships in a path
`length(path)`	Integer	Number of relationships in a path

Examples

```
MATCH (n:Person) RETURN id(n), labels(n)
MATCH ()-[r]->() RETURN type(r)
MATCH p = (a)-[*]->(b) RETURN length(p), nodes(p)
```

Type Conversion Functions

Signature	Returns	Description
`toString(val)`	String	Convert to string; error on unconvertible types
`toInteger(val)`	Integer	Convert to integer; error on unconvertible types
`toFloat(val)`	Float	Convert to float; error on unconvertible types
`toBoolean(val)`	Boolean	Convert to boolean; error on unconvertible types
`toStringOrNull(val)`	String | null	Convert to string; `null` on failure
`toIntegerOrNull(val)`	Integer | null	Convert to integer; `null` on failure

Signature	Returns	Description
`toFloatOrNull(val)`	Float | null	Convert to float; `null` on failure
`toBooleanOrNull(val)`	Boolean | null	Convert to boolean; `null` on failure
`valueType(val)`	String	Returns a string naming the Cypher type: `"INTEGER"`, `"FLOAT"`, `"STRING"`, `"BOOLEAN"`, `"NULL"`, `"LIST"`, `"MAP"`, `"NODE"`, `"RELATIONSHIP"`, `"PATH"`

Examples

```
RETURN toInteger('42')          -- 42
RETURN toFloatOrNull('abc')     -- null
RETURN valueType(3.14)          -- 'FLOAT'
RETURN toBoolean('true')        -- true
```

Temporal Functions

Signature	Returns	Description
`date({year, month, day})`	Date	Construct a date
`time({hour, minute, second})`	Time	Construct a time
`datetime({year, month, day, hour, minute, second})`	DateTime	Construct a datetime
`localdatetime({year, month, day, hour, minute, second})`	LocalDateTime	Construct a local datetime (no timezone)
`duration({days, hours, minutes, seconds})`	Duration	Construct a duration; all fields optional
`datetime.fromepoch(seconds)`	DateTime	DateTime from Unix epoch seconds
`datetime.fromepochmillis(ms)`	DateTime	DateTime from Unix epoch milliseconds
`duration.inDays(d1, d2)`	Duration	Duration between two dates in days
`duration.inSeconds(d1, d2)`	Duration	Duration between two datetimes in seconds
`date.truncate(unit, date)`	Date	Truncate date to `unit`: `'year'`,

Signature	Returns	Description
		`'month'`, `'week'`, `'day'`

Examples

```
RETURN date({year: 2024, month: 3, day: 15})
RETURN datetime.fromepoch(1700000000)
RETURN duration({days: 7, hours: 12})
RETURN date.truncate('month', date({year: 2024, month: 3, day: 15}))
```

Spatial Functions

Signature	Returns	Description
`point({x, y})`	Point	2D Cartesian point
`point({x, y, z})`	Point	3D Cartesian point
`point({latitude, longitude})`	Point	2D geographic point (WGS-84)
`point({latitude, longitude, height})`	Point	3D geographic point (WGS-84)
`distance(p1, p2)`	Float	Distance between two points (meters for geographic, units for Cartesian)
`point.withinBBox(point, lowerLeft, upperRight)`	Boolean	`true` if point is inside bounding box

Examples

```
RETURN point({x: 1.0, y: 2.0})
RETURN distance(point({latitude: 48.8, longitude: 2.3}), point({latitude: 51.5, longitude: -0.1}))
RETURN point.withinBBox(
  point({x: 5, y: 5}),
  point({x: 0, y: 0}),
  point({x: 10, y: 10})
)  -- true
```

Predicate Functions

Signature	Returns	Description
`exists(expr)`	Boolean	`true` if the property or pattern exists and is not `null`
`exists{pattern}`	Boolean	`true` if the pattern matches at least one result (full pattern syntax)
`coalesce(v1, v2, ...)`	Any	First non-null argument; `null` if all arguments are `null`
`nullIf(v1, v2)`	Any | null	Returns `null` if `v1 = v2`; otherwise returns `v1`

Examples

```
MATCH (n:Person) WHERE exists(n.email) RETURN n.name
MATCH (n:Person) WHERE exists{(n)-[:KNOWS]->(:Person)} RETURN n.name
RETURN coalesce(null, null, 'default')   -- 'default'
RETURN nullIf(5, 5)                      -- null
RETURN nullIf(5, 6)                      -- 5
```

CASE Expressions

Simple form

```
CASE expr
  WHEN value1 THEN result1
  WHEN value2 THEN result2
  ELSE default
END
```

Generic form

```
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  ELSE default
END
```

`ELSE` is optional; omitting it returns `null` for unmatched rows.

Examples

```
MATCH (n:Person)
RETURN CASE n.role
  WHEN 'admin' THEN 'Administrator'
  WHEN 'mod'   THEN 'Moderator'
  ELSE 'User'
END AS roleLabel


MATCH (n:Person)
RETURN CASE
  WHEN n.age < 18 THEN 'minor'
  WHEN n.age < 65 THEN 'adult'
  ELSE 'senior'
END AS ageGroup
```

Graph Algorithm Functions

Called inside Cypher queries using `CALL` syntax or inline. See Graph Algorithms for full parameter and return type documentation.

Signature	Description
`pageRank([damping, iterations])`	PageRank centrality
`labelPropagation([iterations])`	Label propagation community detection
`louvain([resolution])`	Louvain modularity community detection
`dijkstra(source, target[, weight_property])`	Weighted shortest path
`astar(source, target[, lat_prop, lon_prop])`	A* heuristic shortest path
`degreeCentrality()`	In/out/total degree per node
`betweennessCentrality()`	Betweenness centrality per node
`closenessCentrality()`	Closeness centrality per node
`eigenvectorCentrality([iterations])`	Eigenvector centrality per node
`weaklyConnectedComponents()`	WCC component assignment
`stronglyConnectedComponents()`	SCC component assignment
`bfs(start[, max_depth])`	Breadth-first traversal
`dfs(start[, max_depth])`	Depth-first traversal
`nodeSimilarity([node1, node2, threshold, top_k])`	Jaccard similarity
`knn(node, k)`	k-nearest neighbors by similarity

Signature	Description
`triangleCount()`	Triangle count and clustering coefficient per node
`apsp()`	All-pairs shortest paths
`shortestPath(pattern)`	Shortest path as a path value

Cypher Operators

Precedence

Operators are listed from highest to lowest precedence. Operators at the same level associate left-to-right unless noted.

Level	Operator(s)	Associativity
1 (highest)	`.` (property access), `[` (subscript/slice)	Left
2	Unary `-` (negation), `NOT`	Right
3	`*`, `/`, `%`	Left
4	`+`, `-`	Left
5	`=`, `<>`, `<`, `>`, `<=`, `>=`	Left
6	`IS NULL`, `IS NOT NULL`	—
7	`STARTS WITH`, `ENDS WITH`, `CONTAINS`, `=~`, `IN`	Left
8	`AND`	Left
9	`XOR`	Left
10 (lowest)	`OR`	Left

Use parentheses to override precedence.

Arithmetic Operators

Operator	Syntax	Description	Example	Result
Addition	`a + b`	Numeric addition; string concatenation	`2 + 3`	`5`
Subtraction	`a - b`	Numeric subtraction	`10 - 4`	`6`
Multiplication	`a * b`	Numeric multiplication	`3 * 4`	`12`
Division	`a / b`	Numeric division; integer inputs yield float	`7 / 2`	`3.5`
Modulo	`a % b`	Remainder	`10 % 3`	`1`
Unary negation	`-a`	Negate a number	`-n.age`	negated

Type behavior

- `Integer + Integer` → Integer
- `Integer + Float` → Float
- `String + String` → String concatenation
- Any arithmetic with `null` → `null`

Comparison Operators

Operator	Syntax	Description
Equals	`a = b`	Value equality
Not equals	`a <> b`	Value inequality
Less than	`a < b`	
Greater than	`a > b`	
Less or equal	`a <= b`	
Greater or equal	`a >= b`	

Type behavior

- Comparing `null` with any operator returns `null` (not `true` or `false`).
- Comparisons between incompatible types return `null`.
- Strings compare lexicographically.

Examples

```
WHERE n.age >= 18
WHERE n.name <> 'Alice'
WHERE n.score = 100
```

Boolean Operators

Operator	Syntax	Description
AND	`a AND b`	`true` if both operands are `true`
OR	`a OR b`	`true` if at least one operand is `true`
NOT	`NOT a`	Logical negation
XOR	`a XOR b`	`true` if exactly one operand is `true`

Three-valued logic (null behavior)

a	b	a AND b	a OR b
true	null	null	true
false	null	false	null
null	null	null	null

`NOT null → null`

Examples

```
WHERE n.age > 18 AND n.active = true
WHERE n.role = 'admin' OR n.role = 'mod'
WHERE NOT n.deleted
WHERE (n.a = 1) XOR (n.b = 1)
```

String Operators

Operator	Syntax	Description	Example
Starts with	`s STARTS WITH prefix`	Prefix match	`n.name STARTS WITH 'Al'`
Ends with	`s ENDS WITH suffix`	Suffix match	`n.email ENDS WITH '.com'`
Contains	`s CONTAINS sub`	Substring search	`n.bio CONTAINS 'engineer'`
Regex match	`s =~ pattern`	PCRE regex; full-string match	`n.name =~ 'Al.*'`
Concatenation	`s1 + s2`	Join two strings	`n.first + ' ' + n.last`

All string operators are case-sensitive. `=~` uses PCRE syntax via the `regexp()` SQL function registered by the extension.

Examples

```
WHERE n.name STARTS WITH 'A'
WHERE n.email ENDS WITH '.org'
WHERE n.bio CONTAINS 'graph'
WHERE n.code =~ '[A-Z]{3}[0-9]+'
```

List Operators

Operator	Syntax	Description	Example
Membership	`x IN list`	`true` if `x` is an element of `list`	`n.role IN ['admin', 'mod']`
Concatenation	`list1 + list2`	Combine two lists	`[1,2] + [3,4]` → `[1,2,3,4]`
Index access	`list[index]`	Element at 0-based index; negative index counts from end	`list[0]`, `list[-1]`
Slice	`list[start..end]`	Sublist from `start` (inclusive) to `end` (exclusive)	`list[1..3]`

Examples

```
WHERE n.status IN ['active', 'pending']
RETURN [1, 2] + [3, 4]       -- [1, 2, 3, 4]
RETURN [10, 20, 30][1]       -- 20
RETURN [10, 20, 30, 40][1..3] -- [20, 30]
```

Null Operators

Operator	Syntax	Description
Is null	`expr IS NULL`	`true` if expression is `null`
Is not null	`expr IS NOT NULL`	`true` if expression is not `null`

Examples

```
WHERE n.email IS NOT NULL
WHERE n.deletedAt IS NULL
```

Property Access Operators

Operator	Syntax	Description
Dot notation	`entity.property`	Access a property by name
String subscript	`entity['property']`	Access a property by string key
Nested dot	`n.a.b`	Access nested JSON field (requires `a` to be a JSON-type property)

String-key subscript (`n['key']`) is normalized at transform time to the same SQL as dot notation. Both forms are equivalent.

Examples

```
RETURN n.name
RETURN n['name']
RETURN n.address.city     -- requires address stored as JSON
```

Graph Algorithms Reference

GraphQLite provides 18 built-in graph algorithms accessible via Cypher functions, the Python `Graph` API, and the Rust `Graph` API.

> For guidance on choosing the right algorithm for your use case, see Using Graph Algorithms.

PageRank

Cypher

```
CALL pageRank([damping, iterations]) YIELD node, score
```

Python

```
graph.pagerank(damping=0.85, iterations=20)
```

Rust

```
graph.pagerank(damping: f64, iterations: usize) -> Result<Vec<PageRankResult>>
```

Parameters

Parameter	Type	Default	Description
`damping`	Float	`0.85`	Damping factor
`iterations`	Integer	`20`	Number of power iterations

Return shape

Python: `list[dict]` with keys `node_id`, `user_id`, `score`

Rust: `Vec<PageRankResult>` — fields: `node_id: i64`, `user_id: String`, `score: f64`

Complexity: O(iterations × (V + E))

Example

```
results = graph.pagerank(damping=0.85, iterations=30)
for r in sorted(results, key=lambda x: x['score'], reverse=True)[:5]:
    print(r['user_id'], r['score'])
```

Degree Centrality

Cypher

```
CALL degreeCentrality() YIELD node, in_degree, out_degree, degree
```

Python

```
graph.degree_centrality()
```

Rust

```
graph.degree_centrality() -> Result<Vec<DegreeCentralityResult>>
```

Parameters: none

Return shape

Python: `list[dict]` with keys `node_id`, `user_id`, `in_degree`, `out_degree`, `degree`

Rust: `Vec<DegreeCentralityResult>` — fields: `node_id: i64`, `user_id: String`, `in_degree: usize`, `out_degree: usize`, `degree: usize`

Complexity: O(V + E)

Example

```
for r in graph.degree_centrality():
    print(r['user_id'], 'in:', r['in_degree'], 'out:', r['out_degree'])
```

Betweenness Centrality

Cypher

```
CALL betweennessCentrality() YIELD node, score
```

Python

```
graph.betweenness_centrality()
```

Rust

```
graph.betweenness_centrality() -> Result<Vec<BetweennessCentralityResult>>
```

Parameters: none

Return shape

Python: `list[dict]` with keys `node_id`, `user_id`, `score`

Rust: `Vec<BetweennessCentralityResult>` — fields: `node_id: i64`, `user_id: String`, `score: f64`

Complexity: O(V × E)

Example

```
results = graph.betweenness_centrality()
```

Closeness Centrality

Cypher

```
CALL closenessCentrality() YIELD node, score
```

Python

```
graph.closeness_centrality()
```

Rust

```
graph.closeness_centrality() -> Result<Vec<ClosenessCentralityResult>>
```

Parameters: none

Return shape

Python: `list[dict]` with keys `node_id`, `user_id`, `score`

Rust: `Vec<ClosenessCentralityResult>` — fields: `node_id: i64`, `user_id: String`, `score: f64`

Complexity: O(V × (V + E))

Example

```
results = graph.closeness_centrality()
```

Eigenvector Centrality

Cypher

```
CALL eigenvectorCentrality([iterations]) YIELD node, score
```

Python

```
graph.eigenvector_centrality(iterations=100)
```

Rust

```
graph.eigenvector_centrality(iterations: usize) ->
Result<Vec<EigenvectorCentralityResult>>
```

Parameters

Parameter	Type	Default	Description
`iterations`	Integer	`100`	Power iteration count

Return shape

Python: `list[dict]` with keys `node_id`, `user_id`, `score`

Rust: `Vec<EigenvectorCentralityResult>` — fields: `node_id: i64`, `user_id: String`, `score: f64`

Complexity: O(iterations × E)

Louvain Community Detection

Cypher

```
CALL louvain([resolution]) YIELD node, community
```

Python

```
graph.louvain(resolution=1.0)
```

Rust

```
graph.louvain(resolution: f64) -> Result<Vec<CommunityResult>>
```

Parameters

Parameter	Type	Default	Description
`resolution`	Float	`1.0`	Resolution parameter controlling community granularity

Return shape

Python: `list[dict]` with keys `node_id`, `user_id`, `community`

Rust: `Vec<CommunityResult>` — fields: `node_id: i64`, `user_id: String`, `community: i64`

Example

```
communities = graph.louvain(resolution=0.5)
```

Leiden Community Detection

Python only

```
graph.leiden_communities(resolution=1.0, random_seed=None)
```

Parameters

Parameter	Type	Default	Description
`resolution`	Float	`1.0`	Resolution parameter
`random_seed`	Integer | None	`None`	Seed for reproducibility

Return shape: `list[dict]` with keys `node_id`, `user_id`, `community`

Label Propagation

Cypher

```
CALL labelPropagation([iterations]) YIELD node, community
```

Python

```
graph.community_detection(iterations=10)
```

Rust

```
graph.community_detection(iterations: usize) -> Result<Vec<CommunityResult>>
```

Parameters

Parameter	Type	Default	Description
`iterations`	Integer	`10`	Maximum iterations

Return shape

Python: `list[dict]` with keys `node_id`, `user_id`, `community`

Rust: `Vec<CommunityResult>` — fields: `node_id: i64`, `user_id: String`, `community: i64`

Weakly Connected Components

Cypher

```
CALL weaklyConnectedComponents() YIELD node, component
```

Python

```
graph.weakly_connected_components()
```

Rust

```
graph.weakly_connected_components() -> Result<Vec<ComponentResult>>
```

Parameters: none

Return shape

Python: `list[dict]` with keys `node_id`, `user_id`, `component`

Rust: `Vec<ComponentResult>` — fields: `node_id: i64`, `user_id: String`, `component: i64`

Complexity: O(V + E)

Strongly Connected Components

Cypher

```
CALL stronglyConnectedComponents() YIELD node, component
```

Python

```
graph.strongly_connected_components()
```

Rust

```
graph.strongly_connected_components() -> Result<Vec<ComponentResult>>
```

Parameters: none

Return shape

Python: `list[dict]` with keys `node_id`, `user_id`, `component`

Rust: `Vec<ComponentResult>` — fields: `node_id: i64`, `user_id: String`, `component: i64`

Complexity: O(V + E) (Tarjan or Kosaraju)

Shortest Path

Cypher (path function)

```
MATCH p = shortestPath((a)-[*]->(b))
RETURN p
```

Cypher (Dijkstra)

```
CALL dijkstra(source, target[, weight_property]) YIELD path, distance
```

Python

```
graph.shortest_path(source, target, weight_property=None)
```

Rust

```
graph.shortest_path(source: &str, target: &str, weight_property: Option<&str>)
-> Result<ShortestPathResult>
```

Parameters

Parameter	Type	Default	Description
`source`	String	required	Source node user ID
`target`	String	required	Target node user ID
`weight_property`	String | None	`None`	Edge property to use as weight; unweighted BFS if `None`

Return shape

Python: `dict` with keys `path` (list of user IDs), `distance` (float), `found` (bool)

Rust: `ShortestPathResult` — fields: `path: Vec<String>`, `distance: f64`, `found: bool`

Example

```
result = graph.shortest_path('alice', 'bob', weight_property='cost')
if result['found']:
    print(result['path'], result['distance'])
```

A* (A-Star)

Cypher

```
CALL astar(source, target[, lat_prop, lon_prop]) YIELD path, distance
```

Python

```
graph.astar(source, target, lat_prop=None, lon_prop=None)
```

Rust

```
graph.astar(source: &str, target: &str, lat_prop: Option<&str>, lon_prop:
Option<&str>) -> Result<AStarResult>
```

Parameters

Parameter	Type	Default	Description
`source`	String	required	Source node user ID
`target`	String	required	Target node user ID
`lat_prop`	String | None	`None`	Node property for latitude
`lon_prop`	String | None	`None`	Node property for longitude

Return shape

Python: `dict` with keys `path`, `distance`, `found`, `nodes_explored`

Rust: `AStarResult` — fields: `path: Vec<String>`, `distance: f64`, `found: bool`, `nodes_explored: usize`

All-Pairs Shortest Path

Cypher

```
CALL apsp() YIELD source, target, distance
```

Python

```
graph.all_pairs_shortest_path()
```

Rust

```
graph.all_pairs_shortest_path() -> Result<Vec<ApspResult>>
```

Parameters: none

Return shape

Python: `list[dict]` with keys `source`, `target`, `distance`

Rust: `Vec<ApspResult>` — fields: `source: String`, `target: String`, `distance: f64`

Complexity: O(V × (V + E))

BFS (Breadth-First Search)

Cypher

```
CALL bfs(start[, max_depth]) YIELD node, depth, order
```

Python

```
graph.bfs(start, max_depth=-1)
```

Rust

```
graph.bfs(start: &str, max_depth: i64) -> Result<Vec<TraversalResult>>
```

Parameters

Parameter	Type	Default	Description
`start`	String	required	Starting node user ID
`max_depth`	Integer	`-1`	Maximum depth; `-1` means unlimited

Return shape

Python: `list[dict]` with keys `user_id`, `depth`, `order`

Rust: `Vec<TraversalResult>` — fields: `user_id: String`, `depth: usize`, `order: usize`

DFS (Depth-First Search)

Cypher

```
CALL dfs(start[, max_depth]) YIELD node, depth, order
```

Python

```
graph.dfs(start, max_depth=-1)
```

Rust

```
graph.dfs(start: &str, max_depth: i64) -> Result<Vec<TraversalResult>>
```

Parameters

Parameter	Type	Default	Description
`start`	String	required	Starting node user ID
`max_depth`	Integer	`-1`	Maximum depth; `-1` means unlimited

Return shape: same as BFS — `list[dict]` / `Vec<TraversalResult>` with `user_id`, `depth`, `order`

Node Similarity

Cypher

```
CALL nodeSimilarity([node1, node2, threshold, top_k]) YIELD node1, node2, similarity
```

Python

```
graph.node_similarity(node1_id=None, node2_id=None, threshold=0.0, top_k=0)
```

Rust

```
graph.node_similarity(node1_id: Option<i64>, node2_id: Option<i64>, threshold: f64, top_k: usize) -> Result<Vec<NodeSimilarityResult>>
```

Parameters

Parameter	Type	Default	Description
`node1_id`	Integer | None	`None`	Fix first node; `None` means all pairs
`node2_id`	Integer | None	`None`	Fix second node; `None` means all pairs
`threshold`	Float	`0.0`	Minimum similarity to include
`top_k`	Integer	`0`	Return at most `top_k` results; `0` means all

Algorithm: Jaccard similarity based on shared neighbors.

Return shape

Python: `list[dict]` with keys `node1`, `node2`, `similarity`

Rust: `Vec<NodeSimilarityResult>` — fields: `node1: String`, `node2: String`, `similarity: f64`

KNN (k-Nearest Neighbors)

Cypher

```
CALL knn(node, k) YIELD neighbor, similarity, rank
```

Python

```
graph.knn(node_id, k=10)
```

Rust

```
graph.knn(node_id: i64, k: usize) -> Result<Vec<KnnResult>>
```

Parameters

Parameter	Type	Default	Description
`node_id`	Integer	required	Source node internal ID
`k`	Integer	`10`	Number of neighbors to return

Return shape

Python: `list[dict]` with keys `neighbor`, `similarity`, `rank`

Rust: `Vec<KnnResult>` — fields: `neighbor: String`, `similarity: f64`, `rank: usize`

Triangle Count

Cypher

```
CALL triangleCount() YIELD node, triangles, clustering_coefficient
```

Python

```
graph.triangle_count()
```

Rust

```
graph.triangle_count() -> Result<Vec<TriangleCountResult>>
```

Parameters: none

Return shape

Python: `list[dict]` with keys `node_id`, `user_id`, `triangles`, `clustering_coefficient`

Rust: `Vec<TriangleCountResult>` — fields: `node_id: i64`, `user_id: String`, `triangles: usize`, `clustering_coefficient: f64`

Complexity: O(V × degree²)

Example

```
for r in graph.triangle_count():
    print(r['user_id'], r['triangles'], r['clustering_coefficient'])
```

Python API Reference

Version: **0.4.4**

Module-level Functions

`graphqlite.connect`

```
graphqlite.connect(database=":memory:", extension_path=None) -> Connection
```

Open a new SQLite connection with GraphQLite loaded.

Parameter	Type	Default	Description
`database`	str	`":memory:"`	SQLite database path or `":memory:"`
`extension_path`	str | None	`None`	Path to the `.dylib` / `.so` / `.dll`; auto-detected if `None`

Returns: `Connection`

`graphqlite.wrap`

```
graphqlite.wrap(conn: sqlite3.Connection, extension_path=None) -> Connection
```

Wrap an existing `sqlite3.Connection` with GraphQLite loaded into it.

Parameter	Type	Default	Description
`conn`	`sqlite3.Connection`	required	An open SQLite connection
`extension_path`	str | None	`None`	Path to extension; auto-detected if `None`

Returns: `Connection`

---

graphqlite.load

```
graphqlite.load(conn, entry_point=None) -> None
```

Load the GraphQLite extension into `conn` without wrapping. Useful when you want to keep a plain `sqlite3.Connection`.

Parameter	Type	Default	Description
`conn`	`sqlite3.Connection`	required	Connection to load into
`entry_point`	str | None	`None`	Extension entry point symbol; auto-detected if `None`

---

graphqlite.loadable_path

```
graphqlite.loadable_path() -> str
```

Return the filesystem path of the bundled extension library. Use to pass to `conn.load_extension()` manually.

Connection

A thin wrapper around `sqlite3.Connection` that adds Cypher query support.

Connection.cypher

```
conn.cypher(query: str, params=None) -> CypherResult
```

Execute a Cypher query.

Parameter	Type	Default	Description
`query`	str	required	Cypher query string
`params`	dict | None	`None`	Parameter map; values substituted for `$name` placeholders

Returns: `CypherResult`

Raises: `sqlite3.Error` on parse or execution failure.

Example

```
result = conn.cypher("MATCH (n:Person) WHERE n.age > $min RETURN n.name",
{"min": 25})
for row in result:
    print(row["n.name"])
```

Connection.execute

```
conn.execute(sql: str, parameters=()) -> sqlite3.Cursor
```

Execute raw SQL. Passes through to the underlying `sqlite3.Connection`.

Connection.commit

```
conn.commit() -> None
```

Commit the current transaction.

Connection.rollback

```
conn.rollback() -> None
```

Roll back the current transaction.

Connection.close

```
conn.close() -> None
```

Close the connection and release all resources.

Connection.sqlite_connection

```
conn.sqlite_connection -> sqlite3.Connection
```

The underlying `sqlite3.Connection` object.

CypherResult

Returned by `Connection.cypher()`. Represents the result set of a Cypher query.

Properties

Property	Type	Description
`.columns`	`list[str]`	Ordered list of column names

Methods

Method	Signature	Description
`len`	`len(result) -> int`	Number of rows
`iter`	`for row in result`	Iterate rows as `dict`
`index`	`result[i]`	Access row by 0-based index; returns `dict`
`to_list`	`result.to_list() -> list[dict]`	Return all rows as a list of dicts

Example

```
result = conn.cypher("MATCH (n:Person) RETURN n.name, n.age")
print(result.columns)      # ['n.name', 'n.age']
print(len(result))         # row count
for row in result:
    print(row["n.name"])
rows = result.to_list()    # list of dicts
```

Graph

High-level graph API built on top of `Connection`. Manages a single named graph in a SQLite database.

Constructor

```
graphqlite.Graph(db_path=":memory:", namespace="default", extension_path=None)
graphqlite.graph(db_path=":memory:", namespace="default", extension_path=None)
-> Graph
```

Parameter	Type	Default	Description
`db_path`	str	`":memory:"`	SQLite database path
`namespace`	str	`"default"`	Graph namespace identifier
`extension_path`	str | None	`None`	Path to extension; auto-detected if `None`

Node Operations

`Graph.upsert_node`

```
graph.upsert_node(node_id: str, props: dict, label: str = "Entity") -> int
```

Insert or update a node. `node_id` is a user-defined string identifier. Returns the internal integer ID.

`Graph.get_node`

```
graph.get_node(id: str) -> dict | None
```

Return all properties of the node with user ID `id`, or `None` if not found. The returned dict includes `"_id"` (internal) and `"_label"`.

`Graph.has_node`

```
graph.has_node(id: str) -> bool
```

Return `True` if a node with user ID `id` exists.

`Graph.delete_node`

```
graph.delete_node(id: str) -> None
```

Delete the node and all its incident edges.

Graph.get_all_nodes

```
graph.get_all_nodes(label: str = None) -> list[dict]
```

Return all nodes. If `label` is given, filter to that label only.

Edge Operations

Graph.upsert_edge

```
graph.upsert_edge(source: str, target: str, props: dict, rel_type: str =
"RELATED") -> int
```

Insert or update an edge from `source` to `target` of type `rel_type`. Returns internal edge ID.

Note: Uses merge semantics — existing properties not included in `props` are preserved, not removed.

Graph.get_edge

```
graph.get_edge(src: str, dst: str, rel_type: str = None) -> dict | None
```

Return edge properties, or `None`. If `rel_type` is `None`, returns the first matching edge.

Graph.has_edge

```
graph.has_edge(src: str, dst: str, rel_type: str = None) -> bool
```

Return `True` if an edge from `src` to `dst` (optionally of `rel_type`) exists.

Graph.delete_edge

```
graph.delete_edge(src: str, dst: str, rel_type: str = None) -> None
```

Delete the matching edge(s).

Graph.get_all_edges

```
graph.get_all_edges() -> list[dict]
```

Return all edges with their properties.

Query Methods

`Graph.node_degree`

```
graph.node_degree(id: str) -> int
```

Total degree (in + out) of the node.

`Graph.get_neighbors`

```
graph.get_neighbors(id: str) -> list[dict]
```

Return nodes connected to `id` in either direction (undirected — includes both incoming and outgoing edges).

`Graph.get_node_edges`

```
graph.get_node_edges(id: str) -> list[dict]
```

Return all edges incident to `id` (in and out).

`Graph.get_edges_from`

```
graph.get_edges_from(id: str) -> list[dict]
```

Return outgoing edges from `id`.

`Graph.get_edges_to`

```
graph.get_edges_to(id: str) -> list[dict]
```

Return incoming edges to `id`.

`Graph.get_edges_by_type`

```
graph.get_edges_by_type(id: str, rel_type: str) -> list[dict]
```

Return edges of a specific type incident to `id`.

`Graph.stats`

```
graph.stats() -> dict
```

Return graph statistics. Keys: `node_count`, `edge_count`.

`Graph.query`

```
graph.query(cypher: str, params: dict = None) -> list[dict]
```

Execute a Cypher query and return all rows as a list of dicts.

Graph Cache

`Graph.load_graph`

```
graph.load_graph() -> dict
```

Load the graph into the in-memory adjacency cache for algorithm use. Returns status dict.

`Graph.unload_graph`

```
graph.unload_graph() -> dict
```

Release the in-memory adjacency cache.

`Graph.reload_graph`

```
graph.reload_graph() -> dict
```

Unload and reload the cache.

`Graph.graph_loaded`

```
graph.graph_loaded() -> bool
```

Return `True` if the adjacency cache is currently loaded.

Graph Algorithms

All algorithm methods return lists of dicts. See Graph Algorithms for full parameter and return field documentation.

Method	Signature
PageRank	`graph.pagerank(damping=0.85, iterations=20)`
Degree centrality	`graph.degree_centrality()`
Betweenness centrality	`graph.betweenness_centrality()`
Closeness centrality	`graph.closeness_centrality()`
Eigenvector centrality	`graph.eigenvector_centrality(iterations=100)`
Label propagation	`graph.community_detection(iterations=10)`
Louvain	`graph.louvain(resolution=1.0)`
Leiden	`graph.leiden_communities(resolution=1.0, random_seed=None)`
Weakly connected components	`graph.weakly_connected_components()`
Strongly connected components	`graph.strongly_connected_components()`
Shortest path	`graph.shortest_path(source, target, weight_property=None)`
A*	`graph.astar(source, target, lat_prop=None, lon_prop=None)`
All-pairs shortest path	`graph.all_pairs_shortest_path()`
BFS	`graph.bfs(start, max_depth=-1)`
DFS	`graph.dfs(start, max_depth=-1)`
Node similarity	`graph.node_similarity(node1_id=None, node2_id=None, threshold=0.0, top_k=0)`
KNN	`graph.knn(node_id, k=10)`
Triangle count	`graph.triangle_count()`

Method Aliases

The following aliases are available on `Graph` for convenience:

Alias	Canonical Method
dijkstra	shortest_path
a_star	astar
apsp	all_pairs_shortest_path
breadth_first_search	bfs
depth_first_search	dfs
triangles	triangle_count

Bulk Operations

Graph.insert_nodes_bulk

```
graph.insert_nodes_bulk(
    nodes: list[tuple[str, dict[str, Any], str]]
) -> dict[str, int]
```

Insert multiple nodes in a single transaction, bypassing Cypher. Each tuple is `(external_id, properties, label)`. Returns a dict mapping each external ID to its internal SQLite rowid.

```
id_map = g.insert_nodes_bulk([
    ("alice", {"name": "Alice", "age": 30}, "Person"),
    ("bob",   {"name": "Bob",   "age": 25}, "Person"),
])
# id_map = {"alice": 1, "bob": 2}
```

Graph.insert_edges_bulk

```
graph.insert_edges_bulk(
    edges: list[tuple[str, str, dict[str, Any], str]],
    id_map: dict[str, int] = None
) -> int
```

Insert multiple edges in a single transaction. Each tuple is `(source_id, target_id, properties, rel_type)`. The optional `id_map` (from `insert_nodes_bulk`) maps external IDs to internal rowids for fast resolution. If `id_map` is `None`, IDs are looked up from the database. Returns the number of edges inserted. Raises `KeyError` if an external ID is not found in `id_map`.

Graph.insert_graph_bulk

```
graph.insert_graph_bulk(
    nodes: list[tuple[str, dict[str, Any], str]],
    edges: list[tuple[str, str, dict[str, Any], str]]
) -> BulkInsertResult
```

Insert nodes and edges together. Internally calls `insert_nodes_bulk` then `insert_edges_bulk`. Returns a `BulkInsertResult` dataclass:

Field	Type	Description
`nodes_inserted`	`int`	Number of nodes inserted
`edges_inserted`	`int`	Number of edges inserted
`id_map`	`dict[str, int]`	Mapping from external IDs to internal rowids

Graph.resolve_node_ids

```
graph.resolve_node_ids(ids: list[str]) -> dict[str, int]
```

Look up internal rowids for existing nodes by their external IDs. Returns `{external_id: internal_rowid}`. Use this to build an `id_map` for `insert_edges_bulk` when connecting to nodes that were inserted in a previous operation.

Batch Operations

> **Non-atomicity warning:** Batch upsert methods call `upsert_node` / `upsert_edge` in a loop. If an operation fails partway through, earlier operations will have already completed. For atomic batch inserts, use the bulk insert methods instead, or wrap the call in an explicit transaction.

Graph.upsert_nodes_batch

```
graph.upsert_nodes_batch(
    nodes: list[tuple[str, dict[str, Any], str]]
) -> None
```

Upsert multiple nodes. Each tuple is `(node_id, properties, label)`. Uses MERGE semantics (update if exists, create if not).

Graph.upsert_edges_batch

```
graph.upsert_edges_batch(
    edges: list[tuple[str, str, dict[str, Any], str]]
) -> None
```

Upsert multiple edges. Each tuple is `(source_id, target_id, properties, rel_type)`. Uses MERGE semantics.

Export

Graph.to_rustworkx

```
graph.to_rustworkx() -> PyDiGraph
```

Export the graph to a `rustworkx.PyDiGraph`. Requires `rustworkx` to be installed.

GraphManager

Manages multiple named graphs stored as separate SQLite files under a base directory.

Constructor

```
graphqlite.GraphManager(base_path: str, extension_path: str = None)
graphqlite.graphs(base_path: str, extension_path: str = None) -> GraphManager
```

Parameter	Type	Description
`base_path`	str	Directory containing graph database files
`extension_path`	str | None	Path to extension; auto-detected if `None`

Methods

Method	Signature	Description
`list`	`manager.list() -> list[str]`	Names of all graphs in `base_path`

Method	Signature	Description
`exists`	`manager.exists(name: str) -> bool`	True if a graph named `name` exists
`create`	`manager.create(name: str) -> Graph`	Create and return a new graph
`open`	`manager.open(name: str) -> Graph`	Open existing graph; raises if not found
`open_or_create`	`manager.open_or_create(name: str) -> Graph`	Open or create
`drop`	`manager.drop(name: str) -> None`	Delete the graph database file
`query`	`manager.query(cypher: str, graphs: list[str] = None, params: dict = None) -> list`	Query across multiple graphs; `graphs=None` queries all
`query_sql`	`manager.query_sql(sql: str, graphs: list[str], parameters: tuple = ()) -> list`	Raw SQL across multiple graphs
`close`	`manager.close() -> None`	Close all open connections

Dunder Methods

Method	Description
`__iter__`	Iterate over all graph names in `base_path` (same as `list()`)
`__contains__`	`name in manager` — True if a graph named `name` exists (same as `exists()`)
`__len__`	`len(manager)` — Number of graphs in `base_path`
`__enter__` / `__exit__`	Context manager support; calls `close()` on exit

Utilities

graphqlite.escape_string

```
graphqlite.escape_string(s: str) -> str
```

Escape a string for safe embedding in a Cypher query literal (single-quote escaping).

graphqlite.sanitize_rel_type

```
graphqlite.sanitize_rel_type(type: str) -> str
```

Normalize a relationship type string to a safe identifier (uppercase, underscores only).

graphqlite.format_props

```
graphqlite.format_props(props: dict, escape_fn=escape_string) -> str
```

Format a properties dict as a Cypher property string. For example, `{"name": "Alice", "age": 30}` becomes `{name: 'Alice', age: 30}`. The `escape_fn` is applied to string values; defaults to `escape_string`.

graphqlite.CYPHER_RESERVED

```
graphqlite.CYPHER_RESERVED -> set[str]
```

Set of all Cypher reserved keywords. Use to check whether an identifier needs quoting.

Rust API Reference

Version: **0.4.4**

Crate: `graphqlite`

Connection

Low-level connection type wrapping `rusqlite::Connection` with Cypher support.

Constructors

```
Connection::open<P: AsRef<Path>>(path: P) -> Result<Connection>
Connection::open_in_memory() -> Result<Connection>
Connection::from_rusqlite(conn: rusqlite::Connection) -> Result<Connection>
Connection::open_with_extension<P: AsRef<Path>>(path: P, ext_path: &str) ->
Result<Connection>
```

Method	Description
`open`	Open or create a database at `path`
`open_in_memory`	Open an in-memory database
`from_rusqlite`	Wrap an existing `rusqlite::Connection`; loads the extension into it
`open_with_extension`	Open database and load extension from explicit path

Methods

`Connection::cypher`

```
fn cypher(&self, query: &str) -> Result<CypherResult>
```

Execute a Cypher query with no parameters.

`Connection::cypher_with_params`

Deprecated since 0.4.0. Use `cypher_builder()` instead.

```
fn cypher_with_params(&self, query: &str, params: &serde_json::Value) ->
Result<CypherResult>
```

Execute a Cypher query with parameters. `params` must be a JSON object; keys correspond to `$name` placeholders.

`Connection::cypher_builder`

```
fn cypher_builder(&self, query: &str) -> CypherQuery
```

Return a `CypherQuery` builder for chaining parameter additions before execution.

`Connection::execute`

```
fn execute(&self, sql: &str) -> Result<usize>
```

Execute raw SQL. Returns the number of rows changed.

`Connection::sqlite_connection`

```
fn sqlite_connection(&self) -> &rusqlite::Connection
```

Borrow the underlying `rusqlite::Connection`.

`CypherQuery`

Builder returned by `Connection::cypher_builder`.

```
cypher_query
    .param("name", "Alice")
    .param("age", 30)
    .run() -> Result<CypherResult>
```

`CypherResult`

Represents the rows returned by a Cypher query.

Methods

Method	Signature	Description
`len`	`fn len(&self) -> usize`	Number of rows
`is_empty`	`fn is_empty(&self) -> bool`	True if zero rows
`columns`	`fn columns(&self) -> &[String]`	Ordered column names
Index	`result[i]`	Returns `&Row` at index `i`
Iterate	`for row in &result`	Iterates `&Row`

Row

A single result row.

`Row::get`

```
fn get<T: FromSql>(&self, column: &str) -> Result<T>
```

Get a column value by name. `T` must implement `FromSql`.

Supported types for `T`

Rust type	Cypher type
`String`	String, converted from any scalar
`i64`	Integer
`f64`	Float
`bool`	Boolean
`Option<String>`	String or null
`Option<i64>`	Integer or null
`Option<f64>`	Float or null
`Option<bool>`	Boolean or null

Example

```
let name: String = row.get("n.name")?;
let age: Option<i64> = row.get("n.age")?;
```

Graph

High-level graph API mirroring the Python `Graph`.

Constructors

```
Graph::open(path: &str) -> Result<Graph>
Graph::open_in_memory() -> Result<Graph>
```

Node Operations

```
fn upsert_node(&self, node_id: &str, props: &serde_json::Value, label: &str) ->
Result<i64>
fn get_node(&self, id: &str) -> Result<Option<serde_json::Value>>
fn has_node(&self, id: &str) -> Result<bool>
fn delete_node(&self, id: &str) -> Result<()>
fn get_all_nodes(&self, label: Option<&str>) -> Result<Vec<serde_json::Value>>
```

Edge Operations

```
fn upsert_edge(&self, source: &str, target: &str, props: &serde_json::Value,
rel_type: &str) -> Result<i64>
fn get_edge(&self, src: &str, dst: &str, rel_type: Option<&str>) ->
Result<Option<serde_json::Value>>
fn has_edge(&self, src: &str, dst: &str, rel_type: Option<&str>) ->
Result<bool>
fn delete_edge(&self, src: &str, dst: &str, rel_type: Option<&str>) ->
Result<()>
fn get_all_edges(&self) -> Result<Vec<serde_json::Value>>
```

Query

```
fn query(&self, cypher: &str, params: Option<&serde_json::Value>) ->
Result<Vec<serde_json::Value>>
fn stats(&self) -> Result<serde_json::Value>
fn node_degree(&self, id: &str) -> Result<i64>
fn get_neighbors(&self, id: &str) -> Result<Vec<serde_json::Value>>
fn get_node_edges(&self, id: &str) -> Result<Vec<serde_json::Value>>
fn get_edges_from(&self, id: &str) -> Result<Vec<serde_json::Value>>
fn get_edges_to(&self, id: &str) -> Result<Vec<serde_json::Value>>
fn get_edges_by_type(&self, id: &str, rel_type: &str) ->
Result<Vec<serde_json::Value>>
```

Graph Cache

```
fn load_graph(&self) -> Result<serde_json::Value>
fn unload_graph(&self) -> Result<serde_json::Value>
fn reload_graph(&self) -> Result<serde_json::Value>
fn graph_loaded(&self) -> Result<bool>
```

Graph Algorithms

All return `Result<Vec<T>>` where `T` is a typed result struct. See Graph Algorithms for parameter defaults.

Method	Signature	Returns
PageRank	`fn pagerank(&self, damping: f64, iterations: usize) -> Result<Vec<PageRankResult>>`	`PageRankResult`
Degree centrality	`fn degree_centrality(&self) -> Result<Vec<DegreeCentralityResult>>`	`DegreeCentralityR`
Betweenness	`fn betweenness_centrality(&self) -> Result<Vec<BetweennessCentralityResult>>`	`BetweennessCentra`
Closeness	`fn closeness_centrality(&self) -> Result<Vec<ClosenessCentralityResult>>`	`ClosenessCentrali`
Eigenvector	`fn eigenvector_centrality(&self, iterations: usize) -> Result<Vec<EigenvectorCentralityResult>>`	`EigenvectorCentra`
Community (label prop)	`fn community_detection(&self, iterations: usize) -> Result<Vec<CommunityResult>>`	`CommunityResult`
Louvain	`fn louvain(&self, resolution: f64) -> Result<Vec<CommunityResult>>`	`CommunityResult`
WCC	`fn weakly_connected_components(&self) -> Result<Vec<ComponentResult>>`	`ComponentResult`
SCC	`fn strongly_connected_components(&self) -> Result<Vec<ComponentResult>>`	`ComponentResult`
Shortest path	`fn shortest_path(&self, source: &str, target: &str, weight_property: Option<&str>) -> Result<ShortestPathResult>`	`ShortestPathResul`
A*	`fn astar(&self, source: &str, target: &str, lat_prop: Option<&str>, lon_prop: Option<&str>) -> Result<AStarResult>`	`AStarResult`

Method	Signature	Returns
APSP	`fn all_pairs_shortest_path(&self) -> Result<Vec<ApspResult>>`	`ApspResult`
BFS	`fn bfs(&self, start: &str, max_depth: i64) -> Result<Vec<TraversalResult>>`	`TraversalResult`
DFS	`fn dfs(&self, start: &str, max_depth: i64) -> Result<Vec<TraversalResult>>`	`TraversalResult`
Node similarity	`fn node_similarity(&self, node1_id: Option<i64>, node2_id: Option<i64>, threshold: f64, top_k: usize) -> Result<Vec<NodeSimilarityResult>>`	`NodeSimilarityRes`
KNN	`fn knn(&self, node_id: i64, k: usize) -> Result<Vec<KnnResult>>`	`KnnResult`
Triangle count	`fn triangle_count(&self) -> Result<Vec<TriangleCountResult>>`	`TriangleCountResu`

GraphManager

Manages named graphs stored as SQLite files in a directory.

Constructor

```
GraphManager::open(path: &str) -> Result<GraphManager>
```

Methods

```
fn list(&self) -> Result<Vec<String>>
fn exists(&self, name: &str) -> bool
fn create(&self, name: &str) -> Result<Graph>
fn open(&self, name: &str) -> Result<Graph>
fn open_or_create(&self, name: &str) -> Result<Graph>
fn drop(&self, name: &str) -> Result<()>
fn query(&self, cypher: &str, graphs: Option<&[&str]>, params:
Option<&serde_json::Value>) -> Result<Vec<serde_json::Value>>
fn query_sql(&self, sql: &str, graphs: &[&str], parameters: &[&dyn
rusqlite::ToSql]) -> Result<Vec<serde_json::Value>>
fn close(self) -> Result<()>
```

Result Types

All are plain structs deriving `Debug`, `Clone`, `serde::Serialize`, `serde::Deserialize`.

PageRankResult

```
pub struct PageRankResult {
    pub node_id: i64,
    pub user_id: String,
    pub score: f64,
}
```

DegreeCentralityResult

```
pub struct DegreeCentralityResult {
    pub node_id: i64,
    pub user_id: String,
    pub in_degree: usize,
    pub out_degree: usize,
    pub degree: usize,
}
```

BetweennessCentralityResult

```
pub struct BetweennessCentralityResult {
    pub node_id: i64,
    pub user_id: String,
    pub score: f64,
}
```

ClosenessCentralityResult

```
pub struct ClosenessCentralityResult {
    pub node_id: i64,
    pub user_id: String,
    pub score: f64,
}
```

EigenvectorCentralityResult

```
pub struct EigenvectorCentralityResult {
    pub node_id: i64,
    pub user_id: String,
    pub score: f64,
}
```

CommunityResult

```
pub struct CommunityResult {
    pub node_id: i64,
    pub user_id: String,
    pub community: i64,
}
```

ComponentResult

```
pub struct ComponentResult {
    pub node_id: i64,
    pub user_id: String,
    pub component: i64,
}
```

ShortestPathResult

```
pub struct ShortestPathResult {
    pub path: Vec<String>,
    pub distance: f64,
    pub found: bool,
}
```

AStarResult

```
pub struct AStarResult {
    pub path: Vec<String>,
    pub distance: f64,
    pub found: bool,
    pub nodes_explored: usize,
}
```

ApspResult

```
pub struct ApspResult {
    pub source: String,
    pub target: String,
    pub distance: f64,
}
```

TraversalResult

```
pub struct TraversalResult {
    pub user_id: String,
    pub depth: usize,
    pub order: usize,
}
```

NodeSimilarityResult

```
pub struct NodeSimilarityResult {
    pub node1: String,
    pub node2: String,
    pub similarity: f64,
}
```

KnnResult

```
pub struct KnnResult {
    pub neighbor: String,
    pub similarity: f64,
    pub rank: usize,
}
```

TriangleCountResult

```
pub struct TriangleCountResult {
    pub node_id: i64,
    pub user_id: String,
    pub triangles: usize,
    pub clustering_coefficient: f64,
}
```

`Error` Enum

```
pub enum Error {
    Sqlite(rusqlite::Error),
    Json(serde_json::Error),
    Cypher(String),
    ExtensionNotFound(String),
    TypeError(String),
    ColumnNotFound(String),
    GraphError(String),
}
```

Variant	Cause
`Sqlite`	SQLite or rusqlite error
`Json`	JSON serialization/deserialization failure
`Cypher`	Cypher parse or execution error
`ExtensionNotFound`	Could not locate the extension library
`TypeError`	Type mismatch when reading a column value
`ColumnNotFound`	Column name not present in result
`GraphError`	General graph operation failure

`Error` implements `std::error::Error` and `std::fmt::Display`.

Example

```
use graphqlite::{Connection, Graph};

fn main() -> graphqlite::Result<()> {
    let conn = Connection::open_in_memory()?;

    conn.cypher("CREATE (:Person {name: 'Alice', age: 30})")?;
    conn.cypher("CREATE (:Person {name: 'Bob', age: 25})")?;
    conn.cypher("MATCH (a:Person {name:'Alice'}), (b:Person {name:'Bob'})
CREATE (a)-[:KNOWS]->(b)")?;

    let result = conn.cypher("MATCH (n:Person) RETURN n.name, n.age ORDER BY
n.age")?;
    for row in &result {
        let name: String = row.get("n.name")?;
        let age: i64 = row.get("n.age")?;
        println!("{} is {}", name, age);
    }

    let graph = Graph::open_in_memory()?;
    graph.upsert_node("alice", &serde_json::json!({"age": 30}), "Person")?;
    let scores = graph.pagerank(0.85, 20)?;
    for r in &scores {
        println!("{}: {:.4}", r.user_id, r.score);
    }

    Ok(())
}
```

SQL Interface Reference

GraphQLite is a standard SQLite extension. Once loaded, it registers SQL scalar functions and creates the graph schema in the current database.

Loading the Extension

SQLite shell

```
.load ./libgraphqlite
SELECT graphqlite_test();
```

Python (manual)

```
import sqlite3
conn = sqlite3.connect(":memory:")
conn.enable_load_extension(True)
conn.load_extension("./libgraphqlite")
```

Python (via graphqlite)

```
import graphqlite
conn = graphqlite.connect(":memory:")
```

Rust

```
let conn = graphqlite::Connection::open_in_memory()?;
```

Entry point symbol: `sqlite3_graphqlite_init`

On initialization the extension:

1. Creates all schema tables (if not already present).
2. Creates all indexes.
3. Registers the SQL functions listed below.

Registered SQL Functions

cypher(query [, params_json])

```
SELECT cypher('MATCH (n:Person) RETURN n.name, n.age');
SELECT cypher('MATCH (n:Person) WHERE n.age > $min RETURN n.name', '{"min":
25}');
```

Arguments

Argument	Type	Required	Description
`query`	TEXT	Yes	Cypher query string
`params_json`	TEXT (JSON)	No	JSON object; keys map to `$name` placeholders

Returns: TEXT — a JSON array of objects. Each object represents one result row. Keys are the column names from the `RETURN` clause. A query with no results returns `[]`.

Result format

```
[
  {"n.name": "Alice", "n.age": 30},
  {"n.name": "Bob",   "n.age": 25}
]
```

For a single-column result the key is the expression text or alias from `RETURN`. For write queries with no `RETURN` clause, the result is a plain text status string such as `"Query executed successfully - nodes created: N, relationships created: M"`. The empty array `[]` is only returned when a `RETURN` clause produced zero matching rows.

Error handling: Sets SQLite error text and returns an error result on parse failure or execution failure.

---

cypher_validate(query)

```
SELECT cypher_validate('MATCH (n:Person) RETURN n.name');
```

Validates a Cypher query without executing it.

Returns: TEXT — a JSON object:

```
{"valid": true}
```

or

```
{"valid": false, "error": "...", "line": 1, "column": 15}
```

---

regexp(pattern, string)

```
SELECT regexp('^Al.*', 'Alice');   -- 1
SELECT regexp('^Al.*', 'Bob');     -- 0
```

POSIX extended regular expression (ERE) match. Used internally to implement the `=~` operator. Returns `1` if `string` matches `pattern`, `0` otherwise. The `(?i)` prefix enables case-insensitive matching.

Arguments

Argument	Type	Description
`pattern`	TEXT	POSIX extended regular expression (ERE)
`string`	TEXT	String to test

Returns: INTEGER (`1` or `0`)

---

gql_load_graph()

```
SELECT gql_load_graph();
```

Load the graph adjacency structure into an in-memory cache for algorithm execution. Must be called before running graph algorithm functions.

Returns: TEXT — JSON status object: `{"status": "loaded", "nodes": N, "edges": M}`. If the graph is already loaded, returns `{"status": "already_loaded", "nodes": N, "edges": M}` instead.

---

gql_unload_graph()

```
SELECT gql_unload_graph();
```

Release the in-memory adjacency cache.

Returns: TEXT — JSON status object: `{"status": "unloaded"}`

gql_reload_graph()

```
SELECT gql_reload_graph();
```

Unload and reload the cache. Use after bulk data changes to refresh the algorithm cache.

Returns: TEXT — JSON status object: `{"status": "reloaded", "nodes": N, "edges": M}`

---

gql_graph_loaded()

```
SELECT gql_graph_loaded();
```

Check whether the adjacency cache is currently loaded.

Returns: TEXT — JSON object: `{"loaded": true, "nodes": N, "edges": M}` if loaded, `{"loaded": false, "nodes": 0, "edges": 0}` if not.

---

graphqlite_test()

```
SELECT graphqlite_test();
```

Smoke-test function. Returns a success string if the extension is loaded and functioning.

Returns: TEXT — `"GraphQLite extension loaded successfully!"`

Query Patterns

Read and iterate rows in Python

```
import json, sqlite3, graphqlite

conn = graphqlite.connect("graph.db")
raw = conn.execute("SELECT cypher('MATCH (n:Person) RETURN n.name,
n.age')").fetchone()[0]
rows = json.loads(raw)
for row in rows:
    print(row["n.name"], row["n.age"])
```

Parameterized query via SQL

```
SELECT cypher(
  'MATCH (n:Person) WHERE n.age > $min RETURN n.name',
  json_object('min', 25)
);
```

Write query

```
SELECT cypher('CREATE (:Person {name: ''Alice'', age: 30})');
```

String literals inside Cypher must use single quotes. To embed a literal single quote in a SQL string, double it: `''`.

Transaction Behavior

- The `cypher()` function participates in the current SQLite transaction.
- Write operations (`CREATE`, `MERGE`, `SET`, `DELETE`, etc.) are not auto-committed; wrap in `BEGIN` / `COMMIT` for explicit control.
- `gql_load_graph()` reads a snapshot at call time; subsequent writes are not reflected until `gql_reload_graph()` is called.

Example

```
BEGIN;
SELECT cypher('CREATE (:Person {name: ''Alice''})');
SELECT cypher('CREATE (:Person {name: ''Bob''})');
COMMIT;
```

Direct Schema Access

The graph schema tables are ordinary SQLite tables. You can query them directly for inspection or integration.

```
-- Count nodes by label
SELECT label, count(*) FROM node_labels GROUP BY label;

-- List all property keys
SELECT key FROM property_keys ORDER BY key;

-- Find all text properties for node 1
SELECT pk.key, np.value
FROM node_props_text np
JOIN property_keys pk ON pk.id = np.key_id
WHERE np.node_id = 1;
```

Direct writes to schema tables bypass Cypher validation and the property key cache. Prefer `cypher()` for mutations.

Database Schema Reference

GraphQLite uses an Entity-Attribute-Value (EAV) schema stored in plain SQLite tables. All tables are created with `CREATE TABLE IF NOT EXISTS` during extension initialization, so they are safe to call multiple times.

Core Tables

nodes

Stores graph nodes. Each node has an auto-assigned integer primary key.

```
CREATE TABLE IF NOT EXISTS nodes (
  id INTEGER PRIMARY KEY AUTOINCREMENT
);
```

Column	Type	Description
`id`	INTEGER PK	Internal node identifier; auto-incremented

User-facing node IDs (strings) are stored as text properties and looked up by the higher-level API. The internal `id` is used in all join operations.

---

edges

Stores directed graph edges.

```
CREATE TABLE IF NOT EXISTS edges (
  id        INTEGER PRIMARY KEY AUTOINCREMENT,
  source_id INTEGER NOT NULL REFERENCES nodes(id) ON DELETE CASCADE,
  target_id INTEGER NOT NULL REFERENCES nodes(id) ON DELETE CASCADE,
  type      TEXT NOT NULL
);
```

Column	Type	Description
`id`	INTEGER PK	Internal edge identifier
`source_id`	INTEGER FK → `nodes.id`	Source node; cascades on delete
`target_id`	INTEGER FK → `nodes.id`	Target node; cascades on delete
`type`	TEXT	Relationship type (e.g. `"KNOWS"`)

node_labels

Maps nodes to their labels. A node may have multiple labels.

```
CREATE TABLE IF NOT EXISTS node_labels (
  node_id INTEGER NOT NULL REFERENCES nodes(id) ON DELETE CASCADE,
  label   TEXT NOT NULL,
  PRIMARY KEY (node_id, label)
);
```

Column	Type	Description
`node_id`	INTEGER FK → `nodes.id`	References `nodes.id`; cascades on delete
`label`	TEXT	Label string (e.g. `"Person"`)

The composite primary key `(node_id, label)` enforces uniqueness.

---

property_keys

Normalized lookup table for property key names. Shared by all node and edge property tables.

```
CREATE TABLE IF NOT EXISTS property_keys (
  id  INTEGER PRIMARY KEY AUTOINCREMENT,
  key TEXT UNIQUE NOT NULL
);
```

Column	Type	Description
`id`	INTEGER PK	Numeric key identifier
`key`	TEXT UNIQUE	Property name string (e.g. `"name"`, `"age"`)

All property value tables reference `property_keys.id` rather than storing key strings directly. An in-memory hash-map cache (`property_key_cache`) avoids repeated lookups during query execution.

Node Property Tables

One table per Cypher type. A property is stored in exactly one table, determined at write time by the value type.

node_props_int

```
CREATE TABLE IF NOT EXISTS node_props_int (
  node_id INTEGER NOT NULL REFERENCES nodes(id) ON DELETE CASCADE,
  key_id  INTEGER NOT NULL REFERENCES property_keys(id),
  value   INTEGER NOT NULL,
  PRIMARY KEY (node_id, key_id)
);
```

node_props_real

```
CREATE TABLE IF NOT EXISTS node_props_real (
  node_id INTEGER NOT NULL REFERENCES nodes(id) ON DELETE CASCADE,
  key_id  INTEGER NOT NULL REFERENCES property_keys(id),
  value   REAL NOT NULL,
  PRIMARY KEY (node_id, key_id)
);
```

node_props_text

```
CREATE TABLE IF NOT EXISTS node_props_text (
  node_id INTEGER NOT NULL REFERENCES nodes(id) ON DELETE CASCADE,
  key_id  INTEGER NOT NULL REFERENCES property_keys(id),
  value   TEXT NOT NULL,
  PRIMARY KEY (node_id, key_id)
);
```

node_props_bool

```
CREATE TABLE IF NOT EXISTS node_props_bool (
  node_id INTEGER NOT NULL REFERENCES nodes(id) ON DELETE CASCADE,
  key_id  INTEGER NOT NULL REFERENCES property_keys(id),
  value   INTEGER NOT NULL CHECK (value IN (0, 1)),
  PRIMARY KEY (node_id, key_id)
);
```

Stores `0` (false) or `1` (true). The `CHECK` constraint enforces this.

node_props_json

```
CREATE TABLE IF NOT EXISTS node_props_json (
  node_id INTEGER NOT NULL REFERENCES nodes(id) ON DELETE CASCADE,
  key_id  INTEGER NOT NULL REFERENCES property_keys(id),
  value   TEXT NOT NULL CHECK (json_valid(value)),
  PRIMARY KEY (node_id, key_id)
);
```

Stores JSON objects and arrays as text. The `CHECK` constraint enforces valid JSON via SQLite's `json_valid()`.

Edge Property Tables

Identical structure to node property tables, with `edge_id` replacing `node_id`.

edge_props_int

```
CREATE TABLE IF NOT EXISTS edge_props_int (
  edge_id INTEGER NOT NULL REFERENCES edges(id) ON DELETE CASCADE,
  key_id  INTEGER NOT NULL REFERENCES property_keys(id),
  value   INTEGER NOT NULL,
  PRIMARY KEY (edge_id, key_id)
);
```

edge_props_real

```
CREATE TABLE IF NOT EXISTS edge_props_real (
  edge_id INTEGER NOT NULL REFERENCES edges(id) ON DELETE CASCADE,
  key_id  INTEGER NOT NULL REFERENCES property_keys(id),
  value   REAL NOT NULL,
  PRIMARY KEY (edge_id, key_id)
);
```

edge_props_text

```
CREATE TABLE IF NOT EXISTS edge_props_text (
  edge_id INTEGER NOT NULL REFERENCES edges(id) ON DELETE CASCADE,
  key_id  INTEGER NOT NULL REFERENCES property_keys(id),
  value   TEXT NOT NULL,
  PRIMARY KEY (edge_id, key_id)
);
```

edge_props_bool

```
CREATE TABLE IF NOT EXISTS edge_props_bool (
  edge_id INTEGER NOT NULL REFERENCES edges(id) ON DELETE CASCADE,
  key_id  INTEGER NOT NULL REFERENCES property_keys(id),
  value   INTEGER NOT NULL CHECK (value IN (0, 1)),
  PRIMARY KEY (edge_id, key_id)
);
```

edge_props_json

```
CREATE TABLE IF NOT EXISTS edge_props_json (
  edge_id INTEGER NOT NULL REFERENCES edges(id) ON DELETE CASCADE,
  key_id  INTEGER NOT NULL REFERENCES property_keys(id),
  value   TEXT NOT NULL CHECK (json_valid(value)),
  PRIMARY KEY (edge_id, key_id)
);
```

Indexes

All indexes use `CREATE INDEX IF NOT EXISTS`.

Edge traversal indexes

```
CREATE INDEX IF NOT EXISTS idx_edges_source ON edges(source_id, type);
CREATE INDEX IF NOT EXISTS idx_edges_target ON edges(target_id, type);
CREATE INDEX IF NOT EXISTS idx_edges_type   ON edges(type);
```

- `idx_edges_source` : supports outgoing edge lookups and type-filtered traversals.
- `idx_edges_target` : supports incoming edge lookups and type-filtered traversals.
- `idx_edges_type` : supports edge type scans (e.g. `MATCH ()-[:TYPE]-()`).

Label index

```
CREATE INDEX IF NOT EXISTS idx_node_labels_label ON node_labels(label,
node_id);
```

Supports label-filtered `MATCH` patterns (e.g. `MATCH (n:Person)`).

Property key index

```
CREATE INDEX IF NOT EXISTS idx_property_keys_key ON property_keys(key);
```

Speeds up property key lookups by name when the in-memory cache is cold.

Node property indexes

```
CREATE INDEX IF NOT EXISTS idx_node_props_int_key_value  ON
node_props_int(key_id, value, node_id);
CREATE INDEX IF NOT EXISTS idx_node_props_text_key_value ON
node_props_text(key_id, value, node_id);
CREATE INDEX IF NOT EXISTS idx_node_props_real_key_value ON
node_props_real(key_id, value, node_id);
CREATE INDEX IF NOT EXISTS idx_node_props_bool_key_value ON
node_props_bool(key_id, value, node_id);
CREATE INDEX IF NOT EXISTS idx_node_props_json_key_value ON
node_props_json(key_id, node_id);
```

Cover index for `WHERE` predicates on node properties. The `(key_id, value, node_id)` order supports equality and range filters without a table scan.

The JSON index omits `value` (JSON columns are not range-indexed) but indexes `(key_id, node_id)` for existence checks.

Edge property indexes

```
CREATE INDEX IF NOT EXISTS idx_edge_props_int_key_value  ON
edge_props_int(key_id, value, edge_id);
CREATE INDEX IF NOT EXISTS idx_edge_props_text_key_value ON
edge_props_text(key_id, value, edge_id);
CREATE INDEX IF NOT EXISTS idx_edge_props_real_key_value ON
edge_props_real(key_id, value, edge_id);
CREATE INDEX IF NOT EXISTS idx_edge_props_bool_key_value ON
edge_props_bool(key_id, value, edge_id);
CREATE INDEX IF NOT EXISTS idx_edge_props_json_key_value ON
edge_props_json(key_id, edge_id);
```

Same structure as node property indexes.

Property Type Inference Rules

When a Cypher write operation stores a property value, the type is inferred from the value and determines which table receives the row.

Condition	Table
Value is a Cypher integer literal or Python `int`	`*_props_int`
Value is a Cypher float literal or Python `float`	`*_props_real`
Value is the string `'true'` or `'false'` (case-insensitive), or Python `bool`	`*_props_bool`
Value is a JSON object (`{…}`) or JSON array (`[…]`)	`*_props_json`
All other values	`*_props_text`

A property key may appear in only one type table per entity at a time. Updating a property with a different type removes the old row and inserts into the new table.

Property Key Cache

The `property_key_cache` is an in-process hash map (djb2 hash, chained buckets) that caches `property_key.id` lookups by key string. It is created per connection during `cypher_executor_create()` and freed when the connection closes. The cache avoids a `SELECT id FROM property_keys WHERE key = ?` round-trip for each property access during query execution.

Cascade Delete Behavior

All `REFERENCES nodes(id)` and `REFERENCES edges(id)` foreign keys include `ON DELETE CASCADE`. This means:

- Deleting a row from `nodes` automatically removes all rows in `node_labels`, `node_props_*`, and all `edges` that reference it.
- Deleting a row from `edges` automatically removes all rows in `edge_props_*`.

SQLite foreign key enforcement must be enabled: `PRAGMA foreign_keys = ON;` (GraphQLite enables this automatically at connection open).

Summary Table List

Table	Rows represent
`nodes`	Graph nodes

Table	Rows represent
`edges`	Graph edges (directed)
`node_labels`	Node-to-label assignments
`property_keys`	Property name registry
`node_props_int`	Integer node properties
`node_props_real`	Float node properties
`node_props_text`	String node properties
`node_props_bool`	Boolean node properties
`node_props_json`	JSON object/array node properties
`edge_props_int`	Integer edge properties
`edge_props_real`	Float edge properties
`edge_props_text`	String edge properties
`edge_props_bool`	Boolean edge properties
`edge_props_json`	JSON object/array edge properties

Architecture Overview

GraphQLite adds a Cypher query language interface to SQLite by functioning as a transpiler: it parses Cypher, translates it to SQL, and executes the resulting SQL against a set of tables that represent a property graph. Understanding this pipeline helps you reason about query behaviour, error messages, and performance.

Why a Transpiler, Not a Custom Engine

The most important architectural decision in GraphQLite is what it chose not to build: a dedicated graph storage engine or query runtime.

Building a purpose-built graph engine would require implementing disk layout, buffer management, query optimisation, transaction handling, concurrency control, and crash recovery. SQLite already provides all of this, and provides it correctly across a wide range of platforms, with a 35-year track record of reliability.

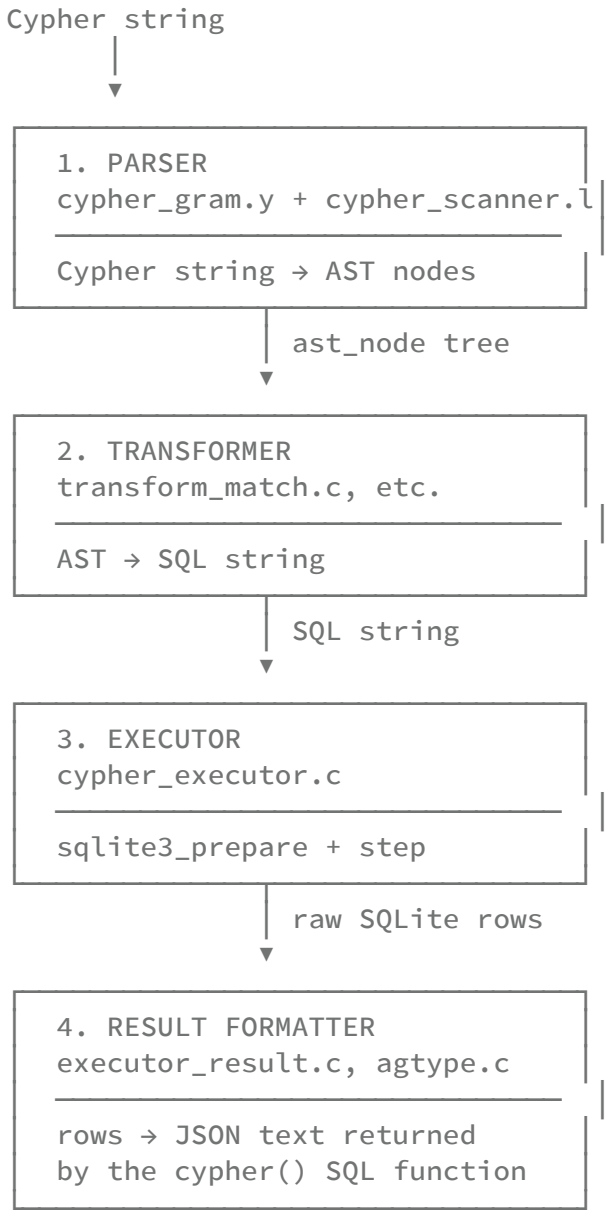
The transpiler approach means:

- **Durability and atomicity come for free.** Every write goes through SQLite's WAL and journalling machinery.
- **Standard tooling works.** The underlying tables are plain SQLite tables. You can inspect them with the SQLite CLI, use SQLite backup APIs, and attach the database to other tools.
- **Query execution is handled by a proven optimiser.** The generated SQL benefits from SQLite's query planner, covering indexes, and prepared statement caching.

The cost of this approach is translation overhead on every query, and the impedance mismatch between graph patterns and relational joins. Both are manageable: the translation is fast (typically under 1ms for simple queries), and the join structure is deterministic once you understand the EAV schema.

The Query Pipeline

A Cypher query passes through four stages before results are returned:



Stage 1: The Parser

The parser is a Bison GLR grammar (`cypher_gram.y`) with a Flex scanner (`cypher_scanner.l`). It produces a typed AST: `ast_node` structs that include `cypher_query` , `cypher_match` , `cypher_create` , `cypher_return` , `cypher_node_pattern` , `cypher_rel_pattern` , and expression types like `cypher_binary_op` , `cypher_property` , `cypher_identifier` , and `cypher_literal_*` .

Why GLR? Cypher has syntactic ambiguities that a standard LALR(1) parser cannot resolve. The most visible example is that `(n)` is simultaneously valid as a parenthesised expression and as a node pattern. GLR allows the parser to pursue both interpretations in parallel and resolve the ambiguity once more context is available. The grammar currently declares `%expect 4` shift/reduce conflicts and `%expect-rr 3` reduce/reduce conflicts — these are known, documented, and intentional.

Identifiers can be regular alphanumeric names or backtick-quoted names (`BQIDENT`), which the scanner strips to their bare text. The `END_P` keyword is also permitted as an identifier through the grammar's `identifier` rule, allowing queries like `MATCH (n) RETURN n.end` .

Error recovery is handled at this stage. When parsing fails, `parse_cypher_query_ext()` returns a `cypher_parse_result` with a populated `error_message` containing position information, which propagates back to the `cypher()` SQL function as a SQLite error.

Stage 2: The Transformer

The transformer walks the AST and emits SQL strings. It is not a general-purpose SQL generator: it knows the exact schema of GraphQLite's EAV tables and generates SQL specifically against those tables.

Key files and responsibilities:

File	Responsibility
`cypher_transform.c`	Entry point; creates transform context
`transform_match.c`	MATCH patterns → SQL FROM/JOIN/WHERE
`transform_return.c`	RETURN items → SQL SELECT list
`transform_expr_ops.c`	Expression operators and property access
`transform_create.c`	CREATE → INSERT INTO nodes/edges/properties
`transform_set.c`	SET → UPDATE on property tables
`transform_delete.c`	DELETE → DELETE FROM
`transform_variables.c`	Variable-to-alias tracking across clauses
`sql_builder.c`	Dynamic string buffer for SQL construction
`transform_func_*.c`	Function dispatch (string, math, path, etc.)

The transform context (`cypher_transform_context`) carries the SQL buffer being built, a variable context (`var_ctx`) that maps Cypher variable names to SQL table aliases, and flags like `in_comparison` that alter how property access is generated.

Concrete translation example. Consider:

```
MATCH (a:Person)-[:KNOWS]->(b)
WHERE a.name = 'Alice'
RETURN b.name
```

The transformer produces SQL roughly equivalent to:

```
SELECT
  (SELECT COALESCE(
    (SELECT npt.value FROM node_props_text npt
     JOIN property_keys pk ON npt.key_id = pk.id
     WHERE npt.node_id = n2.id AND pk.key = 'name'),
    (SELECT CAST(npi.value AS TEXT) FROM node_props_int npi
     JOIN property_keys pk ON npi.key_id = pk.id
     WHERE npi.node_id = n2.id AND pk.key = 'name'),
    ...
  )) AS "b.name"
FROM nodes n1
JOIN node_labels nl1 ON nl1.node_id = n1.id AND nl1.label = 'Person'
JOIN edges e1 ON e1.source_id = n1.id AND e1.type = 'KNOWS'
JOIN nodes n2 ON n2.id = e1.target_id
WHERE (SELECT COALESCE(
    (SELECT npt.value FROM node_props_text npt
     JOIN property_keys pk ON npt.key_id = pk.id
     WHERE npt.node_id = n1.id AND pk.key = 'name'),
    ...
  )) = 'Alice'
```

Each property access becomes a correlated subquery that fans out across all five typed property tables (`node_props_text` , `node_props_int` , `node_props_real` , `node_props_bool` , `node_props_json`) using `COALESCE` to return whichever type holds the value. In comparison contexts (WHERE clauses) the types are preserved natively; in RETURN contexts everything is cast to text.

Nested property access. For expressions like `n.metadata.city` — where `metadata` is stored as a JSON blob — the transformer generates `json_extract(n_metadata_subquery, '$.city')` , recursively building the extraction path.

Stage 3: The Executor

The executor orchestrates the full pipeline and manages the SQLite connection state. Its entry point is `cypher_executor_execute()` , which:

1. Calls `parse_cypher_query_ext()` to get the AST.
2. Calls `cypher_executor_execute_ast()` to dispatch on AST type.
3. For `AST_NODE_QUERY` and `AST_NODE_SINGLE_QUERY` , delegates to `dispatch_query_pattern()` .
4. `dispatch_query_pattern()` analyses the clause combination present in the query (MATCH, RETURN, CREATE, SET, DELETE, etc.) and selects the best-matching handler from the pattern registry.
5. The selected handler calls the appropriate transformer functions to produce SQL, then calls `sqlite3_prepare_v2()` and `sqlite3_step()` to execute it.
6. UNION queries bypass the pattern dispatcher and go directly through the transform layer, which handles them as a special `AST_NODE_UNION` case.

EXPLAIN mode. If the query starts with `EXPLAIN`, the executor runs the transformer but does not execute the SQL. Instead it returns a text result containing the matched pattern name, the clause flags, and the generated SQL string. This is useful for debugging unexpected behaviour.

Stage 4: Result Formatting

Raw SQLite column values are formatted into JSON by `executor_result.c` and `agtype.c`. The `cypher()` SQL function always returns a JSON array of row objects:

```
[{"b.name": "Bob"}, {"b.name": "Carol"}]
```

For rich graph objects (nodes and relationships returned as entities rather than scalar properties), the AGE-compatible `agtype` system serialises them with type annotations. Modification queries without a RETURN clause return a plain-text statistics string.

Extension Architecture

GraphQLite loads into SQLite as a shared library extension. The entry point `sqlite3_graphqlite_init()` registers several SQL functions on the current database connection.

Per-Connection Caching

The most important structural detail is the `connection_cache`:

```
typedef struct {
    sqlite3 *db;
    cypher_executor *executor;
    csr_graph *cached_graph;
} connection_cache;
```

This struct is allocated once per database connection and registered via `sqlite3_create_function`'s user-data pointer. It holds:

- A `cypher_executor` instance, which in turn holds the schema manager and the property key cache. Because executors are expensive to create (schema initialisation, prepared statement allocation), they are created on the first call to `cypher()` and reused for all subsequent calls on the same connection.
- A `csr_graph` pointer for the in-memory graph needed by algorithm functions. This is `NULL` until the user explicitly calls `gql_load_graph()`.

When the database connection closes, SQLite calls the destructor registered with the function, which frees both the executor and any cached graph.

Registered SQL Functions

The extension registers:

Function	Purpose
`cypher(query)`	Execute a Cypher query, return JSON
`cypher(query, params_json)`	Execute with parameters
`graphqlite_test()`	Health check
`gql_load_graph()`	Build CSR from current tables, cache it
`gql_unload_graph()`	Free cached CSR graph
`gql_reload_graph()`	Invalidate and rebuild CSR cache

Schema initialisation (`CREATE TABLE IF NOT EXISTS ...`) happens inside `cypher_executor_create()` , which is called on the first `cypher()` invocation. Extension loading takes approximately 5ms to complete this step.

Language Bindings

Both the Python and Rust bindings wrap the `cypher()` SQL function rather than linking directly against GraphQLite's C API.

Python. `Connection._load_extension()` calls `sqlite3.Connection.load_extension()` with the path to `graphqlite.dylib` (or `.so` / `.dll`). After loading, every `connection.cypher(query, params)` call issues `SELECT cypher(?, ?)` against the underlying `sqlite3.Connection` . The JSON result is parsed and returned as a `CypherResult` object (a list of dicts). This means Python adds one round-trip through `sqlite3_exec` but no C-level coupling beyond the SQLite extension API.

Rust. The Rust binding uses `rusqlite` and similarly loads the extension via `Connection::load_extension()` . Cypher queries are executed as `SELECT cypher(?)` statements. Higher-level helpers in `src/` (graph operations, algorithm wrappers) build Cypher strings and parse the JSON results. The `Graph` struct maintains an open `rusqlite::Connection` with the extension already loaded.

Graph Algorithm Integration

Graph algorithms (PageRank, Betweenness Centrality, Dijkstra, Louvain, etc.) operate on the CSR graph cache rather than on the EAV tables directly. The integration path is:

1. User calls `SELECT gql_load_graph()`. This reads all rows from `nodes` and `edges`, builds a CSR (Compressed Sparse Row) representation in heap memory, and stores it in `connection_cache.cached_graph`.
2. Each subsequent `cypher()` call syncs the `cached_graph` pointer into the current executor: `executor->cached_graph = cache->cached_graph`.
3. When the query dispatcher processes a RETURN-only query and finds a function name like `pageRank()` or `dijkstra()`, it dispatches to the graph algorithm subsystem instead of the normal SQL path.
4. The algorithm reads the CSR structure, runs its computation in C, and returns results as a JSON array, which is formatted and returned by the normal result formatter.

The CSR provides O(1) access to a node's neighbours (via the `row_ptr` and `col_idx` arrays), which is critical for iterative algorithms that traverse the graph many times. The EAV tables do not have this property — following an edge via SQL requires at minimum a B-tree lookup on `edges.source_id`.

If `gql_load_graph()` has not been called, algorithm functions will fail with an error indicating the graph is not loaded. After bulk inserts or other modifications, the cache must be explicitly refreshed with `gql_reload_graph()`.

Storage Model

GraphQLite stores property graphs in SQLite using an Entity-Attribute-Value (EAV) schema. This document explains why that design was chosen, how the tables are structured, and what the trade-offs look like in practice.

Why EAV?

A property graph has two requirements that are in tension with standard relational design:

1. **Schema flexibility.** Different nodes can have completely different properties. A `Person` node might have `name`, `age`, and `email`. A `Document` node might have `title`, `content`, and `created_at`. You cannot know the full set of property names at schema creation time.
2. **Type heterogeneity.** A property named `score` might be an integer on one node and a float on another. Cypher does not enforce types on property keys.

Three storage strategies are common:

Strategy	Approach	Problem
Fixed schema	One table per node type	Requires schema migration for every new property; hard to query across types
JSON blob	Single `properties TEXT` column	No index support on property values; comparisons require full-table scans
EAV	Separate row per property	Flexible schema, indexable values, but more joins per query

GraphQLite uses EAV. This trades query complexity (more JOIN operations per Cypher query) for schema flexibility and efficient indexed lookups on property values.

Table Structure

Core Tables

`nodes` is intentionally minimal:

```
CREATE TABLE nodes (
  id INTEGER PRIMARY KEY AUTOINCREMENT
);
```

A node is just an identity. All semantic content lives in the label and property tables. This allows the core table to stay compact and allows the autoincrement sequence to serve as a reliable surrogate key.

edges carries connectivity and type:

```
CREATE TABLE edges (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  source_id INTEGER NOT NULL REFERENCES nodes(id) ON DELETE CASCADE,
  target_id INTEGER NOT NULL REFERENCES nodes(id) ON DELETE CASCADE,
  type TEXT NOT NULL
);
```

The relationship type (`KNOWS` , `FOLLOWS` , `WORKS_AT` , etc.) is stored inline because it is always present and is the primary filter when traversing the graph. The `ON DELETE CASCADE` constraints mean deleting a node automatically removes all its incident edges without requiring explicit cleanup in Cypher.

node_labels is a many-to-many table between nodes and labels:

```
CREATE TABLE node_labels (
  node_id INTEGER NOT NULL REFERENCES nodes(id) ON DELETE CASCADE,
  label TEXT NOT NULL,
  PRIMARY KEY (node_id, label)
);
```

A node can carry multiple labels (e.g., `Person` and `Employee`). The composite primary key prevents duplicate labels and serves as the natural index for label lookups.

property_keys is a normalisation table:

```
CREATE TABLE property_keys (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  key TEXT UNIQUE NOT NULL
);
```

Rather than storing the property key string (e.g., `"name"`) directly in every property row, GraphQLite stores an integer `key_id` and looks up the string once. This reduces storage for graphs with many nodes sharing the same property names, and enables the property key cache described below.

Property Tables

There are ten property tables in total: five for nodes and five for edges. They follow the same pattern:

```
-- Node properties, integer values
CREATE TABLE node_props_int (
  node_id INTEGER NOT NULL REFERENCES nodes(id) ON DELETE CASCADE,
  key_id  INTEGER NOT NULL REFERENCES property_keys(id),
  value   INTEGER NOT NULL,
  PRIMARY KEY (node_id, key_id)
);

-- Node properties, text values
CREATE TABLE node_props_text (
  node_id INTEGER NOT NULL REFERENCES nodes(id) ON DELETE CASCADE,
  key_id  INTEGER NOT NULL REFERENCES property_keys(id),
  value   TEXT NOT NULL,
  PRIMARY KEY (node_id, key_id)
);

-- node_props_real, node_props_bool, node_props_json follow the same shape
-- edge_props_int, edge_props_text, edge_props_real, edge_props_bool,
edge_props_json likewise
```

Booleans are stored as `INTEGER CHECK (value IN (0, 1))` because SQLite has no native boolean type. JSON values are stored as `TEXT CHECK (json_valid(value))` — the constraint ensures the stored bytes are parseable JSON.

Why Separate Tables per Type?

The type-per-table design might seem verbose. Why not a single `node_props` table with a `type` discriminator column?

Efficient indexes. SQLite's B-tree indexes work best when a column contains values of a single type. An index on `node_props_int(key_id, value, node_id)` allows the query planner to use a range scan when evaluating `a.age > 30`. If integers and strings were mixed in one column, comparisons would degrade to text comparisons, silently changing semantics.

No type coercion surprises. SQLite's flexible type affinity means that storing `42` as text and later comparing it to the integer `42` would require careful `CAST`. Keeping types in separate tables makes the column's affinity unambiguous.

COALESCE fan-out. The transformer generates a `COALESCE(...)` that tries each type table in sequence and returns the first non-null result. This works correctly because a given

`(node_id, key_id)` pair can exist in at most one type table — a property cannot simultaneously be an integer and a string.

Property Type Inference

When Cypher writes a property, the type is inferred from the value:

Value	Table
Integer literal (`42`)	`node_props_int`
Float literal (`3.14`)	`node_props_real`
`true` / `false`	`node_props_bool`
JSON object or array	`node_props_json`
Everything else	`node_props_text`

Python's `bool` type is checked before `int` (since `bool` is a subclass of `int` in Python), so `True` goes to `_bool` rather than `_int`.

Property Key Cache

Looking up a key's integer ID in `property_keys` is necessary for every property read or write. Without caching, a simple `MATCH (n) RETURN n.name, n.age` would issue two `SELECT id FROM property_keys WHERE key = ?` queries per result row, which becomes expensive when returning thousands of rows.

The schema manager maintains a hash table of 1024 slots using the djb2 algorithm:

```
static unsigned long hash_string(const char *str) {
    unsigned long hash = 5381;
    int c;
    while ((c = *str++)) {
        hash = ((hash << 5) + hash) + c;  // hash * 33 + c
    }
    return hash;
}
```

Each slot holds a `property_key_entry` with the string and its integer ID. On a cache hit, no SQL is issued. On a miss, the key is looked up (or inserted) and the result is stored in the cache.

1024 slots is enough to cover most graphs without collision chains. A graph with 50 distinct property keys will have a load factor under 5%, meaning nearly every lookup resolves in O(1)

without a collision. The cache is per-executor, which means per-connection — multiple connections to the same database file each maintain their own independent cache.

How Property Access Translates to SQL

Consider `RETURN a.age` where `a` is a node. The full translation chain:

1. **Parser** produces an `AST_NODE_PROPERTY` node with `expr = identifier("a")` and `property_name = "age"`.
2. **`transform_property_access()`** checks whether the context is a comparison (`WHERE a.age > 30`) or a projection (`RETURN a.age`). This matters because comparisons need the native type, while projections cast everything to text for uniform JSON serialisation.
3. The transformer emits a correlated `SELECT COALESCE(...)` subquery that queries all five type tables:

```
(SELECT COALESCE(
  (SELECT npt.value
   FROM node_props_text npt
   JOIN property_keys pk ON npt.key_id = pk.id
   WHERE npt.node_id = a.id AND pk.key = 'age'),
  (SELECT CAST(npi.value AS TEXT)
   FROM node_props_int npi
   JOIN property_keys pk ON npi.key_id = pk.id
   WHERE npi.node_id = a.id AND pk.key = 'age'),
  (SELECT CAST(npr.value AS TEXT)
   FROM node_props_real npr
   JOIN property_keys pk ON npr.key_id = pk.id
   WHERE npr.node_id = a.id AND pk.key = 'age'),
  (SELECT CASE WHEN npb.value THEN 'true' ELSE 'false' END
   FROM node_props_bool npb
   JOIN property_keys pk ON npb.key_id = pk.id
   WHERE npb.node_id = a.id AND pk.key = 'age'),
  (SELECT npj.value
   FROM node_props_json npj
   JOIN property_keys pk ON npj.key_id = pk.id
   WHERE npj.node_id = a.id AND pk.key = 'age')
))
```

4. **SQLite executes** this subquery. Because the composite indexes on each property table include `(key_id, value, node_id)`, and `property_keys` has an index on `key`, the join between `property_keys` and the property table resolves via index lookup. SQLite evaluates the COALESCE branches lazily — once a branch returns a non-null value, the rest are skipped.

The result is that each property access is effectively two index lookups (one for the key ID, one for the value) in the common case.

JSON and Nested Property Storage

Properties whose values are JSON objects or arrays are stored in `node_props_json` (or `edge_props_json`). The `CHECK (json_valid(value))` constraint on those tables ensures that only valid JSON is stored.

Nested access — `n.metadata.city` — is handled at transform time. When `transform_property_access()` sees that the base of a property access is itself a property access (i.e., the AST has `AST_NODE_PROPERTY` nested inside another `AST_NODE_PROPERTY`), it generates a `json_extract()` call:

```
json_extract(
  (SELECT COALESCE(...) WHERE pk.key = 'metadata'),
  '$.city'
)
```

This means `metadata` is fetched from `node_props_json` as a JSON text value, and `json_extract` then navigates into it. Deeper nesting (`n.a.b.c`) produces nested `json_extract` calls.

String-keyed subscripts like `n['metadata']` are normalised at transform time to behave identically to `n.metadata`. The `AST_NODE_SUBSCRIPT` case in `transform_expression()` checks whether the key is a string literal and, if so, converts it to a property access before generating SQL.

Index Strategy

GraphQLite creates the following indexes at schema initialisation time:

Index	Columns	Purpose
`idx_edges_source`	`edges(source_id, type)`	Outgoing traversal with type filter
`idx_edges_target`	`edges(target_id, type)`	Incoming traversal with type filter
`idx_edges_type`	`edges(type)`	Full-graph type scans
`idx_node_labels_label`	`node_labels(label, node_id)`	Label-to-node lookup
`idx_property_keys_key`	`property_keys(key)`	Key name to ID lookup

Index	Columns	Purpose
`idx_node_props_int_key_value`	`node_props_int(key_id, value, node_id)`	Covered index for int property filters
`idx_node_props_text_key_value`	`node_props_text(key_id, value, node_id)`	Covered index for text property filters
`idx_node_props_real_key_value`	`node_props_real(key_id, value, node_id)`	Covered index for real property filters
`idx_node_props_bool_key_value`	`node_props_bool(key_id, value, node_id)`	Covered index for bool property filters
`idx_node_props_json_key_value`	`node_props_json(key_id, node_id)`	JSON key scans (value omitted; not comparable)
(same pattern for edge_props_*)		

The property indexes use a **covering index** pattern: `(key_id, value, node_id)`. When SQLite evaluates `WHERE a.age = 42`, it can satisfy the entire lookup from the index without touching the table heap — `key_id` filters to the right property, `value` satisfies the predicate, and `node_id` is the output needed to join back to the nodes table.

The JSON index omits the value because JSON blobs are not comparable as a unit; individual JSON paths are accessed via `json_extract()` at query time.

Trade-offs

Read performance. A simple `MATCH (n:Person) RETURN n.name` requires a label scan via `idx_node_labels_label` plus one correlated subquery per returned property per row. For small graphs (under 100K nodes), this is fast. As the result set grows, the correlated subqueries become the bottleneck. The optimizer cannot always lift them into a join, though covering indexes mitigate this significantly.

Write overhead. Creating a single node with three properties requires:

- 1 insert into `nodes`
- 1 insert into `node_labels`

- Up to 3 inserts into `property_keys` (or cache hits)
- 3 inserts into the appropriate `node_props_*` tables

That is 7 or more inserts per node. For bulk loading, the Python and Rust bindings provide `insert_nodes_bulk()` and `insert_edges_bulk()` methods that bypass the Cypher parser and use direct SQL within a single `BEGIN IMMEDIATE` transaction. This is 100–500x faster than issuing `CREATE` queries through `cypher()`.

Schema flexibility. Adding new properties to existing nodes requires no migration. A `Person` node created yesterday with `name` and `age` can have `email` added tomorrow with no schema change. This is a significant advantage for evolving data models.

Query complexity. The generated SQL for even simple Cypher queries is verbose. This makes the `EXPLAIN` prefix particularly valuable: `EXPLAIN MATCH (a)-[:KNOWS]->(b) RETURN b.name` returns the generated SQL so you can understand exactly what SQLite will execute.

Query Dispatch

When GraphQLite receives a Cypher query, the executor does not immediately start generating SQL. First it determines what kind of query it is — a MATCH+RETURN? A CREATE? A MATCH+SET? — and routes it to a handler that knows how to process that specific combination of clauses. This routing mechanism is the query dispatch system.

The Pattern Registry

The dispatch system maintains a static table of `query_pattern` entries. Each entry describes:

- A **name** (used in EXPLAIN output and debug logs).
- A bitmask of **required** clauses that must all be present.
- A bitmask of **forbidden** clauses that must all be absent.
- A **handler** function pointer.
- A **priority** integer used to break ties when multiple patterns match.

The full pattern table, ordered from highest to lowest priority:

Priority	Pattern Name	Required	Forbidden
100	`UNWIND+CREATE`	UNWIND, CREATE	RETURN, MATCH
100	`WITH+MATCH+RETURN`	WITH, MATCH, RETURN	—
100	`MATCH+CREATE+RETURN`	MATCH, CREATE, RETURN	—
90	`MATCH+SET`	MATCH, SET	—
90	`MATCH+DELETE`	MATCH, DELETE	—
90	`MATCH+REMOVE`	MATCH, REMOVE	—
90	`MATCH+MERGE`	MATCH, MERGE	—
90	`MATCH+CREATE`	MATCH, CREATE	RETURN
80	`OPTIONAL_MATCH+RETURN`	MATCH, OPTIONAL, RETURN	CREATE, SET, DELETE, MERGE
80	`MULTI_MATCH+RETURN`	MATCH, MULTI_MATCH, RETURN	CREATE, SET, DELETE, MERGE
70	`MATCH+RETURN`	MATCH, RETURN	OPTIONAL, MULTI_MATCH,

Priority	Pattern Name	Required	Forbidden
			CREATE, SET, DELETE, MERGE
60	`UNWIND+RETURN`	UNWIND, RETURN	CREATE
50	`CREATE`	CREATE	MATCH, UNWIND
50	`MERGE`	MERGE	MATCH
50	`SET`	SET	MATCH
50	`FOREACH`	FOREACH	—
40	`MATCH`	MATCH	RETURN, CREATE, SET, DELETE, MERGE, REMOVE
10	`RETURN`	RETURN	MATCH, UNWIND, WITH
0	`GENERIC`	—	—

The `GENERIC` pattern at priority 0 is the catch-all: it has no required or forbidden clauses, so it matches any query. It uses the full transform pipeline, which handles complex multi-clause queries including WITH chains.

How Pattern Matching Works

The dispatch function `dispatch_query_pattern()` follows three steps:

Step 1: Analyse. `analyze_query_clauses()` walks the clause list of the parsed query and sets bits in a `clause_flags` integer. Each clause type maps to one bit:

```
CLAUSE_MATCH       CLAUSE_RETURN      CLAUSE_CREATE
CLAUSE_MERGE       CLAUSE_SET         CLAUSE_DELETE
CLAUSE_REMOVE      CLAUSE_WITH        CLAUSE_UNWIND
CLAUSE_FOREACH     CLAUSE_LOAD_CSV    CLAUSE_EXPLAIN
CLAUSE_OPTIONAL    CLAUSE_MULTI_MATCH CLAUSE_UNION
CLAUSE_CALL
```

`CLAUSE_OPTIONAL` is set when any MATCH clause has `optional = true`.
`CLAUSE_MULTI_MATCH` is set when more than one MATCH clause is present.

Step 2: Find best match. `find_matching_pattern()` iterates the pattern table and evaluates each entry against the flags:

```
// Required clauses must all be present
if ((present & p->required) != p->required) continue;

// Forbidden clauses must all be absent
if (present & p->forbidden) continue;

// Higher priority wins
if (!best || p->priority > best->priority) best = p;
```

Step 3: Dispatch. The winning pattern's handler is called with the executor, query AST, result, and flags.

Priority Ordering Rationale

The priorities reflect increasing specificity:

Priority 100 entries are the most specific multi-clause combinations. A `WITH+MATCH+RETURN` query is unambiguous — it must use the generic transform pipeline which understands how WITH propagates variables into subsequent MATCH clauses. Putting it at the top prevents it from being matched by `MATCH+RETURN` (priority 70), which uses a simpler, more direct execution path that does not handle WITH.

Priority 90 covers the common write patterns: MATCH followed by SET, DELETE, REMOVE, MERGE, or CREATE. These all require first finding graph elements (via MATCH) and then modifying them. The forbidden clauses on `MATCH+CREATE` (priority 90) exclude RETURN, because `MATCH+CREATE+RETURN` has its own handler at priority 100 with different result-formatting behaviour.

Priority 80 handles OPTIONAL MATCH and multi-MATCH queries. These require the generic transform pipeline for correct LEFT JOIN generation. They are kept separate from the simpler `MATCH+RETURN` (priority 70) because the naive MATCH+RETURN handler assumes a single, non-optional MATCH clause.

Priority 70 is the hot path for the most common read query: `MATCH ... RETURN`. Its forbidden clause list is broad — it excludes OPTIONAL, MULTI_MATCH, and all write operations — ensuring only genuinely simple single-MATCH queries reach this handler.

Priority 50 covers standalone write operations. A bare `CREATE (n:Person)` or `MERGE` without a preceding MATCH is simpler than the MATCH+write variants; the handler can skip the join generation step.

Priority 10 covers standalone RETURN, which is how graph algorithms are invoked: `RETURN pageRank()`. The forbidden list excludes MATCH and WITH to prevent this from matching a `MATCH ... RETURN` accidentally.

Priority 0 is the GENERIC fallback. Any query that reaches this point is handled by the full transform pipeline, which is the most capable but least optimised path.

The GENERIC Fallback

The GENERIC handler creates a transform context and passes the entire query AST to `cypher_transform_generate_sql()`. The transform layer processes clauses sequentially — MATCH generates JOINs, WITH generates a subquery boundary, RETURN generates the SELECT list. This handles:

- WITH chains (`MATCH ... WITH ... MATCH ... RETURN`)
- OPTIONAL MATCH
- Multiple MATCH clauses
- UNWIND with complex logic
- Any combination not covered by a specific handler

The specific handlers at higher priorities exist as optimisations over GENERIC — they take shortcuts that are only valid for their specific clause combinations. If you add a new clause type or combination that GENERIC handles incorrectly, you add a new specific pattern rather than modifying GENERIC.

Multi-Clause Queries and WITH Chains

A query like:

```
MATCH (a:Person)
WITH a, count(*) AS c
WHERE c > 2
MATCH (a)-[:KNOWS]->(b)
RETURN a.name, b.name
```

contains clauses: MATCH, WITH, MATCH, RETURN. `analyze_query_clauses()` sets `CLAUSE_MATCH | CLAUSE_WITH | CLAUSE_RETURN | CLAUSE_MULTI_MATCH`. The pattern `WITH+MATCH+RETURN` (priority 100) matches because it requires MATCH, WITH, and RETURN with no forbidden clauses.

The GENERIC transform pipeline processes this by generating a subquery for the first MATCH+WITH block, then joining the second MATCH into that subquery's output. The WITH clause acts as a boundary: variables named in the WITH are projected out and remain available to subsequent clauses; variables not named are out of scope.

UNION Queries

UNION queries are handled outside the pattern dispatch system. When `cypher_executor_execute_ast()` receives an `AST_NODE_UNION` node, it passes it directly to the transform layer, bypassing `dispatch_query_pattern()` entirely. The transform layer handles UNION by generating `SELECT ... UNION ALL SELECT ...` (or `UNION` for UNION DISTINCT) SQL.

Algorithm Detection

The `RETURN`-only pattern (priority 10) handles standalone RETURN clauses. When the handler processes the RETURN items and finds a function call whose name is a known graph algorithm — `pageRank`, `dijkstra`, `betweenness`, `louvain`, etc. — it dispatches to the graph algorithm subsystem rather than generating SQL.

Detection happens by name in the RETURN handler. If the function name is registered as a graph algorithm, the executor checks whether a CSR graph is loaded (`executor->cached_graph != NULL`) and calls the appropriate algorithm function. If no graph is loaded, an error is returned indicating that `gql_load_graph()` must be called first.

This means graph algorithm calls look syntactically identical to scalar function calls:

```
RETURN pageRank()
RETURN dijkstra('alice', 'bob')
RETURN betweennessCentrality()
```

They are distinguished from SQL functions only inside the executor.

Adding New Patterns

To add a new execution path for a clause combination not currently covered:

1. **Identify the clause combination.** Determine which clauses must be present and which must be absent. Be careful not to create ambiguity with existing patterns.
2. **Write the handler.** Handler functions have the signature:

```
static int handle_my_pattern(
    cypher_executor *executor,
    cypher_query *query,
    cypher_result *result,
    clause_flags flags
);
```

The handler is responsible for setting `result->success` or calling `set_result_error()` on failure. It returns 0 on success and -1 on error.

3. **Add the pattern to the registry** in `query_dispatch.c`, choosing a priority that correctly orders it relative to existing patterns. The registry is scanned sequentially, with the highest-priority match winning, so placement within the array matters only for readability — the priority field determines the winner.

4. **Forward-declare the handler** in the declarations block at the top of `query_dispatch.c`.

5. **Test.** Use `EXPLAIN` to verify that the new pattern is selected for your target queries: `EXPLAIN MATCH (n) ... RETURN n` returns `Pattern: <matched_name>` in its output.

The `EXPLAIN` output also shows the clause flags string (e.g., `MATCH|RETURN|MULTI_MATCH`), which makes it easy to debug pattern selection issues.

Performance Characteristics

GraphQLite's performance profile is shaped by three factors: the overhead of the Cypher-to-SQL translation pipeline, the cost of the EAV schema's join-heavy queries, and the behaviour of the in-memory CSR cache for graph algorithms. Understanding these factors lets you make informed decisions about data loading, query structure, and when to reach for the bulk APIs.

Benchmark Reference Numbers

These figures come from a single-core MacBook workload with an in-memory SQLite database (`:memory:`). Disk-backed databases will be faster with WAL mode enabled and slower when the OS page cache is cold.

Operation	Typical latency
Extension loading (schema init)	~5ms (once per connection)
Simple `CREATE (:Person {name: 'Alice'})`	0.5–1ms
Simple `MATCH (n:Person) RETURN n.name` (10 nodes)	0.5–2ms
`MATCH (a)-[:KNOWS]->(b) RETURN b.name` (100 relationships)	1–5ms
Bulk insert via Python `insert_nodes_bulk()`	100–500x faster than Cypher CREATE
`gql_load_graph()` on 100K nodes/edges	~50–100ms
PageRank on 100K nodes	~180ms
PageRank on 1M nodes	~38s
Property key cache lookup (hit)	O(1), no SQL
Property key cache lookup (miss)	1 SQL roundtrip to `property_keys`

The 5ms extension loading cost is a one-time expense per connection. After the first `cypher()` call, the executor is cached and reused for all subsequent calls on the same connection.

The CSR Graph Cache

Graph algorithms (PageRank, Dijkstra, Betweenness Centrality, Louvain, etc.) cannot run efficiently against the EAV tables. Finding a node's neighbours requires a B-tree lookup on `edges.source_id`, and iterative algorithms like PageRank traverse the full graph hundreds of times. At 1M nodes that would be hundreds of millions of B-tree lookups.

The CSR (Compressed Sparse Row) representation solves this. After `SELECT gql_load_graph()`, GraphQLite:

1. Reads all rows from `nodes` to build the node ID array.
2. Builds a hash table mapping node IDs to CSR array indices.
3. Reads all rows from `edges` twice: first to count out-edges per node (to compute row pointer offsets), then to fill the column index arrays.
4. Builds a parallel in-edges structure for algorithms that need reverse traversal.

The result is two arrays (`row_ptr` and `col_idx`) where `row_ptr[i]` is the offset in `col_idx` where node `i`'s neighbours begin, and `row_ptr[i+1] - row_ptr[i]` is its degree. Neighbour access is O(1): `col_idx[row_ptr[i] .. row_ptr[i+1]]`.

When to Load

The cache must be loaded before running any algorithm, and must be refreshed after structural changes (adding or deleting nodes or edges). The cache persists for the lifetime of the connection; a stale cache causes algorithms to operate on the graph state at the time of the last load, silently ignoring newer data.

For cache management instructions — when and how to call `gql_load_graph()`, `gql_reload_graph()`, and `gql_unload_graph()` — see Using Graph Algorithms.

Memory Implications

The CSR graph holds two integer arrays of length `edge_count` (for `col_idx` and `in_col_idx`) and one array of length `node_count + 1` (for `row_ptr` and `in_row_ptr`). For a graph with N nodes and E edges:

- `row_ptr` arrays: 2 × (N+1) × 4 bytes ≈ 8N bytes
- `col_idx` arrays: 2 × E × 4 bytes ≈ 8E bytes
- Node ID array: N × 4 bytes ≈ 4N bytes
- Hash table: ~4 × N × 4 bytes ≈ 16N bytes (open addressing, load factor ~25%)

A graph with 1M nodes and 5M edges uses approximately 60MB of heap memory for the CSR structure. The user-defined ID strings (if present) add additional allocation per node.

If memory is constrained, `gql_unload_graph()` can be called after algorithm runs to free the CSR heap allocation. The trade-off is that the next algorithm call will require a full reload from the database tables. On a 1M-node graph, `gql_load_graph()` takes approximately 50–100ms, so unloading between algorithm calls is only worthwhile when memory pressure is severe.

Property Key Cache

Every property read or write needs the integer `key_id` for the property name. The property key cache uses djb2 hashing over 1024 slots, held in the `cypher_schema_manager` (which is per-executor, so per-connection).

A typical graph with 20–50 distinct property keys will have a cache load factor well under 10%, meaning:

- **Cache hit**: Hash the key string, index into the slot array, compare the stored string, return the `key_id`. Zero SQL.
- **Cache miss**: Hash lookup fails; issue `SELECT id FROM property_keys WHERE key = ?` (covered by `idx_property_keys_key`); store the result in the cache for future lookups.

Cache misses only occur for property keys not yet seen in this connection's session. After the first query that touches a given key, all subsequent accesses to that key on the same connection are cache hits.

The cache has no eviction policy — it grows monotonically, but with at most 1024 slots before collisions occur. For graphs with more than a few hundred distinct property key names, you may start seeing hash collisions. Collisions do not cause correctness problems (misses fall through to SQL), but they do degrade performance toward one SQL lookup per property access.

Bulk Insert

The most important performance optimisation available to users is bypassing the Cypher pipeline entirely for bulk data loading.

Issuing `SELECT cypher('CREATE (:Person {name: "Alice", age: 30})')` for each node in a large graph is slow because each call:

1. Parses the Cypher string (Bison GLR parse).
2. Transforms the AST to SQL insert statements.
3. Executes three or more SQL statements (insert nodes, insert label, insert properties).
4. Formats the result.

The Python `insert_nodes_bulk()` and `insert_edges_bulk()` methods skip steps 1 and 2 entirely and batch all of step 3 inside a single `BEGIN IMMEDIATE` transaction:

```
id_map = g.insert_nodes_bulk([
    ("alice", {"name": "Alice", "age": 30}, "Person"),
    ("bob",   {"name": "Bob",   "age": 25}, "Person"),
])
g.insert_edges_bulk([
    ("alice", "bob", {"since": 2020}, "KNOWS"),
], id_map)
```

The transaction amortises the cost of page writes across thousands of rows. The `id_map` dictionary eliminates the need for a `SELECT node_id FROM node_props_text WHERE value = ?` lookup per edge source and target.

Benchmarks show 100–500x throughput improvement for bulk loads compared to equivalent Cypher CREATE queries. For a graph with 100K nodes and 500K edges, bulk insert completes in seconds; Cypher CREATE would take minutes.

The Rust binding offers the equivalent `Graph::insert_nodes_bulk()` method with the same semantics.

Index Utilisation

SQLite's query planner makes decisions based on index statistics. For GraphQLite's EAV schema, the most important index patterns are:

Label filtering (`MATCH (n:Person)`): Uses `idx_node_labels_label` which is `(label, node_id)`. The query `WHERE label = 'Person'` is a single B-tree range scan that returns all node IDs with that label. This is the primary entry point for most read queries.

Property equality (`WHERE n.age = 30`): The generated SQL contains a correlated subquery that filters `node_props_int` with `key_id = ? AND value = 30`. The covering index `idx_node_props_int_key_value` on `(key_id, value, node_id)` allows this to be satisfied entirely from the index, returning the `node_id` without a table heap read.

Edge traversal (`MATCH (a)-[:KNOWS]->(b)`): Uses `idx_edges_source` on `(source_id, type)` for outgoing traversal. The type filter is folded into the index scan. Incoming traversal uses `idx_edges_target` on `(target_id, type)`.

When indexes are not used: Range predicates on JSON properties (e.g., `WHERE n.metadata.city = 'London'`) require evaluating `json_extract()` for every row that passes the outer filter. SQLite cannot use a B-tree index to accelerate `json_extract()` comparisons without a generated column or expression index.

SQLite-Specific Optimisations

WAL mode. For disk-backed databases with any level of concurrent access (even one writer, one reader), WAL mode allows readers to proceed while a writer is active. This is the most impactful single setting for mixed read/write workloads of Cypher queries.

Prepared statements. The `cypher_executor` caches the executor per connection. Within each query execution, the generated SQL is prepared and executed, but the prepared statement is finalised after each use because the generated SQL changes with each Cypher query. For repeated identical queries, this means the SQL planning cost is paid each time. If you are calling the same parameterised Cypher query many times (e.g., a lookup by ID), issuing the same query string with different parameter values — rather than constructing slightly different Cypher strings — allows SQLite's prepared statement cache to reuse the plan.

Page cache. SQLite's default page cache is 2MB (512 pages × 4KB). For graphs with many nodes, the EAV tables span many pages. A larger cache reduces I/O on repeated queries, with the trade-off of higher baseline memory use per connection.

Synchronous mode. Reducing the synchronous setting eliminates fsync calls and can roughly double write throughput for bulk loads. The trade-off is reduced durability: a crash during a write can leave the database in an inconsistent state. This setting is appropriate for analytics workloads on expendable data, but should never be used for production data without explicit acceptance of that risk.

For the specific PRAGMA values to use and when to apply each setting, see the how-to guides.

Scaling Characteristics

Under 10K nodes: Performance is dominated by connection overhead and query parsing. The EAV join pattern is fast because the tables fit in the SQLite page cache. Simple MATCH+RETURN queries complete in under 1ms.

10K–100K nodes: Property lookup correlated subqueries become noticeable. Queries that return many rows with many properties per row can take 5–50ms. The covering indexes keep most lookups out of the table heap, but the sheer number of subquery evaluations adds up. Bulk insert becomes worthwhile at this scale.

100K–1M nodes: The EAV fan-out is the dominant cost. A full-graph scan (no label filter, no index pushdown) requires visiting every row in the relevant label and property tables. Graph algorithms should always operate via the CSR cache at this scale, not via Cypher queries that generate SQL. PageRank on 100K nodes via CSR takes ~180ms; the equivalent SQL-based traversal would be orders of magnitude slower.

Above 1M nodes: Memory usage for the CSR cache becomes significant (60MB+ for 1M nodes, 5M edges). Disk-backed databases benefit strongly from WAL mode and a large page cache. PageRank on 1M nodes takes ~38s on a single core. For workloads at this scale, consider whether batching algorithm results, pre-computing centrality scores and storing them as properties, or partitioning the graph into sub-graphs makes sense for your use case.

Memory Usage Guidelines

Component	Approximate size
`cypher_executor` struct	~1KB (plus schema manager)
Property key cache (1024 slots)	~50KB empty, grows with distinct keys
CSR graph (N nodes, E edges)	~(20N + 8E) bytes
SQL buffer per query	1–50KB depending on query complexity
Result data	Proportional to row count × column count

For a graph with 100K nodes and 500K edges, the CSR cache uses approximately 6MB. The property key cache for a graph with 100 distinct property names uses approximately 100KB. The executor and schema manager overhead is negligible.